

A Lean 4 Certificate of Scott’s Lectures on a Mathematical Theory of Computation (1981) generated by LLM and registered by Palomar

Lars Ericson¹

¹ORCID: 0000-0001-8299-9361

¹Catskills Research Company

¹lars.ericson@catskillsresearch.com

September 12, 2026

Github: <https://github.com/catskillsresearch/scott1980>

Palomar Registration:

<https://palomar-registry.org/entry?id=PALOMAR-2026-08-24-000001&version=2>

Abstract

In November 1969, Dana Scott formulated a mathematical program to construct a non-degenerate, purely mathematical model (D_∞) for Alonzo Church’s untyped λ -calculus — the construction later published in *Continuous Lattices* (1972). That paper is often described as giving the first such model; this formalization does not adjudicate historical priority. Scott’s 1972 framework relied on point-set topology, which remained a barrier for computer scientists seeking a practical tool for programming-language semantics.

When Scott delivered his Oxford lectures in Michaelmas Term 1980 — published May 1981 as *Lectures on a Mathematical Theory of Computation* (Technical Monograph PRG-19) — he pivoted to discrete information presented as *domains*. That presentation was intended to be more accessible without training in point-set topology and lattice theory.

This Lean 4 library formalizes the numbered definitions, theorems, examples, and exercises of PRG-19, with one documented deferral: Exercise 8.17 Part 2 (whether $\mathcal{U} + \mathcal{U}$, $\mathcal{U} \times \mathcal{U}$, or $\mathcal{U} \rightarrow \mathcal{U}$ is isomorphic to $\mathcal{U}$), which Scott himself left unresolved for the function-space case. Data constructions are kept choice-free where possible. Prop-level results may use `Classical.choice`; axiom audits record the footprint. Palomar Comparator selects the principal sourced theorem `theorem_8_8` and two new formal corollaries, `P4_embeds` and `Ssys_embeds`; their permitted footprint is `{propext, Classical.choice, Quot.sound}`.

Proofs in this note are summarized at the level of their main constructions. Short representative Lean fragments are copied from the library. The Appendix gives a short description of each library file and a link to that file in the Palomar-preserved copy of this repository.

0.0.1 Palomar compared claim (first sentence of Theorem 8.8)

The principal sourced result is **only the first sentence of Theorem 8.8**: every countable neighbourhood system $\mathcal{D}$ embeds as a subdomain of $\mathcal{U}$ ($\mathcal{D} \trianglelefteq \mathcal{U}$, Lean `theorem_8_8`). Comparator additionally selects two new formal corollaries, not separately stated by Scott: specializing the theorem to Example 1.5 gives `P4_embeds`, and specializing it to Exercise 7.22’s system S gives `Ssys_embeds`. Scott’s effective projection-pair sentence and finitary-projection correspondence are proved in the library as `theorem_8_8_b` / `theorem_8_8_c` but are **not** Comparator targets. Scott assumes from the outset that the master token set Δ is non-empty; `NeighborhoodSystem.master_nonempty` now enforces that assumption structurally. See `Challenge.lean` and `formalization.yaml`.

1 Introduction

To make domain theory accessible, the 1980 monograph introduces three key conceptual and structural shifts:

1.1 The Information-Theoretic Ordering

In contrast to the topological open sets of 1972, the 1980 lectures treat domains strictly as partially ordered sets (posets) representing states of incomplete information. An element within a domain is framed as a “partial description” of a computation. The ordering relation ($\sqsubseteq$) is explicitly interpreted as approximation: $x \sqsubseteq y$ means x contains less information than, or approximates, y .

1.2 Neighbourhood Systems and Finite Approximations

To bypass the complexities of continuous geometric spaces, Scott introduced **Neighbourhood Systems**. He recognized that real-world computing machines only ever interact with finite, checkable tokens of data. In this framework, an infinite computational process (such as an infinite stream or a complex recursive function) is defined as the limit of an ever-tightening sequence of these finite neighbourhoods. This shifted the underlying mathematics away from general topology and toward formal logic and order theory.

1.3 Solving Universal Recursive Domain Equations

While Scott’s 1969 discovery was a specialized solution to the specific self-referential equation $D \cong [D \rightarrow D]$, the 1980 monograph provides a universal factory blueprint. Scott uses inverse limits over Directed-Complete Partial Orders (CPOs) to solve arbitrary recursive domain equations. This generalized framework allowed computer scientists to give rigorous mathematical meaning to standard recursive computer data structures, such as lists, trees, and stream types.

1.4 Formalization Target: Consolidating “Scott Domains”

This Lean 4 artifact formalizes the mathematical objects that these 1980 lectures ultimately standardized for the computer science community, known today as **Scott Domains**. A Scott Domain is characterized as a poset that is: 1. **Directed-Complete (CPO)**: Every directed subset has a least upper bound, ensuring that infinite computations have well-defined limits. 2. **ω -algebraic**: Every element in the domain can be represented as the supremum of a countable set of compact (finite) elements, mirroring how infinite data is built from finite tokens. 3. **Consistently Complete**: If any two pieces of information do not outright contradict each other, they possess a join (least upper bound), allowing consistent computation streams to merge safely.

1.5 A caveat: topology gives way to recursion-theoretic bookkeeping, not to simplicity

Scott’s 1980 pivot (§1–3 above) trades point-set topology for order theory and finite tokens, and for most of the monograph that trade genuinely simplifies the formalization: neighbourhood systems, approximable maps, and the domain-equation solutions of Lectures I–VI are combinatorial objects with comparatively little topological machinery behind them. But it would be a mistake to read this as “the 1980 lectures are elementary throughout.” From Lecture VII onward, Scott’s subject is **computability in domains** — effectively given presentations, computable elements, computable maps — and this content is intrinsically recursion theory, not topology. Mechanizing it honestly means mechanizing recursion theory, and that turns out to be the more expensive

half of this formalization by a wide margin, for a reason specific to this project’s foundational choices rather than to Scott’s mathematics:

Mathlib’s own recursion theory (`Nat.Primrec`, `Primrec`, `Partrec`, `ComputablePred`) is unusable here because its correctness lemmas are proved via tactics (`grind`, `lia`, the `Nat.unpair_pair` simp set) that route through `Classical.choice`, which would silently taint every downstream computability result in this project with an axiom this formalization otherwise avoids (see the choice discipline below). The practical consequence is that `Recursive.lean` — a *from-scratch*, choice-free recursion theory (primitive-recursive `Nat.pair/unpair` round-trips, `Nat.sqrt` correctness, recursively-decidable predicates and their closure lemmas, course-of-values recursion via fuel-indexed tables) — is now **4,000+ lines**, roughly **9× the size of `Basic.lean`** (450 lines), the file containing the actual domain-theoretic definitions (neighbourhood systems, elements, subsystems) that this whole formalization is nominally *about*. A representative data point: Exercise 8.15’s “computable elements correspond to effectively presented domains” half needed perhaps 30–40 lines of genuine neighbourhood-system reasoning (an intersection dichotomy, a filter-closure invariant), sitting on top of 250+ lines encoding finite binary trees as `Nat.pair`-coded naturals and proving their evaluation is primitive recursive — a fact any computability textbook states in one sentence, made expensive here purely because Lean’s `Nat.pair/Nat.unpair` are not kernel-reducible and Mathlib’s off-the-shelf apparatus is off-limits by this project’s own discipline.

In short: the 1980 lectures did succeed in retiring topology as the dominant technical overhead, but Lecture VII onward replaces it with a different overhead — recursion-theoretic encoding and course-of-values bookkeeping — that is, in this formalization, actually the larger of the two by line count. This is a cost of the choice-free discipline adopted below, not a property of Scott’s mathematics; a formalization willing to inherit `Classical.choice` through Mathlib’s recursion theory would find Lecture VII’s exercises considerably shorter.

1.6 Chapter inclusion hierarchy (Lean imports)

Each node is a lecture (Scott chapter) at the point its Lean modules are first introduced. An edge $A \rightarrow B$ means some module belonging to lecture B has a direct `import` of a module belonging to lecture A (transitive imports through intermediate lectures are *not* drawn). Edge labels count those direct import sites. This is the dependency picture relevant to splitting the monograph into separate arXiv papers.

2 Methodology

This section records the proof-engineering conventions of the formalization—the parts of the development workflow that are of general academic interest, distilled from the project’s internal handoff notes.

2.1 Source material and inventory

The primary source is Dana Scott’s *Lectures on a Mathematical Theory of Computation* (Oxford, 1980; Technical Report PRG-19). OCR transcriptions live in `sources/PRG19_vision.md`; the structured inventory of every numbered Definition, Theorem, Example, and Exercise—with formalization status and proof notes—is maintained in this document (`arxiv.md`). Each item is keyed to Scott’s original numbering and cross-linked to its Lean module. Status values distinguish **Pass** (mechanized, builds green, zero `sorry`). Remaining mismatches versus the monograph are documented deferrals or expository items (Exercise 8.17 Part 2; Exercise 8.15 item

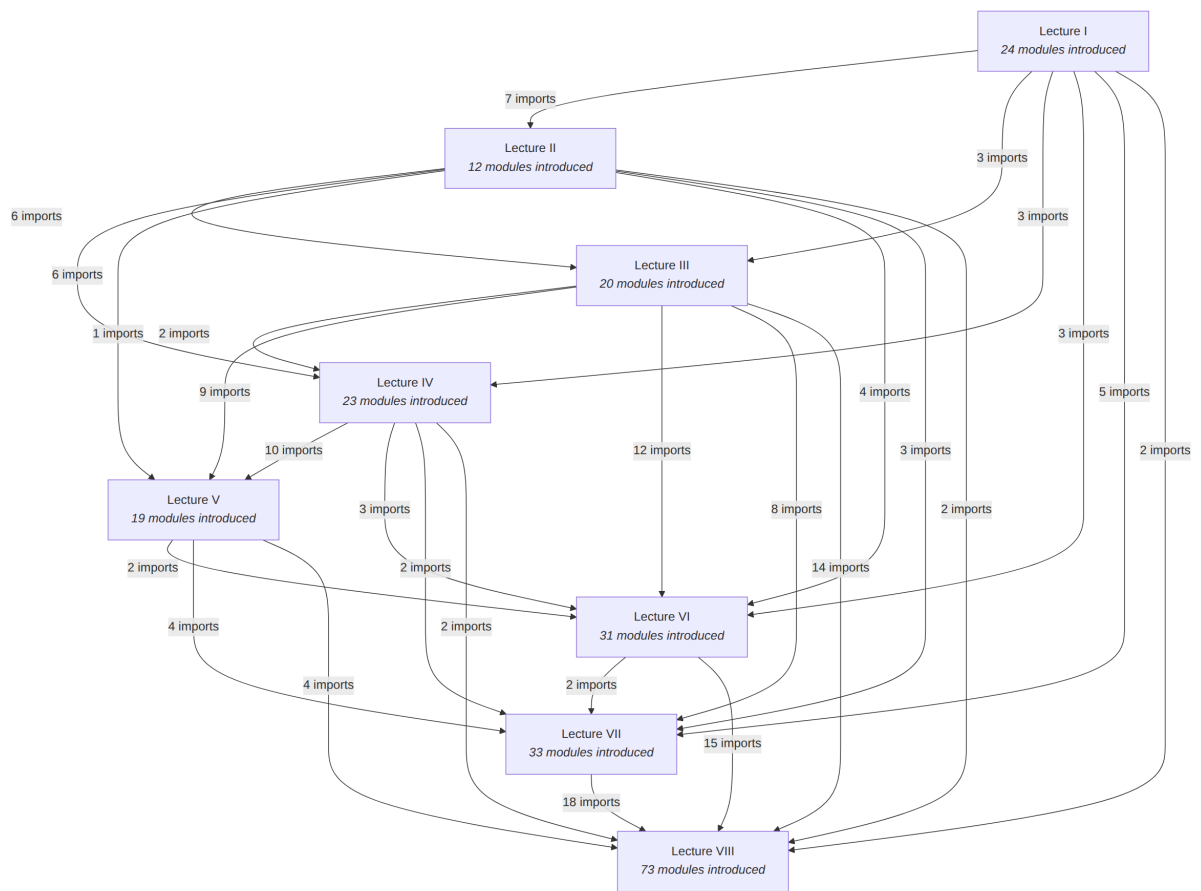


Figure 1: Chapter inclusion hierarchy (Lean imports).

6g; pictures / “discuss” prompts), not unfinished numbered proofs. The older **Partial** / **Not Yet** / **Deferred** legend described an earlier inventory state.

2.2 Neighbourhood systems as the uniform substrate

Following Scott’s 1980 pivot away from point-set topology, domains are encoded uniformly as **neighbourhood systems**: a master set Δ , a family $\mathcal{D}$ of neighbourhoods (filters on Δ), and domain elements as filters over $\mathcal{D}$. Approximable maps, products, function spaces, sums, and fixed-point combinators are built on this substrate in `Basic.lean`, `Approximable.lean`, `Product.lean`, and `FunctionSpace.lean`. Positive systems (Exercise 1.19) and effectively given presentations (Definition 7.1) are layered on top when Scott’s exercises demand computability content.

2.3 Custom recursion theory (Lecture VII)

For **effectively given** domains Scott requires two index relations to be *recursively decidable*: (i) intersection equality $X_n \cap X_m = X_k$, and (ii) consistency $\exists k. X_k \subseteq X_n \cap X_m$. Rather than mathlib’s `Computable/ComputablePred` development—which pulls `Classical.choice` through tactics such as `grind`, `lia`, and `Nat.unpair_pair`—we rebuilt the needed slice in `Recursive.lean`:

- `RecDecidable p :=  $\exists f, \text{Nat.Primrec } f \wedge \forall n, p\ n \leftrightarrow f\ n = 1$` (and the binary/ternary pair-codings `RecDecidable2`, `RecDecidable3`);
- choice-free correctness for `Nat.sqrt`, `Nat.pair/unpair`, and primitive-recursive `+/*`;
- closure lemmas (`RecDecidable.of_iff`, `.comp`, `.and`, `.or`, `.not`, bounded $\forall/\exists$ via `bForallFn/bExistsFn`);
- r.e. layers `REPred/REPred2` as projections of decidable relations.

Target axiom footprint for data constructions and core proofs: $\subseteq \{\text{propext}, \text{Quot.sound}\}$. `Classical.choice` is permitted only for genuinely unavoidable **Prop-level** steps (e.g. classical case splits on membership in an arbitrary system) and is always called out in proof notes. Each completed module is audited with `#print axioms`.

2.4 Incremental proof development

Large exercises are decomposed into small, revert-safe sessions rather than monolithic proofs. The inventory’s granularity — how far a single numbered item is split into lettered or numbered sub-goals before being discharged — was set by hand, session by session, rather than by a fixed rule. Some exercises (Exercise 7.22’s fifteen-plus sub-items above are the extreme case) were deliberately drilled down into small, independently provable pieces *before* any Lean was written, because the informal statement was large enough that a single monolithic proof attempt would have wasted effort on a failing whole-proof search. Others were left as a single inventory row even though the underlying Lean proof has considerable internal structure, because the statement was small enough that further decomposition would only add bookkeeping without changing how the proof was actually attacked. The deciding factors were pragmatic and varied by session: the size and combinatorial shape of the informal statement, the point at which further upfront planning showed diminishing returns, and how much compute budget seemed prudent to spend on decomposition versus a direct proof attempt. Consequently, the depth of decomposition visible in the inventory does not track a formal complexity metric of Scott’s original exercises; it is an artefact of session-by-session engineering judgment and should be read as such, not as a claim about which parts of PRG-19 are intrinsically harder to formalize.

3 Chronological Formalization Narrative

Below is the chronological narrative of the formalization, organized step-by-step using Dana Scott’s original numbering system from the PRG-19 monograph.

3.1 Lecture I: Domains by Neighbourhoods

Definition 1.1

A neighbourhood system is a membership predicate on sets together with Scott’s master neighbourhood Δ and the two closure conditions. Lean keeps Δ as a field (`master`) rather than hard-wiring `Set.univ`, and records that Δ is non-empty (`master_nonempty`). Finite intersection is the recursive convention `interUpTo` ($0 \setminus \text{mapsto} \Delta$, successor intersects the next set).

Lean 4 source

```
structure NeighborhoodSystem (a : Type*) where
  mem : Set a -> Prop
  master : Set a
  master_nonempty : master.Nonempty
  master_mem : mem master
  inter_mem : forall {X Y Z : Set a}, mem X -> mem Y -> mem Z ->
    Z subseq X I Y -> mem (X I Y)
  sub_master : forall {X : Set a}, mem X -> X subseq master
```

Factoid 1.1a

`interUpTo`, `interUpTo_zero` ($\bigcap_{i < 0} X_i = \Delta$).

Factoid 1.1b

`interUpTo_succ` ($\bigcap_{i < n+1} X_i = (\bigcap_{i < n} X_i) \cap X_n$).

Theorem 1.1c

`interUpTo_mem` (extend (ii) to finite seqs) + `consistent_iff_interUpTo_mem` (consistency $\iff \bigcap \in \mathcal{D}$); aux `Consistent`, `interUpTo_subset`.

Example 1.2

Scott’s first worked example: $\Delta = \{0,1\}$ (`Token := Fin 2`, `master := Set.univ`), $\mathcal{D} = \{\Delta, \{0\}, \{1\}\}$. We build `neighborhoodSystem : NeighborhoodSystem Token` — the only real obligation is condition (ii), discharged by `inter_eq` (the nine pairwise intersections each reduce to $\Delta, \{0\}, \{1\}$, or $\emptyset$ via `master_inter/inter_master/Set.inter_self/zero_inter_one`), the $\emptyset$ case being impossible since a witness $Z \subseteq \emptyset$ would force $\emptyset \in \mathcal{D}$ (`not_mem_empty`).

The mathematical payoff is the **element classification** (`element_classification`): every filter is one of exactly three — `bot` = $\{\Delta\}$, `elemZero` = $\{\Delta, \{0\}\}$, `elemOne` = $\{\Delta, \{1\}\}$. The argument: a filter x either contains $\{0\}$ (then `up_mem+inter_mem` force $x = \text{elemZero}$; it cannot also contain $\{1\}$ since $\{0\} \cap \{1\} = \emptyset \notin \mathcal{D}$), or $\{1\}$ (symmetric), or neither (then $x = \text{bot}$). Hence `bot_is_unique_partial`: $\perp$ is the sole *partial* element, with `bot_lt_elemZero`, `bot_lt_elemOne` placing the two total elements strictly above it — exactly Scott’s “there is only one partial element”. Being a concrete finite computation it leans on `Mathlib.Tactic (fin_cases/simp)`, so its footprint is the classical `[propext, Classical.choice, Quot.sound]`; the constructive guarantee is reserved for the §1 *core* in `Basic.lean`.

Example 1.3

Scott’s second worked example: $\Delta = \{0,1,2\}$ (`Token := Fin 3`, `master := Set.univ`), $\mathcal{D} = \{\Delta, \{1,2\}, \{2\}\}$ — a **linear chain** under reverse inclusion (more information = smaller set). We build `neighborhoodSystem : NeighborhoodSystem Token`; condition (ii) is discharged by `inter_eq` with only **three** outcomes ($\Delta, \{1,2\}, \{2\}$) — every pairwise intersection is nested, so there is no empty-intersection case (contrast Example 1.2’s nine-case analysis).

The element classification (`element_classification`) yields exactly three filters in a linear chain: `bot` = $\{\Delta\}$, `elemTwelve` = $\{\Delta, \{1,2\}\}$, `elemTwo` = $\{\Delta, \{1,2\}, \{2\}\}$. The argument fol-

lows the same “case on minimal non-master neighbourhood” pattern as 1.2: if $\{2\} \in x$ then $x = \text{elemTwo}$; else if $\{1,2\} \in x$ then $x = \text{elemTwelve}$; else $x = \text{bot}$. Order lemmas `bot_lt_elemTwelve`, `elemTwelve_lt_elemTwo`, and `elemTwo_maximal` capture Scott’s narrative: approximation proceeds in **two steps** to the total element (token 2); tokens 0 and 1 are not total (they appear in larger neighbourhoods but do not determine filters); the direction of approximation is **unique** (no branching). Unlike 1.2 (one partial, two total), 1.3 has **two partial** elements and **one total**.

Example 1.4

Scott’s third worked example and the first with **branching**: the depth-2 binary tree $\Delta = \{\Delta, 0, 1, 00, 01, 10, 11\}$ (Token := Fin 7, with $\Delta=0, \dots, 11=6$), neighbourhoods the subtrees $\mathcal{D} = \{\Delta, \text{left}=\{0,00,01\}, \text{right}=\{1,10,11\}, \{00\}, \{01\}, \{10\}, \{11\}\}$ — encoded as `left={1,3,4}`, `right={2,5,6}`, and the four leaf singletons. Condition (ii) reduces to the “nested-or-disjoint” table: of the 49 pairwise intersections, each is again a neighbourhood or $\emptyset$. Rather than search, `inter_eq` rewrites $X \cap Y$ to its canonical value via a complete `simp only` set of the 24 distinct intersection lemmas (both orders) plus `master_inter/inter_master/Set.inter_self`, so the matching disjunct closes by `rfl` — deterministic and fast (the naive 49×8 first ladder times out). The $\emptyset$ outcomes are inadmissible in `inter_mem` because a witness $Z \subseteq \emptyset$ would force $\emptyset \in \mathcal{D}$ (`not_mem_empty`).

The payoff is the **seven-filter classification** (`element_classification`): the bottom $\perp = \{\Delta\}$, two branch partials `elemZero={Δ, left}` / `elemOne={Δ, right}`, and four total leaf filters `elem00, ..., elem11`. The proof cases on the minimal non-master neighbourhood: a leaf in x pins the total filter (`mem_leafXY_imp`, using that distinct leaves and cross-branch neighbourhoods intersect to $\emptyset$); otherwise `left/right` membership gives a branch partial, else $\perp$. The order lemmas realize the **tree with choice**: `bot_lt_elemZero/elemOne` (two incomparable partials above $\perp$), `elemZero_lt_elem00/01`, `elemOne_lt_elem10/11` (each partial below its two leaves), and `elemXY_maximal` for the four leaves (each leaf filter is maximal — a total element). Contrast the prior examples: 1.2 is a fork at the bottom (one partial, two total), 1.3 a linear chain (two partial, one total), and 1.4 a genuine tree (three partial, four total) where branching encodes the choice in extending a partial sequence.

Factoid 1.4a

`NestedOrDisjoint + NeighborhoodSystem.ofNestedOrDisjoint`: “*nested-or-disjoint*” $\implies$ neighbourhood system (the “very special circumstance” of 1.2–1.4); choice-free.

Example 1.5

$\Delta = \{0, 1, 2, 3\}$ (Token := Fin 4) with $\mathcal{D} =$ all **non-empty** subsets (`P4Mem X := X.Nonempty`, `master := Set.univ`). Condition (ii) is immediate and choice-free: a non-empty witness $Z \subseteq X \cap Y$ makes $X \cap Y$ non-empty (`P4_inter_mem`). **Factoid 1.5a** (`consistent_iff_inter_nonempty`) is Scott’s remark that “sets are consistent iff they have a non-empty intersection”: reusing the `Basic Consistent/interUpTo` infrastructure, a prefix is consistent ($\exists Z, Z.\text{Nonempty} \wedge Z \subseteq \bigcap$) iff $\bigcap_{i < n} X_i$ is non-empty ($\rightarrow$ shrinks the witness, $\leftarrow$ takes the intersection as its own witness). Notably this example needs **no fin_cases/decide**. This is a constructive contrast to the finite Examples 1.2–1.4. Palomar compares `P4_embeds : neighborhoodSystem  $\leq$  U`.

Factoid 1.5a

in 1.5: `consistent_iff_inter_nonempty` (`consistent  $\iff$  non-empty intersection`); $\mathcal{D}$ is a system.

Factoid 1.5b

`limitFamily, SeqEquiv, limitFamily_eq_iff`: limit-family $x = \{Z \in \mathcal{D} \mid \exists n, X_n \subseteq Z\}$ equal $\iff$ sequences equivalent; choice-free.

Definition 1.6

Element V is Scott’s filter (Def 1.6): a membership predicate `mem : Set  $\alpha \rightarrow$  Prop` with `sub` ($x \subseteq \mathcal{D}$), `master_mem` ($\Delta \in x$), `inter_mem` (closed under $\cap$), and `up_mem` (upward closed in $\mathcal{D}$). Mirroring `InfoSys.Element`, the early helper `Element.ext` (membership-equality $\implies$

equality, proved by `rcases` on both structures + `funext/propext`, *not congr*) keeps the `PartialOrder` instance (Def 1.8's approximation order $x \sqsubseteq y \iff x \subseteq y$) choice-free: `le_antisymm` is just `Element.ext fun X => ⟨h1 X, h2 X⟩`.

Definition 1.7

Scott's *principal filter* $\uparrow X = \{Y \in \mathcal{D} \mid X \subseteq Y\}$ is `principal (hX : V.mem X) : V.Element`, with `mem Y := V.mem Y ∧ X ⊆ Y`. The four filter laws: `sub` is the first projection; `master_mem` = $\langle V.master_mem, V.sub_master\ hX \rangle$ (this is where the new `sub_master` field earns its keep — $X \subseteq \Delta$); `inter_mem` combines `Set.subset_inter` (from $X \subseteq Y_1, X \subseteq Y_2$) with one use of `V.inter_mem`, taking X itself as the consistency witness $X \subseteq Y_1 \cap Y_2$; `up_mem` is $\subseteq$ transitivity. `mem_principal` is the membership `rfl`-unfolding.

Factoid 1.7a (one-one + inclusion-reversing). `principal_le_iff`: $\uparrow X \sqsubseteq \uparrow Y \iff Y \subseteq X$ — Scott's $X \subseteq Y \iff \uparrow Y \sqsubseteq \uparrow X$, the **variance flip** (smaller neighbourhood $\Rightarrow$ larger principal filter $\Rightarrow$ more information). $\rightarrow$ evaluates $\sqsubseteq$ at the token X (using $X \in \uparrow X$ since $X \subseteq X$) and reads $Y \subseteq X$ off $X \in \uparrow Y$; $\leftarrow$ chains $Y \subseteq X \subseteq Z$. Injectivity `principal_injective` ($\uparrow X = \uparrow Y \implies X = Y$) feeds both `le_of_eq` directions through `principal_le_iff` into `Set.Subset.antisymm`.

Factoid 1.7b (density of finite elements). `eq_iUnion_principal`: $x.mem\ Z \iff \exists X, \exists hX : x.mem\ X, (\uparrow X).mem\ Z$ — Scott's $x = \bigcup \{\uparrow X \mid X \in x\}$ written as union membership (concrete, avoiding $\bigcup$ over a `Set (Set α)`). $\rightarrow$ uses $X = Z$ ($Z \in \uparrow Z$); $\leftarrow$ is one application of upward closure `x.up_mem` ($X \subseteq Z$ with $Z \in \mathcal{D}$).

Factoid 1.7a

“*obvious*”: $X \mapsto \uparrow X$ one-one & inclusion-reversing — `principal_le_iff` ($\uparrow X \sqsubseteq \uparrow Y \iff Y \subseteq X$) + `principal_injective`.

Factoid 1.7b

“*also obvious*”: $x = \bigcup \{\uparrow X \mid X \in x\}$ for every $x \in |\mathcal{D}|$ — `eq_iUnion_principal`.

Definition 1.8 (order)

`Element V` is Scott's filter (Def 1.6): a membership predicate `mem : Set α → Prop` with `sub` ($x \subseteq \mathcal{D}$), `master_mem` ($\Delta \in x$), `inter_mem` (closed under $\cap$), and `up_mem` (upward closed in $\mathcal{D}$). Mirroring `InfoSys.Element`, the early helper `Element.ext` (membership-equality $\implies$ equality, proved by `rcases` on both structures + `funext/propext`, *not congr*) keeps the `PartialOrder` instance (Def 1.8's approximation order $x \sqsubseteq y \iff x \subseteq y$) choice-free: `le_antisymm` is just `Element.ext fun X => ⟨h1 X, h2 X⟩`.

Definition 1.8 ($\perp$, total)

`bot` := `principal master_mem (⊥={Δ}=↑Δ)`, `mem_bot` ($Y \in \perp \iff Y = \Delta$); `IsTotal x` := $\forall y, x \sqsubseteq y \rightarrow y \sqsubseteq x$ (predicate only, existence = Ex 1.24, out of scope).

Factoid 1.8a

`bot_le` ($\perp \sqsubseteq x$ for all x) + `instance OrderBot Element`; constructive.

Factoid 1.8b

`eq_principal_of_isMin` (filter with $\subseteq$ -minimum member X is $\uparrow X$) — constructive core of “finite $\implies$ principal”; the finiteness $\implies$ min step left implicit.

Example 1.B

$B = \{\sigma \Sigma^* \mid \sigma \in \Sigma^*\}$ (binary), generalizing 1.4 — `Str := List Bool`, cone $\sigma = \sigma \Sigma^*$, B via `ofNestedOrDisjoint` from prefix `cone_trichotomy`.

Exercise 1.B-sys

“*should be done as an exercise*”: B is a neighbourhood system — `nestedOrDisjoint` (cones pairwise nested-or-disjoint).

Exercise 1.B-elt

“*an exercise here*”: $\sigma x \in |B|$ for $x \in |B|$ — `sigmaElt` $\sigma\ x$ (witness $\sigma(X_1 \cap X_2)$ is a cone); `sigmaElt` $\sigma\ \perp = \sigma\ \perp$ (`sigmaElt_bot`).

Factoid 1.B-mono

$\sigma_0 \perp \subseteq \sigma_1 \perp \iff \sigma_0$ is an initial segment of σ_1 — `sigmaBot_le_iff` ($\sigma_0 \perp \sqsubseteq \sigma_1 \perp \iff \sigma_0 <+ \sigma_1$).

Factoid 1.B-lim

$x = \bigcup_n \sigma_n \perp$ (element = limit of finite approx.) — `mem_iff_exists_sigmaBot` (union-of- $\sigma \perp$ form; chain enumeration left to prose / choice).

Definition 1.9

$\mathcal{D}_0 \cong \mathcal{D}_1$: order-iso of $|\mathcal{D}_0|$ and $|\mathcal{D}_1|$ — `DomainIso` := $V_0.\text{Element} \simeq_o V_1.\text{Element}$, `Isomorphic/ $\cong^D$` := `Nonempty DomainIso` with `refl/symm/trans`; $\simeq_o$ *reflects* $\sqsubseteq$ (`map_rel_iff`) = Scott's two-way inclusion-preservation.

Lean 4 source

```
def Isomorphic {a b : Type*} (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) : Prop
:=
```

Theorem 1.10

element-token system: $[X] = \{x \mid X \in x\}$ (`bracket`); `tokenSystem` : `NeighborhoodSystem` $|\mathcal{D}|$; $\mathcal{D} \cong^D$ `tokenSystem` via `tokenIso/isomorphic_tokenSystem` (mutually-inverse to `Token/ofToken`). Facts: `bracket_master` (1), `bracket_inter_nonempty_iff` (2), `bracket_inter` (3), `principal_mem_bracket` (4); one-one `bracket_injective`, preserving `bracket_subset_iff`.

Lean 4 source

```
theorem isomorphic_tokenSystem : V ~ = ? V.tokenSystem
```

Theorem 1.11

$|\mathcal{D}|$ closed under countable $\bigcap$ (`iInter`, no proviso) and ascending $\bigcup$ (`iUnion`, `Monotone x`) — each again a filter; `GLB iInter_le/le_iInter`, `LUB le_iUnion/iUnion_le`; `mem_iInter/mem_iUnion`.

Lean 4 source

```
def iInter (x : Nat -> V.Element) : V.Element where
```

Exercise 1.12

$\Delta = \mathbb{N}$, final-segment `tail n = {m | n ≤ m}`; `neighborhoodSystem` (`chain` via `ofNestedOrDisjoint`); finite elts `fin n = ↑(tail n)` (`fin_strictMono`); unique limit/total `top` (`le_top`, `top_isTotal`, `isTotal_iff_top`); `element_eq` (every elt `fin n` or `top`, classical) (`Exercise112.lean`).

Lean 4 source

```
def neighborhoodSystem : NeighborhoodSystem Nat :=
```

Exercise 1.13

assertions about `B = ExampleB.lean`; this file adds the **limit nodes**: `branch p =  $\bigcup_n (p \upharpoonright n) \perp$` (via Thm 1.11 `iUnion`), `branch_mem_iff`, `branchSeq_le_branch`, and `branch_isTotal` (each infinite path is a total/maximal element) (`Exercise113.lean`).

Lean 4 source

```
def B : NeighborhoodSystem Str :=
```

Exercise 1.14

$\Delta = \mathbb{N}$, $\mathcal{D}$ = finite non-empty subsets $\cup \{\Delta\}$; `neighborhoodSystem` (`manual inter_mem`, not `nested-or-disjoint`); finite elts `fin h = ↑X`; total elts = singletons `singleton_isTotal` (`↑{n}` maximal) (`Exercise114.lean`).

Lean 4 source

```
def neighborhoodSystem : NeighborhoodSystem Nat where
```

Exercise 1.15

two infinite finite-element domains: `flat` ($\{\mathbb{N}\} \cup \{\{n\}\}$, fully classified: `flat_classify`, `flat_atom_maximal`, `flat_no_three_chain`, `flat_no_infinite_chain`, `flat_all_finite`) and `stem` ($\{\mathbb{N}, \{0,1\}\} \cup \{\{n\}\}$, `stem_three_chain`); `not_isomorphic` (3-chain transports under $\simeq$) (`Exercise115.lean`).

Lean 4 source

```
def flat : NeighborhoodSystem Nat :=
```

Exercise 1.16

$\Delta = \mathbb{N}$, $\mathcal{D}$ = cofinite subsets; $|\mathcal{D}| \cong \mathcal{P}(\mathbb{N})$ under $\subseteq$ — `cofiniteSystem`, `ofExcluded/toExcluded`, `cofiniteIso` (excluded-point set), `mem_compl_of_finite` ($\bigcap_{n \in F} \{n\}^c = F^c$); total elt `ofExcluded` $\mathbb{N}$ (`ofExcluded_univ_isTotal`); second $\cap$ -closed `fullSystem` (`Cofinite ns`).

Lean 4 source

```
def cofiniteSystem : NeighborhoodSystem Nat where
```

Exercise 1.17

$\Delta = \mathbb{R}$, $\mathcal{D}$ = rational open intervals $\cup \{\Delta\}$; `ratIntervalSystem` (`inter_mem'` via `loo_inter_loo+max/min`), `filterAt` $t = \{X | t \in X\}$ is a filter, `filterAt_injective` ($\mathbb{R} \hookrightarrow |\mathcal{D}|$); full total-elt classification documented as out-of-scope (`RatInterval ns`).

Lean 4 source

```
def ratIntervalSystem : NeighborhoodSystem ? where
```

Exercise 1.18

consistent $C \subseteq \mathcal{D}$ (`FinitelyConsistent`); pairwise-but-not-jointly `triSys/family` (`family_pairwise_nonempty`, `not_finitelyConsistent`); `leastFilter` $\supseteq C$ (`subset_leastFilter/leastFilter_le`, via `interUpTo_appendSeq`); `sInf` of a non-empty family of filters is a filter (`sInf_le/le_sInf`) (`Exercise118.lean`).

Lean 4 source

```
def FinitelyConsistent (C : Set (Set a)) : Prop :=
  forall (n : Nat) (X : Nat -> Set a), (forall i, i < n -> X i in C) -> V.Consistent
    X n
```

Exercise 1.19

positive nbhd system (ii': $X \cap Y \neq \emptyset \iff X \cap Y \in \mathcal{D}$) — `IsPositive`, `ofPositive` (`positive` $\implies$ system, in `Basic.lean`); `positive` `positiveExample`; non-positive `notPositiveSystem` ($\{\Delta, \{0,1\}, \{1,2\}\}$, intersection $\{1\} \notin \mathcal{D}$; smaller than Hoare's $\mathbb{N} \times \mathbb{N}$) `not_isPositive` (`Exercise119.lean`).

Lean 4 source

```
def positiveExample : NeighborhoodSystem (Fin 2) :=
```

Exercise 1.20

$\Delta' = \mathcal{D}$, $\mathcal{D}' = \{\uparrow X\}$ with $\uparrow X = \{Y \in \mathcal{D} \mid Y \subseteq X\}$ (`upSet`, $\neq$ principal); `powerSystem`, `powerSystem_isPositive`; $|\mathcal{D}| \cong |\mathcal{D}'|$ via `toPower/ofPower/powerIso`, `isomorphic_powerSystem`; tokens $\leftrightarrow$ finite elements one-one (`toPower_principal`).

Lean 4 source

```
def upSet (X : Set a) : Set (Set a)
```

Exercise 1.21

(detail Thm 1.10) $\{[X]\}$ over $|\mathcal{D}|$ is *positive* (`tokenSystem_isPositive`) and *complete* (`IsComplete`, `tokenSystem_complete`: every filter fixed by a unique point of `Token y`; `tokenSystem_toToken_bijective`); consistency $\{X_i | i < n\} \iff \bigcap_{i < n} [X_i] \neq \emptyset$ (`consistent_iff_iInter_bracket_nonempty`).

Lean 4 source

```
theorem tokenSystem_isPositive : V.tokenSystem.IsPositive
```

Exercise 1.22

(for topologists) the $[X]$ topologize $|\mathcal{D}|$; open sets = (i) $\sqsubseteq$ -upper $\wedge$ (ii) basic-nbhd; $\sqsubseteq$ = specialization order — `basicOpen`, `instTopologicalSpaceElement`, `isOpen_basicOpen`, `isOpen_iff_upper_basic`, `le_iff_isOpen_imp`, `specializes_iff_le`.

Exercise 1.23

countable system (`enum/henum/hsurj`) + `[DecidablePred V.mem]` $\implies$ greedy sequence Y_n/acc gives a **total** element: `greedyElement`, `greedyElement_isTotal` (choice-free, `Y_prefix_consistent`); every filter is sequence-determined `filters_sequence_determined` (classical) (`Exercise123.lean`).

Lean 4 source

```
def acc (n : Nat) : Set a
```

Exercise 1.24

(set theorists) the union of a non-empty **chain** of filters is a filter — `chainUnion` (`inter_mem` via `IsChain.total`), `le_chainUnion`; **with Zorn** every element extends to a total one `exists_total_ge` (`zorn_le_nonempty_Ici0`, `IsMax = IsTotal`) — **classical** (`Exercise124.lean`).

Lean 4 source

```
def chainUnion (C : Set V.Element) (hne : C.Nonempty) (hchain : IsChain (* <= *) C) : V.Element where
```

Exercise 1.25

(set theorists) Δ linearly+well-ordered, $\mathcal{D}$ = non-empty upper sets (`finalSegmentSystem`); $|\mathcal{D}| \cong \{\text{non-empty lower sets}\}$ under $\subseteq$ — `finalSegmentClassify` (`lowerSetOf/ofLowerSet`); top element `topElement` is the unique total element (`topElement_isTotal`, `eq_topElement_of_isTotal`); with no maximum it is *not* finite/principal (`topElement_not_principal_of_noMax`).

Lean 4 source

```
def finalSegmentSystem (Delta : Type*) [LinearOrder Delta] [Nonempty Delta] : NeighborhoodSystem Delta :=
```

Exercise 1.26

(algebraists) commutative ring A (`[DecidableEq A]`), Δ = finite $F \subseteq A$, $I(F) = \{G \mid F \subseteq \langle G \rangle\}$ (`IFamily`, `IFamily_inter`); `ringSystem`; $|\mathcal{D}| \cong$ ideals of A under $\subseteq$ — `ringIso` (`idealOf/ofIdeal` mutually inverse).

Lean 4 source

```
def IFamily (F : Finset A) : Set (Finset A)
```

Exercise 1.27

bounded $X \subseteq |\mathcal{D}|$ (`Bounded`, `sSup = sInf` of `upperBounds`, `le_sSup/sSup_le`); $\{U, W\}$ consistent in $\mathcal{D} \iff \{\uparrow U, \uparrow W\}$ `bounded` `consistent_pair_iff_bounded` (choice-free); X bounded $\iff$ every finite subset `bounded` `bounded_iff_finite_bounded` (uses 1.18).

Lean 4 source

```
def Bounded (X : Set V.Element) : Prop
```

3.2 Lecture II: Approximable Mappings

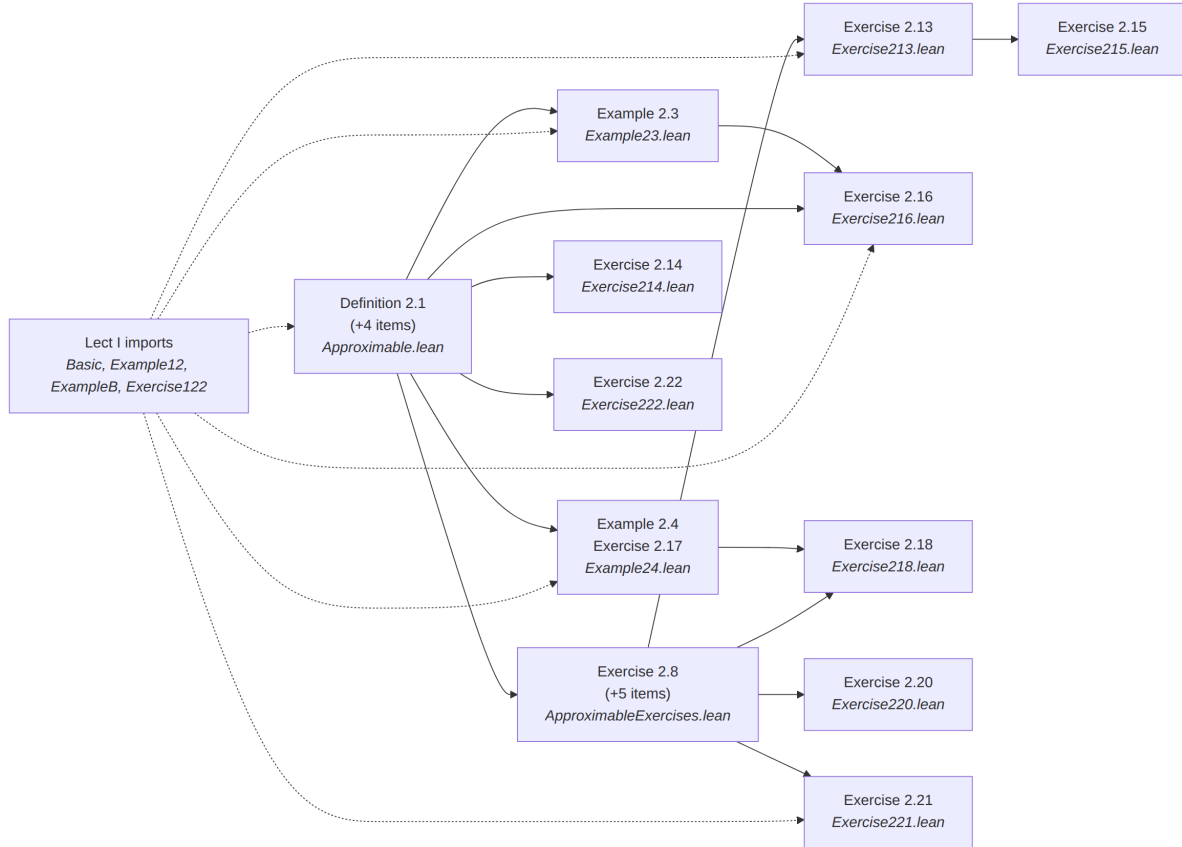


Figure 3: Lean module dependencies for Lecture II: Approximable Mappings.

Definition 2.1

ApproximableMap: relation $\text{rel} \subseteq \mathcal{D}_0 \times \mathcal{D}_1$ ($\text{rel_dom}/\text{rel_cod}$) with (i) `master_rel`, (ii) `inter_right`, (iii) `mono`; relation-extensionality `ext` (`Approximable.lean`).

Lean 4 source

```
structure ApproximableMap (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) where
  rel : Set a -> Set b -> Prop
  rel_dom : forall {X Y}, rel X Y -> V_0.mem X
  rel_cod : forall {X Y}, rel X Y -> V_1.mem Y
  master_rel : rel V_0.master V_1.master
  inter_right : forall {X Y Y'}, rel X Y -> rel X Y' -> rel X (Y I Y')
```

Proposition 2.2

`toElementMap` ($f(x) = \{Y \mid \exists X \in x, X f Y\}$, all of 2.1 used), `mem_toElementMap`, `rel_iff_mem_principal` ($X f Y \iff Y \in f(\uparrow X)$), `toElementMap_mono`, `ext_of_toElementMap` (2.2(iv)) (`Approximable.lean`)

Lean 4 source

```
def toElementMap (f : ApproximableMap V_0 V_1) (x : V_0.Element) : V_1.Element where
```

Example 2.3

parityMap : B → T: parity of 0's before first 1 via scanner scan/valElt (scan_append stability ⇒ mono); T=two-token domain of Ex 1.2 (Example23.lean).

Lean 4 source

```
def scan : Str -> Option Bool
```

Example 2.4

runMap : B → B: eliminate first run of 1's via state machine out/del; out_mono (prefix-monotone) ⇒ mono; total $1\infty \rightarrow \text{partial } \perp$ (Example24.lean, choice-free).

Lean 4 source

```
def out : Str -> Str
```

Theorem 2.5

category of nbhd systems + approximable maps: identity idMap ($X \text{ I_D } Y \iff X \subseteq Y$), composition comp g f ($X \text{ g of } Z \iff \exists Y, X \text{ f } Y \wedge Y \text{ g } Z$), laws idMap_comp/comp_idMap/comp_assoc (Approximable.lean).

Lean 4 source

```
def idMap (V : NeighborhoodSystem a) : ApproximableMap V V where
```

Proposition 2.6

elementwise functor: toElementMap_idMap (I_D(x)=x), toElementMap_comp ((g of f)(x)=g(f(x))) — concrete category of sets & functions (Approximable.lean).

Lean 4 source

```
theorem toElementMap_comp (g : ApproximableMap V_1 V_2) (f : ApproximableMap V_0 V_1) (x : V_0.Element) :
```

Theorem 2.7

every domain iso $e: |\mathcal{D}_0| \simeq_o |\mathcal{D}_1|$ comes from an approximable map ofIso e (toElementMap_ofIso: (ofIso e)(x)=e(x); exists_approximable_of_iso); finite→finite exists_principal_eq_apply_principal via directed union sSupDirected (choice-free).

Lean 4 source

```
theorem toElementMap_ofIso (e : V_0.Element ?o V_1.Element) (x : V_0.Element) :
```

Exercise 2.8

determined by finite elements eq_of_toElementMap_principal; any monotone fn on finite elements extends: ofMono, toElementMap_ofMono_principal (ApproximableExercises.lean).

Lean 4 source

```
theorem eq_of_toElementMap_principal {f g : ApproximableMap V_0 V_1}
(h : forall (X : Set a) (hX : V_0.mem X),
```

Exercise 2.9

approximable f satisfies $f(x) = \bigcup \{f(\uparrow X) \mid X \in x\}$ — toElementMap_mem_iff_principal (ApproximableExercises.lean)

Lean 4 source

```
theorem toElementMap_mem_iff_principal (f : ApproximableMap V_0 V_1) (x : V_0.Element) {Y : Set
  b} :
```

Exercise 2.10

Prop 2.6 (done in `Approximable.lean`); pointwise `meet` $h(x)=f(x)\cap g(x)$ — `interMap`, `mem_toElementMap_interMap` (`ApproximableExercises.lean`).

Lean 4 source

```
def interMap (f g : ApproximableMap V_0 V_1) : ApproximableMap V_0 V_1 where
```

Exercise 2.11

directed $a:I\rightarrow |D| \implies \bigcup_i a(i)$ is a filter (`iSupDirected`, `mem/le/le_`); approximable maps preserve directed $\bigcup$ — `toElementMap_iSupDirected`.

Lean 4 source

```
def iSupDirected {a : Type*} {V : NeighborhoodSystem a} {I : Type*} [Nonempty I]
  (a : I -> V.Element) (hdir : forall i j, exists k, a i <= a k /\ a j <= a k) : V.
  Element :=
```

Exercise 2.12

directed family $\{f_i\}$ of approximable maps: pointwise union $\bigcup_i f_i$ approximable — `iSupMap`, `mem_toElementMap_iSupMap` (`ApproximableExercises.lean`).

Lean 4 source

```
def iSupMap {I : Type*} [Nonempty I] (f : I -> ApproximableMap V_0 V_1)
  (hdir : forall i j, exists k, (forall X Y, (f i).rel X Y -> (f k).rel X Y) /\
    (forall X Y, (f j).rel X Y -> (f k).rel X Y)) : ApproximableMap V_0 V_1 where
```

Exercise 2.13

(topologists) approximable maps = continuous maps between the $|D|$ spaces of Ex 1.22 — `continuous_toElementMap`, `ofContinuous`, `toElementMap_ofContinuous`, `mem_iff_principal_of_continuous` (choice-free).

Lean 4 source

```
theorem continuous_toElementMap (f : ApproximableMap V_0 V_1) :
```

Exercise 2.14

domain iso e and nbhd correspondence φ from Thm 2.7; `phi/phi_spec`, `rel_ofIso_iff` (`(ofIso e).rel X Y  $\iff \varphi X \subseteq Y$`), `phi_inter` (`$\varphi(X \cap X') = \varphi X \cap \varphi X'$` for consistent X, X') (`Exercise214.lean`).

Lean 4 source

```
theorem phi_spec (e : V_0.Element ?o V_1.Element) {X : Set a} (hX : V_0.mem X) :
```

Exercise 2.15

The one-token system 0 (master $\{*\}$, neighbourhoods $\{\emptyset?, \{*\}\}$) is Scott's Sierpiński domain: its two elements are $\perp \sqsubset \top$. Building on Ex 2.13, open subsets of $|D|$ correspond bijectively to approximable maps $D \rightarrow 0$: `openToMap`/`mapToOpen` are mutually inverse, packaged as the equivalence `openSet_equiv_map`. The bijection uses choice (`equivSetNat`-style classical packaging of the open $\leftrightarrow$ characteristic-map data), so the footprint is `[propext, Classical.choice, Quot.sound]`.

Lean 4 source

```
def 0 : NeighborhoodSystem (Fin 1) :=
```

Exercise 2.16

$x \mapsto \sigma x$ on $|B|$ is approximable; the parity map $f: B \rightarrow T$ of Ex 2.3 is **uniquely** determined among approximable maps by $f(1x) = \text{true}$, $f(01x) = \text{false}$, $f(00x) = f(x)$.

First half: $\text{sigmaMap } \sigma : \text{ApproximableMap } B \ B, \text{toElementMap_sigmaMap } ((\text{sigmaMap } \sigma)(x) = \sigma x = \text{sigmaElt } \sigma \ x).$

Lean 4 source

```
theorem toElementMap_sigmaMap (sigma : Str) (x : B.Element) :
```

Exercise 2.17

$g: B \rightarrow B$ of Ex 2.4 is approximable (runMap); runMap is **uniquely** determined among approximable maps by $g(0x) = 0g(x)$, $g(11x) = g(1x)$, $g(10x) = 0x$, $g(1) = \perp$ — no equation is missing.

First half: $\text{runMap}: \text{ApproximableMap } B \ B$, relation $X \ g \ Y \leftrightarrow \exists \sigma, X = \sigma \Sigma^* \wedge Y \in B \wedge (\text{out } \sigma) \Sigma^* \subseteq Y$.
Second half (uniqueness): `SatisfiesRunEquations` packages Scott’s four equations (the last, $g(1) = \perp$, is a *value* equation at $x = \perp$ only, not quantified over x); `runMap_satisfies` confirms `runMap` itself meets them via two “shift formulas” — `runMap_toElementMap_sigmaElt` (input-side: $g(\sigma x)$ depends on x only through some cone $\text{cone } \tau \in x$, at $\text{out}(\sigma ++ \tau)$)

Lean 4 source

```
def runMap : ApproximableMap B B where
```

Exercise 2.18

“spacing” map $h: B \rightarrow B$ ($b \mapsto b0$) and left inverse k ; hMap/kMap , kMap_comp_hMap ($k \circ h = \text{I}_B$), $\text{kMap_not_injective}$, $\text{hMap_not_surjective}$ (h not an iso) (`Exercise218.lean`, choice-free).

Lean 4 source

```
def hMap : ApproximableMap B B where
```

Exercise 2.19

two-variable approximable maps $f: \mathcal{D}_0 \times \mathcal{D}_1 \rightarrow \mathcal{D}_2$ as ternary relations — `ApproximableMap2`, `toElementMap2`, `rel2_iff_mem_principal`, `toElementMap2_mono` (`ApproximableExercises.lean`).

Lean 4 source

```
theorem mem_iSupDirected {a : Type*} {V : NeighborhoodSystem a} {I : Type*} [Nonempty I]
  (a : I -> V.Element) (hdir : forall i j, exists k, a i <= a k /\ a j <= a k) {Z :
    Set a} :
```

Exercise 2.20

powerset domain $\mathcal{P}$ (cofinite nbhds over $\mathbb{N}$); equivSetNat ($|\mathcal{P}| \simeq \text{Set } \mathbb{N}$); $\text{unionMap}/\text{interMap}_2$ ($\cup, \cap$ via Ex 2.19), $\text{succMap}/\text{predMap}$ ($x \pm 1$).

Lean 4 source

```
def equivSetNat : powerSet.Element ?o Set Nat where
```

Exercise 2.21

system $C \supseteq B$ with finite *and* infinite total sequences (terminator singletons $\{\sigma\}$); $\text{isTotal_singletonElt}$, bot_lt_Lambda ($\perp \sqsubset \Lambda$); juxtaposition $\text{juxtapose} : C \times C \rightarrow C$ with juxtapose_cone (left bias) / $\text{juxtapose_singleton_mem}$ (`Exercise221.lean`, choice-free).

Lean 4 source

```
theorem isTotal_singletonElt (sigma : Str) : C.IsTotal (singletonElt sigma)
```

Exercise 2.22

(set theorists) any family $\mathcal{C}$ closed under non-empty $\bigcap$ + directed $\bigcup$ is inclusion-iso to a domain — closure Cl , $\text{reprSystem}(\text{nbhds } \mathcal{C}(F) = \{G \mid F \subseteq \bar{G}\})$, $\text{reprIso} : |\text{reprSystem}| \simeq_o \mathcal{C}$ (classical).

Lean 4 source

```
def Cl (F : Set tau) : Set tau
```

3.3 Lecture III: Domain Constructs

Definition 3.1

prod , prodNbhd ($\text{Sum.inl } ' ' X \cup \text{Sum.inr } ' ' Y$), element pairing pair , Element.fst/snd (Product.lean).

Lean 4 source

```
def prod (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) : NeighborhoodSystem (a ? b)
  where
```

Proposition 3.2

prod is a nbhd system; $\text{prodEquiv} : |\mathcal{D}_0 \times \mathcal{D}_1| \simeq_o |\mathcal{D}_0| \times |\mathcal{D}_1|$; pair_le_pair_iff .

Lean 4 source

```
def prod (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) : NeighborhoodSystem (a ? b)
  where
```

Definition 3.3

projections proj_0 , proj_1 ; paired map paired ; multivariate via prod (Product.lean).

Lean 4 source

```
def paired (f : ApproximableMap V_2 V_0) (g : ApproximableMap V_2 V_1) :
```

Proposition 3.4

$\text{proj}_0/\text{proj}_1/\text{paired}$ approximable; proj_comp_paired ; $\text{toElementMap_paired_apply}(\langle f, g \rangle(w) = \langle f(w), g(w) \rangle)$ (Product.lean).

Lean 4 source

```
theorem prodNbhd_inter (X X' : Set a) (Y Y' : Set b) :
```

Theorem 3.5

$\text{toMap}_2/\text{ofMap}_2/\text{map}_2\text{Equiv}$: $\text{ApproximableMap}(\text{prod } V_0 V_1) V_2 \simeq \text{ApproximableMap}_2 V_0 V_1 V_2$ (joint $\iff$ separate) (Product.lean).

Lean 4 source

```
theorem prodNbhd_inter (X X' : Set a) (Y Y' : Set b) :
```

Lemma 3.6

constant map constMap ; $\text{toElementMap_constMap}$ (Product.lean).

Lean 4 source

```
def constMap (V_0 : NeighborhoodSystem a) (b : V_1.Element) : ApproximableMap V_0 V_1 where
```

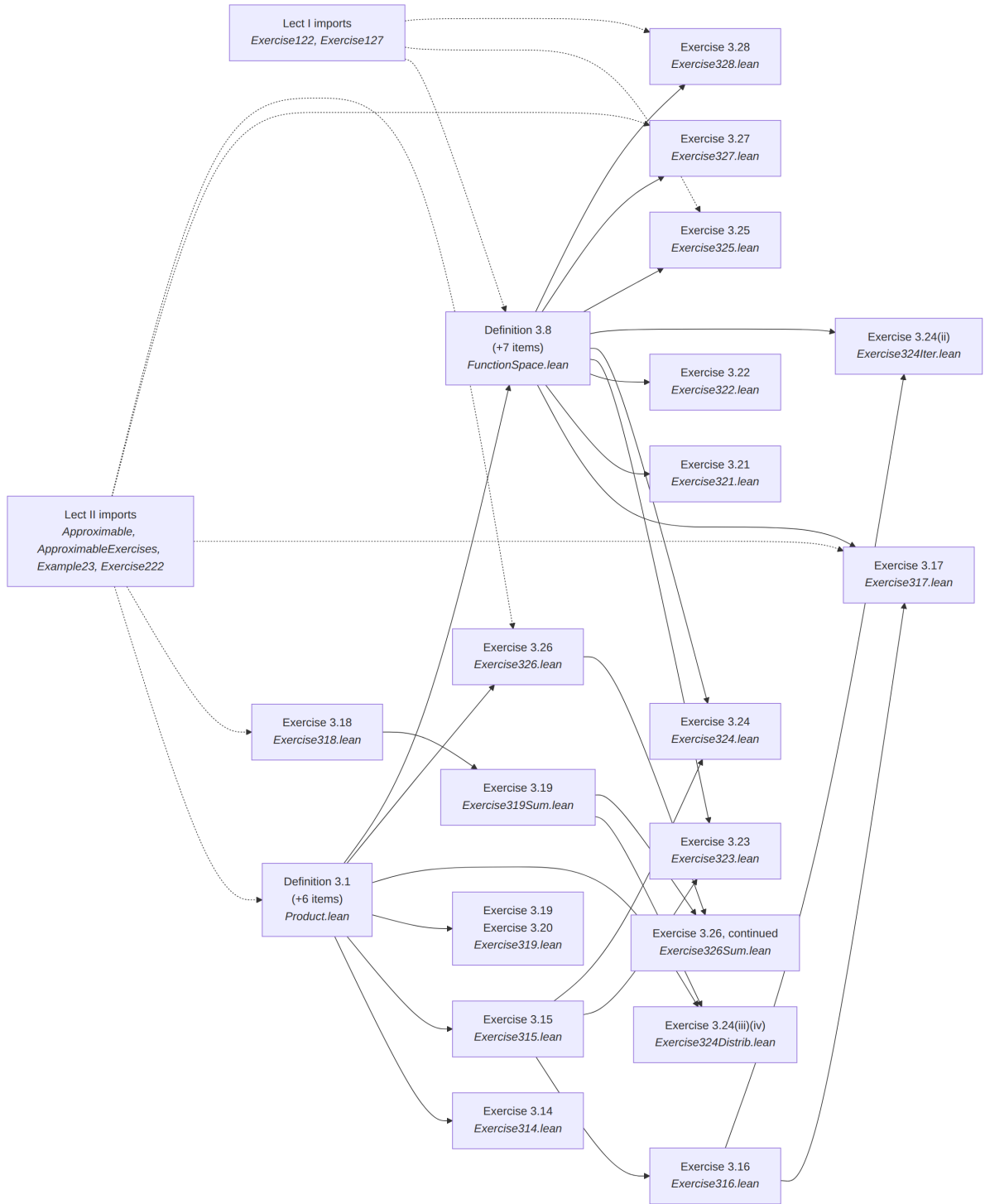


Figure 4: Lean module dependencies for Lecture III: Domain Constructs.

Proposition 3.7

substitution_toElementMap: multivariate functions closed under substitution (Product.lean).

Lean 4 source

```
theorem substitution_toElementMap (F : ApproximableMap (prod V_0 V_1) V_2)
  (a : ApproximableMap V_3 V_0) (b : ApproximableMap V_3 V_1) (w : V_3.Element) :
```

Definition 3.8

step ([X,Y]={f|X f Y}), stepFun, funSpace; algebra step_inter_right/step_subset/step_master_eq/step_mem (FunctionSpace.lean).

Lean 4 source

```
def step (X : Set a) (Y : Set b) : Set (ApproximableMap V_0 V_1)
```

Proposition 3.9

interYs, leastMap (cond. (ii) $X \sqsubseteq_0 Y \iff \bigcap \{Y_i | X \subseteq X_i\} \subseteq Y$), leastMap_mem_stepFun, leastMap_le (minimal element), stepFun_subset_step_iff (remark after 3.9) (FunctionSpace.lean).

Lean 4 source

```
def interYs (m : Set b) : List (Set a x Set b) -> Set a -> Set b
```

Theorem 3.10

funSpaceEquiv : $|\mathcal{D}_0 \rightarrow \mathcal{D}_1| \simeq_0$ ApproximableMap $V_0 V_1$ (toApproxMap/toFilter); completeness, inclusion-preserving.

Lean 4 source

```
def toApproxMap (phi : (funSpace V_0 V_1).Element) : ApproximableMap V_0 V_1 where
```

Theorem 3.11

eval : ApproximableMap₂ (funSpace $V_1 V_2$) $V_1 V_2$, evalMap; evalMap_apply (eval(f,x)=f(x)) (FunctionSpace.lean).

Lean 4 source

```
def evalMap (V_1 : NeighborhoodSystem b) (V_2 : NeighborhoodSystem ?) :
```

Theorem 3.12

curry, uncurry; toElementMap_curry_apply; uncurry_curry/curry_uncurry; eval_comp_curry/curry_eval_comp; curryEquiv (adjunction) (FunctionSpace.lean).

Lean 4 source

```
def curry (g : ApproximableMap (prod V_0 V_1) V_2) : ApproximableMap V_0 (funSpace V_1 V_2)
  where
```

Theorem 3.13(i)

le_iff_toElementMap_le ($f \sqsubseteq g \iff \forall x, f(x) \sqsubseteq g(x)$) (FunctionSpace.lean).

Lean 4 source

```
theorem le_iff_toElementMap_le {f g : ApproximableMap V_0 V_1} :
```

Theorem 3.13(ii)

mapsBounded_iff_pointwiseBounded (F bounded $\iff \{f(x)\}$ bounded $\forall x$) (FunctionSpace.lean).

Lean 4 source

theorem mapsBounded_iff_pointwiseBounded {F : Set (ApproximableMap V_0 V_1)} :

Theorem 3.13(iii)

sSupMaps + toElementMap_sSupMaps ($(\sqcup F)(x) = \sqcup \{f(x)\}$) (FunctionSpace.lean).

Lean 4 source

```
def sSupMaps (F : Set (ApproximableMap V_0 V_1)) (hF : PointwiseBounded F) : ApproximableMap
  V_0 V_1 :=
```

Exercise 3.14

tagged product $0\Delta_0 \cup 1\Delta_1$ (disjointness unnecessary); $\text{diag}: D \rightarrow D \times D$; n-fold products.

Lean 4 source

```
theorem proj_0_comp_diag : (proj_0 V V).comp (diag V) = idMap V :=
```

Exercise 3.15

product isomorphisms: commutativity, associativity, empty product, functoriality.

Lean 4 source

```
theorem prod_comm_isomorphic : prod V_0 V_1  ~ = ? prod V_1 V_0
```

Exercise 3.16

$\mathcal{D}_\infty$ over Δ_∞ ; $\mathcal{D}_\infty \cong \mathcal{D} \times \mathcal{D}_\infty$; elements = infinite sequences of $|\mathcal{D}|$ elements.

Lean 4 source

```
theorem fiber_mono {W W' : Set (Nat x a)} (h : W subseq W') (i : Nat) : fiber W i
  subseq fiber W' i :=
```

Exercise 3.17

$B \rightarrow T_\infty$ and $T_\infty \rightarrow B$ approximable; section/retraction; iso questions.

Lean 4 source

```
theorem bitNbhd_mem (b : Bool) : T.mem (bitNbhd b)
```

Exercise 3.18

sum system $\mathcal{D}_0 + \mathcal{D}_1$; injections in_i , projections out_i ; $\text{out}_i \circ \text{in}_i = I$; n-term sums.

Lean 4 source

```
theorem inj_0_inter (X X' : Set a) :
```

Exercise 3.19

functorial $f \times g$ and $f+g$ on products/sums; $f \times g = \langle f \circ p_0, g \circ p_1 \rangle$; $\text{out}_i \circ (f+g) \circ \text{in}_i = f/g$.

Lean 4 source

```
theorem toElementMap_prodMap (f : ApproximableMap V_0 V_0') (g : ApproximableMap V_1 V_1')
  (w : (prod V_0 V_1).Element) :
```

Exercise 3.20

(category theorists) + and $\times$ are functors; $\times$ is the categorical product.

Lean 4 source

```
theorem toElementMap_prodMap (f : ApproximableMap V_0 V_0') (g : ApproximableMap V_1 V_1')
  (w : (prod V_0 V_1).Element) :
```

Exercise 3.21

$[Y, Z]$ in $(D_1 \rightarrow D_2)$ uniquely determines Y, Z when $Z \neq \Delta_2$; edge case $Z = \Delta_2$.

Lean 4 source

```
theorem interYs_single_subset_eq {YX : Set a} {Z : Set b} {X : Set a} (hZ : V_1.mem Z)
```

Exercise 3.22

composition $\text{comp} : (D_1 \rightarrow D_2) \times (D_0 \rightarrow D_1) \rightarrow (D_0 \rightarrow D_2)$ approximable; $\text{comp}(g, f) = g \circ f$; from `eval + curry`.

Lean 4 source

```
theorem toElementMap_compApp (Gphi : (funSpace V_1 V_2).Element) (Fphi : (funSpace V_0 V_1).
  Element)
  (x : V_0.Element) :
```

Exercise 3.23

(category theorists) domains + approximable maps form a cartesian closed category (3.11, 3.12).

Lean 4 source

```
theorem toUnit_unique (f : ApproximableMap V unitSys) : f = toUnit V
```

Exercise 3.24

more function-space isos: (i) $(D_0 \rightarrow D_1 \times D_2) \cong (D_0 \rightarrow D_1) \times (D_0 \rightarrow D_2)$, (ii) $(D_0 \rightarrow D_1 \infty) \cong (D_0 \rightarrow D_1) \infty$; (iii)(iv) as canonical mapping relationships (separated-sum bottom obstructs iso).

Lean 4 source

```
theorem funProd_isomorphic :
```

Exercise 3.25

(topologists) open subsets of $|D|$ form a domain (uses 3.10, Exercises 1.21 & 2.13).

Lean 4 source

```
theorem mapOfOpen_rel_empty (U : Set V.Element) (X : Set a) :
```

Exercise 3.26

conditional $\text{cond} : T \times D \times D \rightarrow D$ ($\text{cond}(\text{true}, x, y) = x$, etc.); sum variant $\text{condSum} : T \times D_0 \times D_1 \rightarrow D_0 + D_1$; which: $D_0 + D_1 \rightarrow T$ with $\text{cond}(\text{which } x, \text{in}_0 \text{ out}_0 x, \text{in}_1 \text{ out}_1 x) = x$.

Lean 4 source

```
theorem zero_ne_one : (Example12.zero : Set Example12.Token) != Example12.one
```

Exercise 3.27

(set theorists) alt proof $(D_0 \rightarrow D_1)$ is a domain via Ex 2.22; compare with 3.9/3.10.

Lean 4 source

```
theorem graph_injective : Function.Injective (graph (V_0
```

Exercise 3.28

minimal element of $\bigcap [X_i, Y_i]$ in function space: $f_0(x) = \bigsqcap \{\uparrow Y_i \mid x \in [X_i]\}$.

Lean 4 source

```
theorem Element.mem_interYs_of (z : V_1.Element) {L : List (Set a x Set b)} {X : Set a}
```

3.4 Lecture IV: Fixed Points and Recursion

Theorem 4.1

every approximable $f:D \rightarrow D$ has a **least** fixed point $\text{fix}(f) = \sqcup_n f^n(\perp)$.

Lean 4 source

```
theorem toElementMap_fixElement (f : ApproximableMap V V) :
```

Theorem 4.2

the fixed-point operator $\text{fix}:(D \rightarrow D) \rightarrow D$ is itself approximable; $\text{fix}(f) = \sqcup_n f^n(\perp)$.

Lean 4 source

```
def fixMap (V : NeighborhoodSystem a) : ApproximableMap (funSpace V V) V :=
```

Example 4.3

the natural-number domain $\mathbb{N}$ (infinite generalization of Ex 1.2); 0, successor, predecessor.

Lean 4 source

```
abbrev N : NeighborhoodSystem Nat :=
```

Example 4.4

the domain $\mathbb{C}$ of finite/infinite binary sequences (Ex 2.21) as a structured domain.

Lean 4 source

```
abbrev C : NeighborhoodSystem Str :=
```

Definition 4.5

model for Peano's Axioms $\langle \mathbb{N}, 0, + \rangle$ (zero not a successor, successor injective, induction).

Lean 4 source

```
theorem Graph.swap {P : PeanoModel M} {Q : PeanoModel N} {m : M} {n : N}
```

Theorem 4.6

all models of Peano's Axioms are isomorphic.

Lean 4 source

```
theorem Graph.swap {P : PeanoModel M} {Q : PeanoModel N} {m : M} {n : N}
```

Exercise 4.7

$a \sqsubseteq f(a) \implies$ is there a fixed point $x=f(x)$ with $a \sqsubseteq x$?

Yes — replace $\perp$ by a : the chain $a \sqsubseteq f(a) \sqsubseteq f^2(a) \sqsubseteq \dots$ is increasing hence directed, so $\text{fixAbove } a \ f := \sqcup_n f^n(a)$ is a genuine element (approximable maps preserve directed unions, `toElementMap_iSupDirected`), giving a fixed point above a — `fixAbove_isFixed`, `le_fixAbove`, `fixAbove_least`; choice-free)

Lean 4 source

```
theorem fixAbove_isFixed (f : ApproximableMap V V) {a : V.Element} (ha : a <= f.toElementMap a) :
```

Exercise 4.8

$f:D \rightarrow D$, $S \subseteq |D|$ closure conditions for fixed points — `fix_induction` (fixed-point induction) + the $S=\{x \mid a(x)=b(x)\}$ corollary `fix_induction_eq`.

Lean 4 source

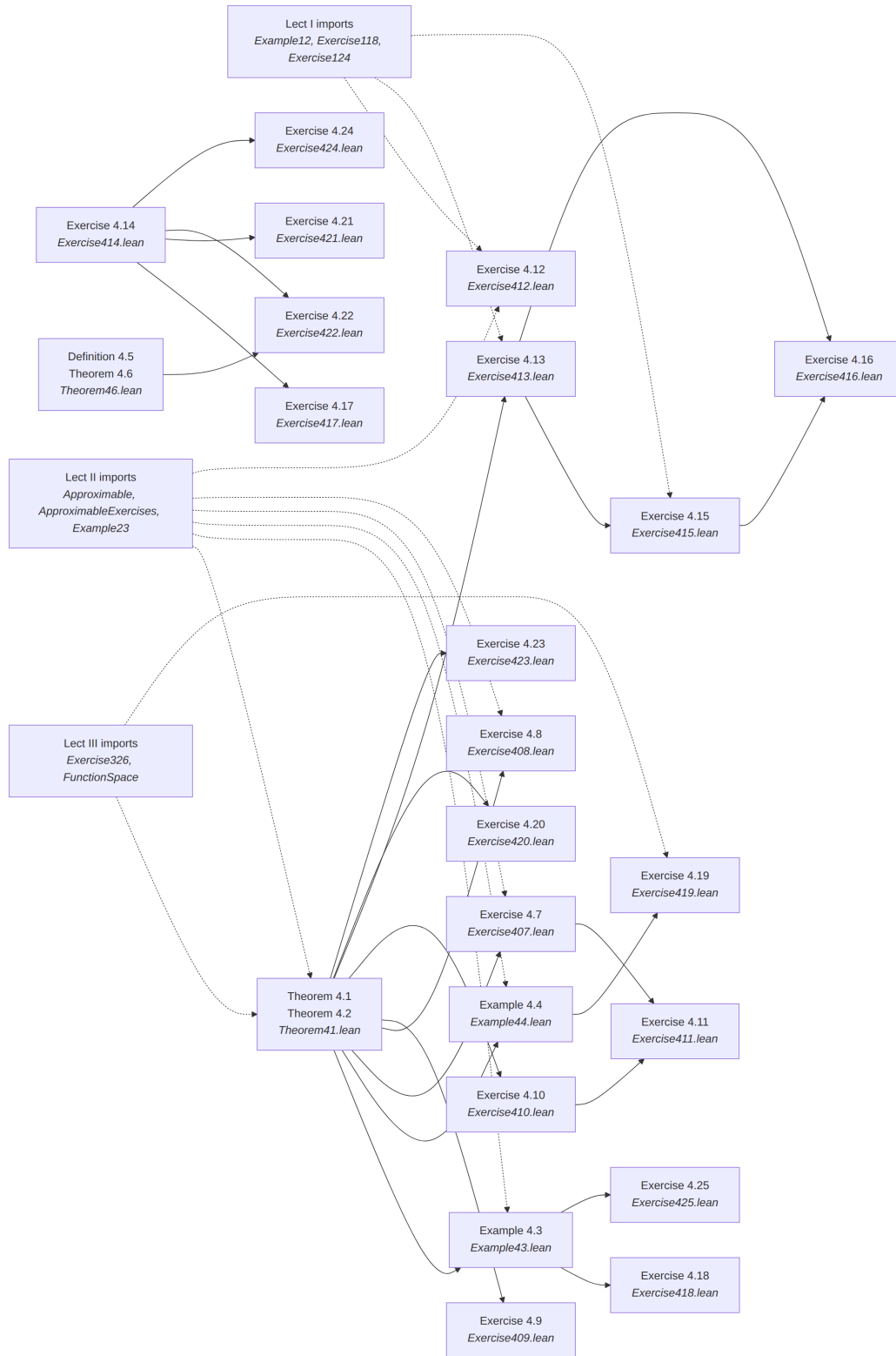


Figure 5: Lean module dependencies for Lecture IV: Fixed Points and Recursion.

```

theorem fix_induction (f : ApproximableMap V V) (P : V.Element -> Prop)
  (hbot : P V.bot)
  (hstep : forall x, P x -> P (f.toElementMap x))
  (hsup : forall (s : Nat -> V.Element) (hmono : Monotone s), (forall n, P (s n)) -> P
    (supChain s hmono)) :

```

Exercise 4.9

an approximable operator (least fixed point over a family).

the operator $\Psi : ((\mathcal{D} \rightarrow \mathcal{D}) \rightarrow \mathcal{D}) \rightarrow ((\mathcal{D} \rightarrow \mathcal{D}) \rightarrow \mathcal{D})$, built from the cartesian-closed combinators as $\Psi = \text{curry } \Phi$ (bigPsi) with bigPsi_apply : $\Psi(\theta)(f) = f(\theta(f))$; fix_eq_fixElement_bigPsi : $\text{fix} = \text{fix}(\Psi)$ via bigPsi_fix+bigPsi_least; operator data choice-free)

Lean 4 source

```

def bigPsi (V : NeighborhoodSystem a) :

```

Exercise 4.10

construct the relativized domain D_a (elements above a).

relSystem a (mem X := a.mem X) is order-isomorphic to $\{x \in |\mathcal{D}| \mid x \sqsubseteq a\}$ via relIso; if $f(a) = a$ (e.g.

Lean 4 source

```

def relIso (a : V.Element) : (relSystem a).Element ?o {x : V.Element // x <= a} where

```

Exercise 4.11

(Plotkin) fix uniquely determined by general conditions on $D \rightsquigarrow F_D$.

fix satisfies uniformity (fixElement_uniform: $h \circ f_0 = f_1 \circ h$, $h(\perp) = \perp \implies h(\text{fix } f_0) = \text{fix } f_1$, by induction + preservation of directed unions); fix_unique_of_uniform via the inclusion inclMap : $D_a \hookrightarrow D$ + Ex 4.10's unique fixed point)

Lean 4 source

```

theorem fixElement_uniform {b ? : Type*} {W_0 : NeighborhoodSystem b} {W_1 : NeighborhoodSystem
  ?}
  (f_0 : ApproximableMap W_0 W_0) (f_1 : ApproximableMap W_1 W_1) (h : ApproximableMap W_0
    W_1)
  (hbot : h.toElementMap W_0.bot = W_1.bot)
  (hintw : forall x, h.toElementMap (f_0.toElementMap x) = f_1.toElementMap (h.toElementMap
    x)) :

```

Exercise 4.12

need f have a *maximum* fixed point? example with many fixed points.

Lean 4 source

```

theorem elemOne_not_le_elemZero : not elemOne <= elemZero

```

Exercise 4.13

eliminate the apparent circularity between 4.1 and 4.6.

monoFix = $\bigcap \{x \mid f(x) \sqsubseteq x\}$ least fixed point of monotone f (choice-free); (3) exists_unique_nat_rec primitive recursion; (4)

Lean 4 source

```

theorem exists_unique_nat_rec {Z : Type*} (z : Z) (op : Z -> Z) :

```

Exercise 4.14

need monotone $f : PA \rightarrow PA$ have a maximum fixed point?.

Lean 4 source

```
theorem subset_gfpSet (f : Set A -> Set A) {x : Set A} (hx : x subseq f x) : x subseq
  gfpSet f :=
```

Exercise 4.15

(set theorists) monotone $f: |D| \rightarrow |D|$ has a *maximal* fixed point (Zorn) — `exists_maximal_fixedPoint` via `zorn_le0` on post-fixed points + `chainUnion`; `exists_least_fixedPoint` via `monoFix`; classical.

Lean 4 source

```
theorem exists_maximal_fixedPoint (f : V.Element -> V.Element) (hf : Monotone f) :
```

Exercise 4.16

(fixed-point nuts) the *optimal* fixed point.

for non-empty S a set of fixed points of monotone f , $f(\bigcap S) \subseteq \bigcap S$ (`f_sInf_le`), so $\bigcap S$ carries a least fixed point `optimalFix S`, itself $\subseteq \bigcap S$; `optimalFix` below/consistent with every fixed point in S — `optimalFix_le`, `optimalFix_consistent`; choice-free data)

Lean 4 source

```
theorem f_sInf_le (f : V.Element -> V.Element) (hf : Monotone f) (S : Set V.Element)
```

Exercise 4.17

(algebraists) semigroup $\langle S, 1, \cdot \rangle$, PS a domain; least x .

the least x with $x = \{1\} \cup \{a, b\} \cup x \cdot x$ is the submonoid generated by $\{a, b\}$ — `lfpSet (F a b) =  $\langle \{a, b\} \rangle$  (lfpSet_eq_closure), Knaster–Tarski via Ex 4.14; non-unique — Set.univ also fixed (fixedPoint_not_unique)`

Lean 4 source

```
theorem lfpSet_eq_closure (a b : S) :
```

Exercise 4.18

verify the assertions about N , F in Example 4.3.

Lean 4 source

```
theorem not_N_mem_empty : not N.mem ({ } : Set Nat)
```

Exercise 4.19

verify Example 4.4; `one: C → T` from the rest by a fixed-point equation.

Lean 4 source

```
theorem peano_nil_ne_cons (b : Bool) (sigma : Str) : ([] : Str) != b :: sigma
```

Exercise 4.20

$\text{fix}(f \circ g) = f(\text{fix}(g \circ f))$.

Lean 4 source

```
theorem comp_fixElement_isFixed (f g : ApproximableMap V V) :
```

Exercise 4.21

$\leq \subseteq N \times N$ as a unique fixed-point equation; addition/multiplication.

$\leq$ is the unique fixed point of $\ell = \{(n, n)\} \cup \{(n, m^+) \mid (n, m) \in \ell\}$ in $P(N \times N)$ — `leRel_isFixed` + uniqueness `leOp_unique` (induction on the second coordinate); the up-sets `[m] = upSet m`

with `upSet_zero/upSet_succ/upSet_unique` (4.13(3)); the addition iso `addIso :  $\mathbb{N} \simeq [m]$` (`addIso_apply/_zero/_succ`); multiplication `mulOp_lfp_eq_multiples` (least solution = multiples))

Lean 4 source

```
theorem leRel_isFixed : leOp leRel = leRel
```

Exercise 4.22

N^* satisfying (i)(ii) $\implies$ subset N satisfying (i)(ii)(iii)?.

Yes — the least fixed point `nats` of $g(x)=\{0\}\cup x^+$ in $P(N^*)$ (Knaster–Tarski, Ex 4.14) is the smallest subset containing 0 and closed under $^+$, `zero_mem_nats/succ_mem_nats/nats_induction`; `peanoSub : PeanoModel {m // m  $\in$  nats}` (all three axioms)

Lean 4 source

```
def nats : Set M
```

Exercise 4.23

(Eilenberg) unique fixed point under an approximation a_n scheme.

given f approximable and a sequence a_n with (i) $a_0=\perp$, (ii)+(iii)

Lean 4 source

```
theorem f_unique_fixedPoint (f : ApproximableMap V V) (a : Nat -> ApproximableMap V V)
  (ha0 : forall x, (a 0).toElementMap x = V.bot)
  (hlub : forall x, IsLUB {y | exists n, y = (a n).toElementMap x} x)
  (hcomm : forall n x, (a (n + 1)).toElementMap (f.toElementMap x) =
```

Exercise 4.24

(set theorists) Schröder–Bernstein via the fixed-point theorem (Tarski).

for one-one $f:A\rightarrow B$, $g:B\rightarrow A$, a fixed point X of $T(X)=(A-g(B))\cup g(f(X))$ in $P A$ (Knaster–Tarski, Ex 4.14) splits A into $h(a)=f(a)$ on X , $h(a)=g^{-1}(a)$ off X (choice-free), bijection `sbFun` with `sbFun_injective/sbFun_surjective  $\implies$  schroeder_bernstein + schroeder_bernstein_equiv :  $A \simeq B$` ; classical)

Lean 4 source

```
theorem sbFun_injective (hf : Injective f) : Injective (sbFun f g)
```

Exercise 4.25

the system C_1 over $\{1\}^*$ analogous to N .

$C_1 = \{\text{tail } n\} \cup \{\{1^n\}\}$ (nested-or-disjoint over unary strings $1^n \leftrightarrow n$), total elements `oneElem/partial oneBot`, successor `consMap (consMap_oneElem/_oneBot)`, the infinite fixed point `infElt =  $1\infty$` (`infElt_eq`) distinguishing non-flat C_1 from flat N , and the relating map `relateNToC1 :  $N \rightarrow C_1$` ; data choice-free `oneElem/oneBot`, successor `consMap (consMap_oneElem/_oneBot)`, the infinite fixed point `infElt =  $1\infty$` (`infElt_eq`)

Lean 4 source

```
def oneElem (n : Nat) : C1.Element
```

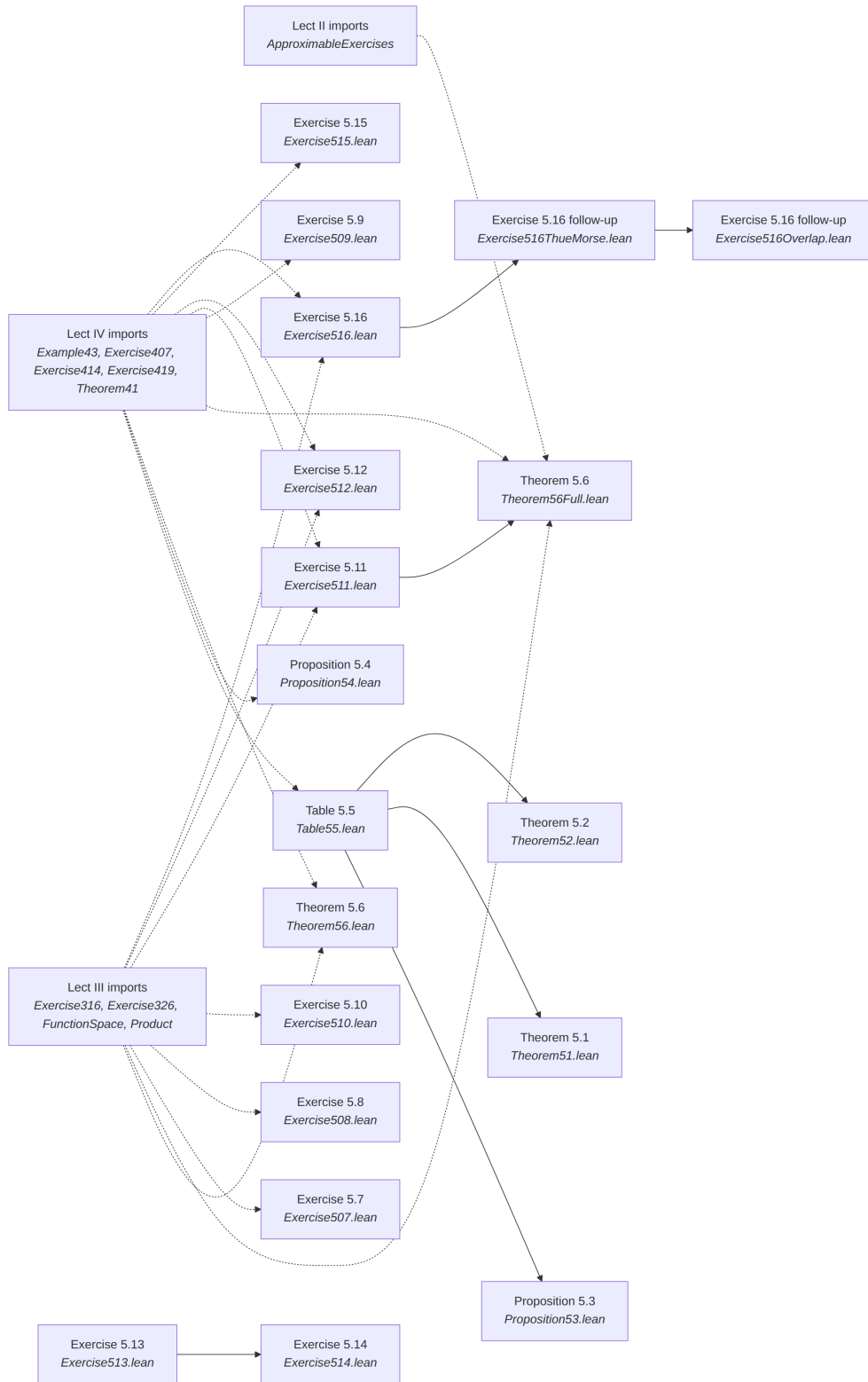


Figure 6: Lean module dependencies for Lecture V: Typed λ -Calculus.

3.5 Lecture V: Typed λ -Calculus

Theorem 5.1

every typed λ -term defines an approximable function of its free variables.

Lean 4 source

```
theorem var_fst (x : V_0.Element) (y : V_1.Element) :
```

Theorem 5.2

the conversion/substitution equation for suitably typed λ -terms.

Lean 4 source

```
theorem subst_value (g : ApproximableMap (prod V_0 V_1) V_2) (v : V_0.Element) (x : V_1.Element) :
```

Proposition 5.3

least fixed point of a pair-valued λ , coordinatewise (Bekić).

Lean 4 source

```
theorem sectionX_apply (tau : ApproximableMap (prod V_0 V_1) V_0) (y : V_1.Element) (x : V_0.Element) :
```

Proposition 5.4

fixed-point equation for $g: (D \rightarrow D)$.

Lean 4 source

```
theorem section_2_apply (tau : ApproximableMap (prod V V) V) (x y : V.Element) :
```

Table 5.5

summary table: combinators defined via λ -notation.

Lean 4 source

```
theorem pairC_apply (x : V_0.Element) (y : V_1.Element) :
```

Theorem 5.6

every partial recursive $h: \mathbb{N} \rightarrow \mathbb{N}$ is λ -definable (over primitives `cond/succ/pred/zero/0`).

+ Theorem56Full.lean: the full closure `partrec_lamDef` wired against `Mathlib Nat.Primrec' / Nat.Partrec'` on the universal arg domain $\mathcal{N} = \mathbb{N}^\infty$, with `rfind` divergence via the directed-sup continuity, and Scott's 1-ary corollary `partrec_one`

Lean 4 source

```
theorem T_bot_eq : (Example43.T).bot = Example23.botElt :=
```

Exercise 5.7

multi-variable λ /application from one-variable forms (p_0, p_1, pair) .

Lean 4 source

```
theorem surjective_pairing (z : (prod V_0 V_1).Element) :
```

Exercise 5.8

(combinator nuts) combinatory completeness: bracket abstraction $(I/K/S)$ eliminates λ , $\sigma(\tau)$ only.

Lean 4 source

`theorem toApproxMap_toFilter {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}`

Exercise 5.9

commuting f, g have a least common fixed point (cf. 4.20).

Lean 4 source

`theorem commute_apply {f g : ApproximableMap V V} (hcomm : f.comp g = g.comp f) (x : V.Element) :`

Exercise 5.10

the *smash product* $D_0 \otimes D_1$, the *strict function space* $D_0 \rightarrow \perp D_1$, and the adjunction $(D_0 \otimes D_1) \rightarrow \perp D_2 \simeq D_0 \rightarrow \perp (D_1 \rightarrow \perp D_2)$.

Lean 4 source

`theorem inter_ne_master_left {X X' : Set a} (hX : V_0.mem X) (hXne : X != V_0.master) :`

Exercise 5.11

D_∞ as bottomless *stacks*; stack combinators (head/tail/push/diag/map).

Lean 4 source

`theorem iterProdIso_apply (z : (iterSys V).Element) :`

Exercise 5.12

the `while` combinator on D by least fixed point.

Lean 4 source

`theorem bodyMap_apply (w : (funSpace V V).Element) (x : V.Element) :`

Exercise 5.13

a one-one pairing $\text{num} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$.

closed form $\text{num}(n, m) = (n+m)(n+m+1)/2 + m$ (Cantor diagonal), the three recurrences + `num_injective`, the bijection $\text{numEquiv} : \mathbb{N} \times \mathbb{N} \simeq \mathbb{N}$ (choice-free inverse `unnum`); power-set domains as $(\text{Set } \cdot, \subseteq)$, setCongr order-iso $\implies P \ N \cong P(N \times N)$, $P \ N \cong P \ N \times P \ N$, $P(N \times N) \cong P \ N \times P \ N$; choice-free)

Lean 4 source

`theorem num_injective : Function.Injective numP`

Exercise 5.14

approximable `fun/graph` mappings.

the tag coding $[n_0, \dots, n_k] = \text{num}(n_0, [n_1, \dots, n_k])$, packaged as `tag : List N × N → N` (`tag_injective`; `tag_surjective` by strong induction, decreasing via `num_succ_left_gt`). With `Fun u x = {m | ∃ ns ⊆ x, tag ns m ∈ u}`, `Graph f = {tag ns m | m ∈ f(entries ns)}` and `IsApprox (mono-tone + finite-approx) : Fun Graph (fun ∘ graph = λ f. f for continuous f)`, `id_le_Graph_Fun (graph ∘ fun ⊇ λ x. x)`, `Fun_isApprox` (every `Fun u` is approximable); $P\omega = (\text{Set } \mathbb{N}, \subseteq)$ per 4.17/5.13; choice-free)

Lean 4 source

`theorem tag_injective : forall {ns_1 ns_2 : List Nat} {m_1 m_2 : Nat},`

Exercise 5.15

(algebraists) free semigroup $\{0, 1\}^*$, $P\{0, 1\}^*$ as a domain.

the powerset domain $P\ S = (\text{Set } S, \subseteq)$ of a monoid S , pointwise product $s \cdot t = \{u \cdot v \mid u \in s, v \in t\}$, as a fact about the Kleene algebra $(\text{Set } S, \cup, \cdot, \emptyset, \{1\})$ for any monoid S . $\text{star } z = \bigcup_n z^n$ (recursive kpow , $\text{star_eq} : z^* = \Lambda \cup z \cdot z^*$).

Lean 4 source

```
def kpow (z : Set S) : Nat -> Set S
```

Exercise 5.16

a fixed-point definition of $\text{neg} : C \rightarrow C$.

$\text{tail} : C \rightarrow C$ ($\text{tail}(bx) = x$, $\text{tail}(\Lambda) = \perp$, the item left to the reader) via `Exercise419.liftC`.

Lean 4 source

```
theorem neg_cons_false : negMap.comp (consMap false) = (consMap true).comp negMap
```

3.6 Lecture VI: Domain Equations

Example 6.1

iterating $D \times D$ indefinitely into a single domain (D_∞ -style construct).

$D^\dagger$, the tree algebra over D , solving $D^\dagger \cong D + (D^\dagger \times D^\dagger)$. Tokens live in $\Gamma = \{1, 2\}^* 0 \Delta$, modelled as $\text{List Bool} \times \alpha$ with master $\Gamma = \{t \mid t.2 \in \Delta\}$ ($\text{true}=1$, $\text{false}=2$); the three neighbourhood embeddings $\text{embZero } X = 0X$, $\text{embL } P = 1P$, $\text{embR } Q = 2Q$, $\text{embPair } P \ Q = 1P \cup 2Q$ with their intersection/subset/injectivity/disjointness API.

Lean 4 source

```
theorem memS_inter {D : NeighborhoodSystem a} (hD : forall X, D.mem X -> X.Nonempty) :
  forall {X : Set (List Bool x a)}, MemS D X -> forall {Y : Set (List Bool x a)},
    MemS D Y ->
```

Example 6.2

B, C as solutions of domain equations (isomorphisms).

$B \cong B+B$ and $C \cong \{\{\Lambda\}\}+C+C$, both presented over $\text{Str} = \text{List Bool}$ with its intersection/subset/injectivity/disjointness API and the neighbourhood-shape classification `memB_cases` (master Σ^* , $0X$, $1X$); the forward/inverse filter maps `toBB/fromBB` and the order-iso `bbEquiv : |B|  $\simeq_o$  |B + B|` against the project's $+$ (Ex 3.18) give `B_domain_equation : B  $\cong^D$  sum B B`

Lean 4 source

```
theorem memB_cases {W : Set Str} (hW : B.mem W) :
```

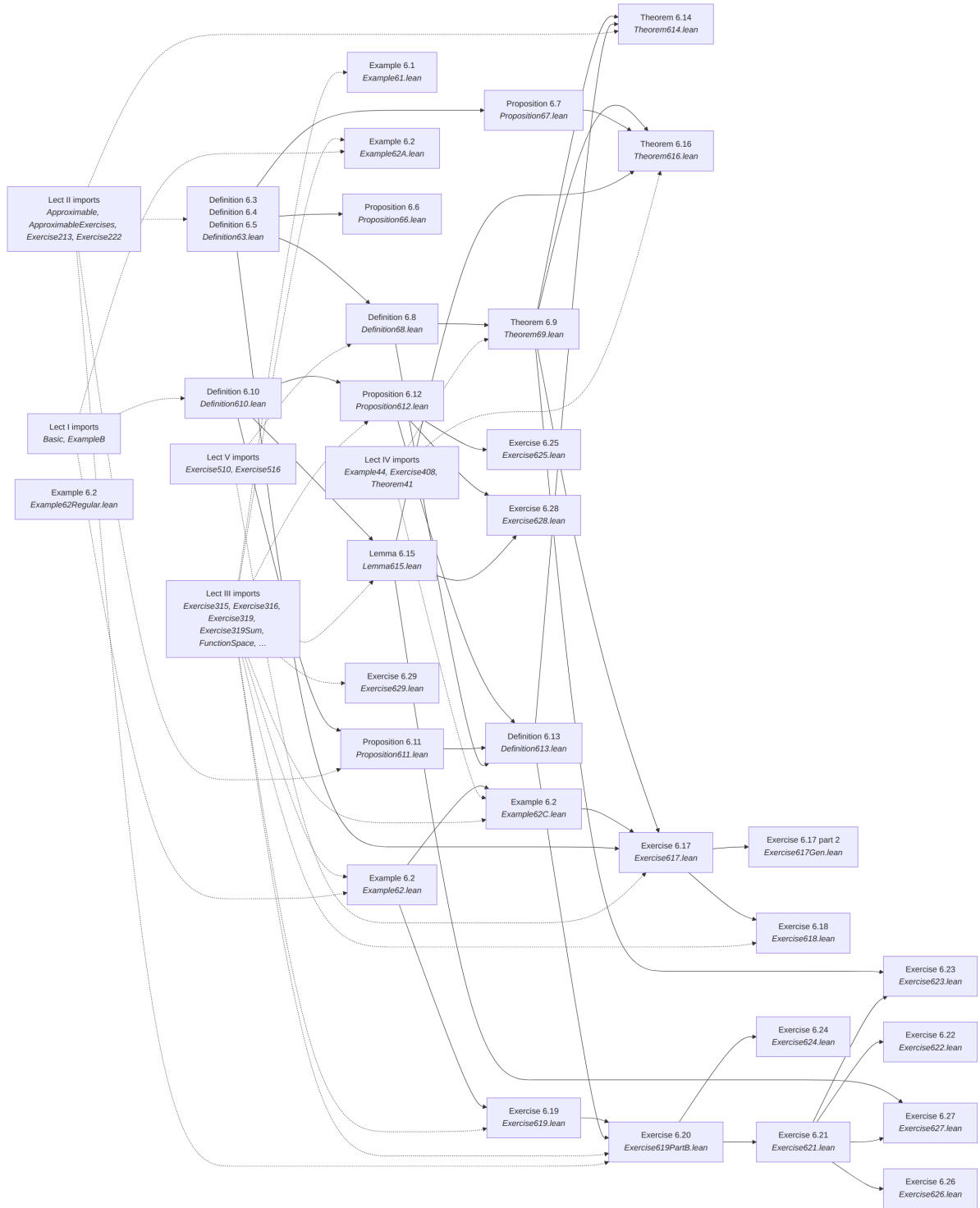
Definition 6.3

a *functor* T on the category of domains.

a small, self-contained `Category/Functor` abstraction, generic over an arbitrary category as Scott stresses, the witness instance on `DomainObj/ApproximableMap` (laws = Thm 2.5), and `Endofunctor` (the *endofunctor* of Def 6.3, with `map_id/map_comp`).

Lean 4 source

```
class Category (Obj : Type u) where
  Hom : Obj -> Obj -> Type v
  id : (X : Obj) -> Hom X X
  comp : {X Y Z : Obj} -> Hom Y Z -> Hom X Y -> Hom X Z
```



```

id_comp : forall {X Y : Obj} (f : Hom X Y), comp (id Y) f = f
comp_id : forall {X Y : Obj} (f : Hom X Y), comp f (id X) = f
assoc : forall {W X Y Z : Obj} (h : Hom Y Z) (g : Hom X Y) (f : Hom W X),

```

Definition 6.4

a *T*-algebra $T(E) \rightarrow E$.

structure `TAlgebra T` = a carrier object E with a structure map $\text{str} : T(E) \rightarrow E$; structure `AlgHom A B` = a morphism $\text{hom} : E \rightarrow F$ carrying the commuting-square field $\text{comm} : \text{hom} \odot A.\text{str} = B.\text{str} \odot T.\text{map hom}$.

Lean 4 source

```

class Category (Obj : Type u) where
  Hom : Obj -> Obj -> Type v
  id : (X : Obj) -> Hom X X
  comp : {X Y Z : Obj} -> Hom Y Z -> Hom X Y -> Hom X Z
  id_comp : forall {X Y : Obj} (f : Hom X Y), comp (id Y) f = f
  comp_id : forall {X Y : Obj} (f : Hom X Y), comp f (id X) = f
  assoc : forall {W X Y Z : Obj} (h : Hom Y Z) (g : Hom X Y) (f : Hom W X),

```

Definition 6.5

an *initial* *T*-algebra.

structure `IsInitial A` bundles the existence datum $\text{desc} : (B : TAlgebra T) \rightarrow \text{AlgHom A B}$ with the uniqueness field $\text{uniq} : \forall B (h : \text{AlgHom A B}), h = \text{desc B}$ — a *unique* homomorphism into every algebra.

Lean 4 source

```

class Category (Obj : Type u) where
  Hom : Obj -> Obj -> Type v
  id : (X : Obj) -> Hom X X
  comp : {X Y Z : Obj} -> Hom Y Z -> Hom X Y -> Hom X Z
  id_comp : forall {X Y : Obj} (f : Hom X Y), comp (id Y) f = f
  comp_id : forall {X Y : Obj} (f : Hom X Y), comp f (id X) = f
  assoc : forall {W X Y Z : Obj} (h : Hom Y Z) (g : Hom X Y) (f : Hom W X),

```

Proposition 6.6

any two initial *T*-algebras are uniquely isomorphic.

the textbook diagram chase. For initial A, B , initiality gives unique homs each way; the helper `'comp_desc_eq_id hA hB : (hB.desc A).comp (hA.desc B)`

Lean 4 source

```

theorem comp_desc_eq_id (hA : IsInitial A) (hB : IsInitial B) :

```

Proposition 6.7

$i : T(D) \rightarrow D$ initial $\implies T(i)$ initial and i is an isomorphism.

formalises the decisive half — the structure map of an initial algebra is an iso. With $A = (D, i)$, the functor builds $\text{tStr A} = (T(D), T(i))$ and `'strHom A : (T(D), T(i))`

Lean 4 source

```

theorem str_comp_desc (A : TAlgebra T) (hA : IsInitial A) :

```

Definition 6.8

a functor *continuous on maps*.

`ContinuousOnMaps T` — for all domains D, E the induced action $\lambda f. T(f)$ on Scott's **strict** function space is approximable.

Lean 4 source

```
theorem continuousOnMaps_id : ContinuousOnMaps (idEndofunctor DomainObj)
```

Theorem 6.9

continuous T with $D \cong T(D) \implies$ a homomorphism $D \rightarrow E$ to any T -algebra.

```
'nonempty_algHom_of_continuousOnMaps (hT : ContinuousOnMaps T) (iso : Iso (T.obj
D))
```

Lean 4 source

```
theorem nonempty_algHom_of_continuousOnMaps
  (T : Endofunctor DomainObj) (hT : ContinuousOnMaps T)
  {D : DomainObj} (iso : Iso (T.obj D) D)
  (B : TAlgebra T) (hk : IsStrict B.str) :
```

Definition 6.10

the subsystem relation $D \triangleleft E$.

structure Subsystem $D E$ bundles $\text{sub} (D \subseteq E : D.\text{mem } X \rightarrow E.\text{mem } X)$, and the essential inter_closed (consistency is inherited from $E : D.\text{mem } X \rightarrow D.\text{mem } Y \rightarrow E.\text{mem } (X \cap Y) \rightarrow D.\text{mem } (X \cap Y)$).

Lean 4 source

```
theorem refl (D : NeighborhoodSystem a) : D ? D where
```

Proposition 6.11

the subsystems of E form a domain.

$\text{subsystemReprIso} : \{D // D \triangleleft E\} \simeq_o |\text{reprSystem} (\text{subFam } E) \dots|$ — the set of subsystems $\{D \mid D \triangleleft E\}$, ordered by $\triangleleft$, *forms a domain* (Scott's one-line corollary of the directed-union remark).

Lean 4 source

```
theorem subFam_sInter_mem (E : NeighborhoodSystem a) (? : Set (Set (Set a)))
  (hne : ?.Nonempty) (h? : ? subseteq subFam E) : ?_0 ? in subFam E :=
```

Proposition 6.12

$D \triangleleft E \implies$ a projection pair i, j .

$i = \text{Subsystem.inj } h : D \rightarrow E$ (element-wise $i(x) = \{Y \in E \mid \exists X \in x, X \subseteq Y\} = \text{toElementMap_inj}$) and $j = \text{Subsystem.proj } h : E \rightarrow D$ ($\text{rel } Y \text{ j } X \leftrightarrow E.\text{mem } Y \wedge D.\text{mem } X \wedge Y \subseteq X$, element-wise $j(y) = y \cap D = \text{toElementMap_proj}$).

Lean 4 source

```
theorem toElementMap_inj (h : D ? E) (x : D.Element) {Y : Set a} :
```

Definition 6.13

a functor *monotone / continuous on domains*.

$\text{MonotoneOnDomains } T := \forall D E, D \triangleleft E \rightarrow T(D) \triangleleft T(E)$ but the pair i, j of 6.12 is mapped to $T(i), T(j)$: carrier_eq (the two image carriers $(T.\text{obj } \langle \alpha, E \rangle).\text{carrier}, (T.\text{obj } \langle \alpha, D \rangle).\text{carrier}$ coincide — needed since the abstract T may change token type, so $T(D) \triangleleft T(E)$ only typechecks once carriers agree), sub (the transported $T(D) \triangleleft T(E)$), and inj_heq/proj_heq (the canonical 6.12 pair of sub is $(T.\text{map } h.\text{inj}, T.\text{map } h.\text{proj})$, up to the carrier transport — hence HEq).

Lean 4 source

```
theorem monotoneOnDomains_id : MonotoneOnDomains (idEndofunctor DomainObj.{w})
```

Theorem 6.14

(main) continuous monotone T with a generating set $\Gamma \implies \text{solution } D \cong T(D)$.

hypotheses bundled in **Setup** (T continuous on maps + monotone + continuous on domains, generating system Γ with ceq/hsub realizing $\{\Gamma\} \triangleleft T(\{\Gamma\})$). The **iterated-functor tower** $\text{iter}/\text{Dsys}/\text{Dceq}/\text{Dchain}$ builds $T^n(\{\Gamma\})$ over the common token type Tok (carrier-type juggling via the choice-free transport lemmas $\text{subsystem_cast}/\text{rec_trans}/\text{mem_cast}/\text{set_rec_trans}$ and $\text{MonotoneAt.carrier_eq}$), with $\text{Dsys_master}/\text{chain_le}$ (the chain $T^n \triangleleft T^m$).

Lean 4 source

```
structure Setup where
  T : Endofunctor DomainObj.{w}
  hmaps : ContinuousOnMaps T
  hmono : MonotoneOnDomains T
  hcont : ContinuousOnDomains T
```

Lemma 6.15

projection pair i, j with $j \circ i = \text{id}_D$, $i \circ j \sqsubseteq \text{id}_E \implies D \leq E$ (converse to 6.12).

$\text{'trianglelefteq_of_projectionPair } (i : D \rightarrow E) (j : E \rightarrow D)$

Lean 4 source

```
theorem trianglelefteq_of_projectionPair (hji : j.comp i = idMap D)
  (hij : i.comp j <= idMap E) : D ? E :=
```

Theorem 6.16

initial T -algebra $D \implies D \leq E$ for any $E \cong T(E)$.

$\text{'trianglelefteq_of_isInitial } (hT : \text{ContinuousOnMaps } T) (\text{Dalg})$

Lean 4 source

```
theorem trianglelefteq_of_isInitial
  (T : Endofunctor DomainObj.{w}) (hT : ContinuousOnMaps T)
  (Dalg : TAlgebra T) (hinit : IsInitial Dalg)
  (E : DomainObj.{w}) (isoE : Iso (T.obj E) E) :
```

Exercise 6.17

algebras for which C is initial.

Pass (both parts) (`Exercise617.lean`, `Exercise617Gen.lean`): C is the initial T -algebra for $T(X) = \mathbb{K} + X + X$ ($\text{CisInitial} : \text{IsInitial Calg}$). A bespoke $\emptyset$ -free category `StrictDomainObj` (token type + system + $\emptyset \notin \mathcal{D}$; morphisms = `StrictMap`)

Lean 4 source

```
structure StrictDomainObj : Type (w + 1) where
  carrier : Type w
  sys : NeighborhoodSystem carrier
  nonempty : forall X, sys.mem X -> X.Nonempty
```

Exercise 6.18

D_∞ (Ex 3.16) as an initial algebra / domain-equation solution.

D^∞ is the **initial algebra** of the product endofunctor $T(X) = \mathcal{D} \times X$ for a fixed $\emptyset$ -free $\mathcal{D}$; the **domain-equation half** $D^\infty \cong \mathcal{D} \times D^\infty$ is Exercise 3.16 (`iter_isomorphic/iterProdIso`). Done in the bespoke `StrictDomainObj` category of Ex 6.17 (where `IsInitial` is Scott's universal property among strict algebras).

Lean 4 source

```
theorem prod_nonempty {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall X, V_0.mem X -> X.Nonempty) (h_1 : forall Y, V_1.mem Y -> Y.Nonempty)
  :
```

Exercise 6.19

sum & product on the category of strict maps.

Pass (both parts) (Exercise619.lean, Exercise619PartB.lean): Scott’s uniform token-level sum/product over $\Delta \subseteq \{0,1\}^*$ ($\Lambda = \square$, $\emptyset \notin \mathcal{D}$) and the answer to “correct up to isomorphism?” — **yes**.

Lean 4 source

```
def sumTokEquiv : (sumTok D_0 D_1 h_0 h_1).Element ?o (sum D_0 D_1 h_0 h_1).Element where
```

Exercise 6.20

the tok(D) function on systems.

$\lambda\Gamma. \text{tok}(T(\{\Gamma\}))$ is continuous on $\{\Gamma \subseteq \{0,1\}^* \mid \Lambda \in \Gamma\}$, hence a fixed point $\Gamma = \text{tok}(T(\{\Gamma\}))$ exists and $\{\Gamma\} \triangleleft T(\{\Gamma\})$ (so Thm 6.14 applies), for any T from 6.19.

Lean 4 source

```
def mFun : FExpr -> Set Str -> Set Str
```

Exercise 6.21

functors generated by the operations.

extends 6.19B/6.20 with the *coalesced* sum $\oplus$ and *smash* product $\otimes$, and generalizes all of $+, \times, \oplus, \otimes$ to several terms. **Objects:** $\text{oplusTok } D_0 \ D_1 \ h_0 \ h_1$ ($= \text{sumTok}$ with the improper copies $0\Delta_0, 1\Delta_1$ deleted: ‘ $\text{mem } W := W = M \vee (\exists X \in \mathcal{D}_0, X \neq \Delta_0, W = 0X)$ ’)

Lean 4 source

```
theorem oplusTok_nonempty : forall W, (oplusTok D_0 D_1 h_0 h_1).mem W -> W.Nonempty
```

Exercise 6.22

comment on given domain equations.

the “comment on” exercise, formalized as recognising each of the three equations as an instance of the 6.21/6.20 fixed-point machinery, so each has a solution ($\Gamma = \text{tok}(T(\{\Gamma\}))$, $\{\Gamma\} \triangleleft T(\{\Gamma\})$, Thm 6.14 applies). **Constants:** $\text{Cnat} = \{\{0\}, \{0, \Lambda\}\}$ — the two-point chain $\{0\} \sqsubset \Delta$ ($0 = [\text{false}]$, $\Lambda = \square$), built directly with nested-pair inter_mem via $\text{inter_eq_self_of_subset_left/right}$, $\emptyset$ -free + rooted (nil_mem_Cnat); $\text{Cone} = \text{singletonSys } \{\Lambda\}$ = the one-point $\not\models$ (nil_mem_Cone).

Lean 4 source

```
theorem nil_mem_Cnat : ([ ] : Str) in Cnat.sys.master
```

Exercise 6.23

the initial solution to a domain equation.

Pass — all 4 phases (Exercise623.lean, namespace Domain.Neighborhood.Exercise619): the *concrete solution domain* Exp for $\text{Exp} \cong N \oplus ((\text{Exp} \times \text{Exp}) + (\text{Exp} \times \text{Exp}))$. Functor $\text{Tex}p \ N = \oplus(\text{const } N, +(\times(\text{var}, \text{var}), \times(\text{var}, \text{var})))$ as a GExpr (Ex 6.21).

Lean 4 source

```
def Exp (N : ScottSys) (hN : ([ ] : Str) in N.sys.master) : ScottSys :=
```

Exercise 6.24

existence of domains satisfying given equations.

the **double fixed-point** method for the coupled system $D \cong D+(D \times E)$, $E \cong D+E$. **Tokens decided:** both D, E are $\emptyset$ -free systems over the single type $\text{Str}=\{0,1\}^*$ (Ex 6.19's uniform category).

Lean 4 source

```
theorem gTok_mono {p p' q q' : Set Str} (hp : p ⊆ p') (hq : q ⊆ q') : gTok p
  q ⊆ gTok p' q'
```

Exercise 6.25

projection-pair g, h identities on elements.

Exercise625.lean, ns Subsystem.ProjectionPair. Galois conn.

Lean 4 source

```
def lowerSet (P : ProjectionPair D E) (y : E.Element) : Set D.Element :=
```

Exercise 6.26

the lifting $\mathcal{D}_\perp$ over $\{0,1\}^*$.

Exercise626.lean, ns Exercise619. $\text{liftTok } D_hD = \{\{\Lambda\} \cup 0\Delta\} \cup \{0X \mid X \in D\}$ (master $\text{liftTokMaster} = \text{insert } [] (0\Delta)$, proper $0X = \text{embBit false } X$), $\emptyset$ -free (liftTok_nonempty), packaged ScottSys.lift .

Lean 4 source

```
theorem liftTok_nonempty (hD : forall X, D.mem X -> X.Nonempty) :
```

Exercise 6.27

which subsystem relationships hold.

Exercise627.lean, ns Exercise627. **Verdict: first five hold for all $\mathcal{D}, \mathcal{E}$; the sixth $\mathcal{D} \trianglelefteq \mathcal{D} \otimes \mathcal{E}$ fails in general.** $\trianglelefteq$ is Lemma 6.15's *embeds-as-subdomain* (Triangleleftteq); concrete $\{0,1\}^*$ constructors $\text{sumTok}/\text{prodTok}/\text{oplusTok}/\text{otimesTok}$ (Ex 6.19/6.21)

Lean 4 source

```
theorem otimesTok_subsystem_prodTok (D_0 D_1 : NeighborhoodSystem Str) :
```

Exercise 6.28

(Plotkin) finite systems D, E .

Exercise628.lean, ns Domain.Neighborhood. **Finite Cantor–Schröder–Bernstein:** if $|\mathcal{D}|, |\mathcal{E}|$ finite and $\mathcal{D} \trianglelefteq \mathcal{E} \trianglelefteq \mathcal{D}$ then $\mathcal{D} \cong^D \mathcal{E}$ ($\text{isomorphic_of_triangleleftteq_both}$); faithful “finite system” = finitely many nbhds version is $\text{isomorphic_of_finite_system}$ ($\text{hyps NeighborhoodSystem.IsFinite}$). $:= \text{Finite } \{X // D.\text{mem } X\}$, via $\text{finite_element_of_isFinite}$: $x \mapsto \{p \mid x.\text{mem } p.1\}$ injects $|\mathcal{D}|$ into $\text{Set } \{X // D.\text{mem } X\}$.

Lean 4 source

```
theorem isomorphic_of_triangleleftteq_both [Finite D.Element] [Finite E.Element]
```

Exercise 6.29

generalize $+$, $\times$ to infinitary operations.

Exercise629.lean, ns Exercise629. **Verdict: $+$, $\times$, $\oplus$ all generalize to an index family $D : \forall i, \mathcal{D}_i$ over αi ; $\otimes$ does NOT (infinite smash degenerates).** Tokens: $\Sigma i, \alpha i$ (product-like)

Lean 4 source

def iprod : NeighborhoodSystem (Sigma i, a i) where

3.7 Lecture VII: Computability in Effectively Given Domains

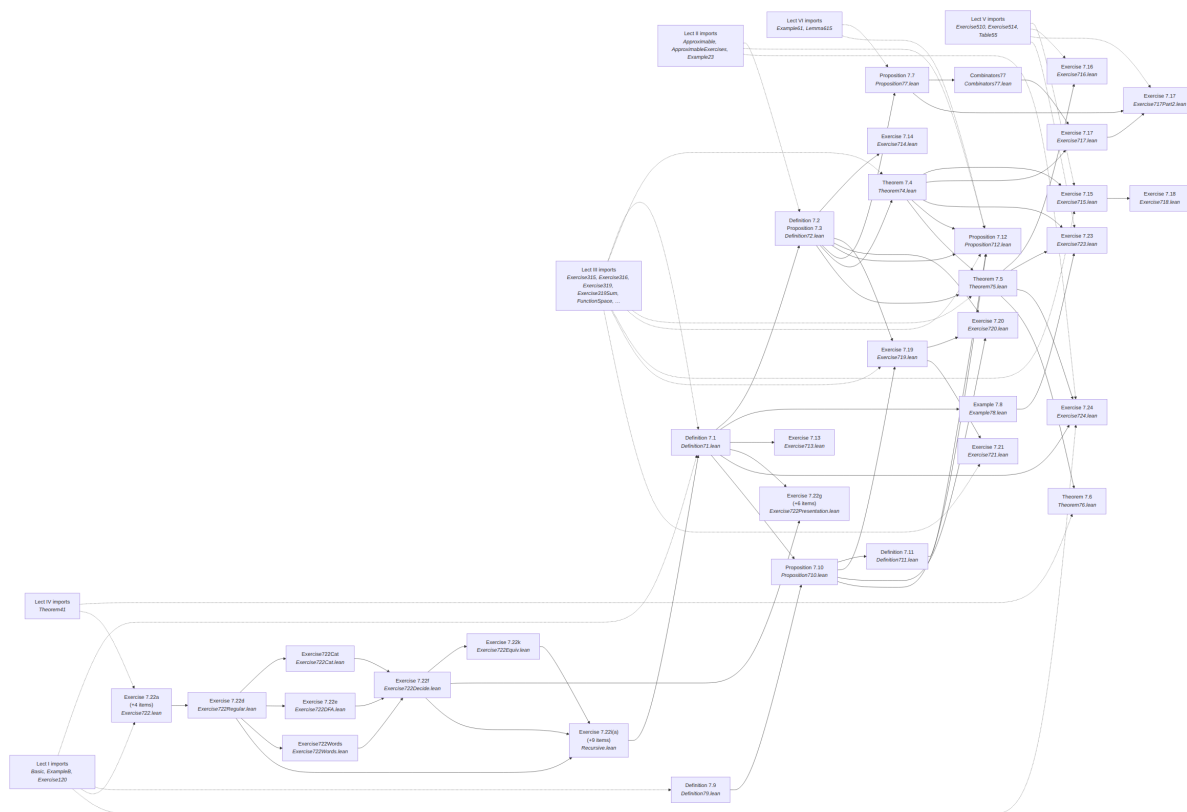


Figure 8: Lean module dependencies for Lecture VII: Computability in Effectively Given Domains.

Lecture VII establishes the recursion-theoretic foundations of domain theory.

Definition 7.1

a *computable presentation* of a neighbourhood system.

Definition71.lean (+ Recursive.lean), ns Domain.Neighborhood. ComputablePresentation
V: enumeration $X: \mathbb{N} \rightarrow \text{Set } \alpha$ with mem_X (each $X_n \in \mathcal{D}$)

Lean 4 source

```
structure ComputablePresentation (V : NeighborhoodSystem a) where
  X : Nat -> Set a
  mem_X : forall n, V.mem (X n)
  surj : forall {Y : Set a}, V.mem Y -> exists n, X n = Y
  interEq_computable : RecDecidable_3 (fun n m k => X n ∩ X m = X k)
  cons_computable : RecDecidable_2 (fun n m => exists k, X k subseq X n ∩ X m)
```

Definition 7.2

computable map between recursively presented domains.

Definition72.lean, ns Domain.Neighborhood. ****IsComputableMap** P Q f := REPred₂
(fun n m ↦ f.rel (X_n))

Lean 4 source

```
theorem idMap_isComputable (P : ComputablePresentation V) :
```

Proposition 7.3

identity is computable; computable maps compose.

Definition72.lean, ns Domain.Neighborhood. Both halves + Scott's stated consequence.

Lean 4 source

```
theorem idMap_isComputable (P : ComputablePresentation V) :
```

Theorem 7.4

$D_0 + D_1$ and $D_0 \times D_1$ are effectively given if D_0, D_1 are.

Theorem74.lean, ns Domain.Neighborhood. **Product half (done)**: prodPresentation $P_0 P_1$ is a ComputablePresentation of prod $V_0 V_1$ (over $\alpha \oplus \beta$, Product.lean)

Lean 4 source

```
theorem prod_isEffectivelyGiven (h_0 : V_0.IsEffectivelyGiven) (h_1 : V_1.IsEffectivelyGiven) :
```

Theorem 7.5

$(D_0 \rightarrow D_1)$ is effectively given; eval/curry computable; computable elements = computable maps.

Theorem75.lean (+ Recursive.lean), ns Domain.Neighborhood. **Math core (Prop 3.9(i), choice-free)**: a function-space nbhd $\bigcap [X_i, Y_i]$ is non-empty iff for every sublist-selection whose inputs share a lower nbhd in $\mathcal{D}_0$ the selected outputs are consistent in $\mathcal{D}_1$; modelled over **coded entry-lists** (funPair $P_0 P_1$ $e = (X_0.\{e.1\}, Y_1.\{e.2\})$, funListOf, stepFun_funListOf_nonempty_ iff), with the reverse built via leastMap + the **choice-free** $\mathcal{D}_0$ -inclusion test ($P_0.incl_computable.em$, replacing the library rel_interYs's classical by_cases $X \subseteq X_i$).

Lean 4 source

```
def funListOf (el : List Nat) : List (Set a x Set b)
```

Theorem 7.6

$\text{fix} : (D \rightarrow D) \rightarrow D$ is computable on effectively given D .

Theorem76.lean, ns Domain.Neighborhood. **fixMap_isComputable**: relative to the function-space presentation funPresentation $P P \dots$ (Theorem 7.5)

Lean 4 source

```
theorem fixMap_isComputable
  (gN incl eq : Nat -> Nat)
  (hgN : forall c, gN c = 1 <-> (stepFun (funListOf P P (decodeList c))
    : Set (ApproximableMap V V)).Nonempty) (hgNp : Nat.Primrec gN)
  (hincl : forall s, incl s = 1 <-> P.X s.unpair.1 subseq P.X s.unpair.2) (hincp :
    Nat.Primrec incl)
  (heq : forall s, eq s = 1 <-> P.X s.unpair.1 = P.X s.unpair.2) (heqp : Nat.Primrec eq) :
```

Proposition 7.7

$D_\S$ is effectively given; the Example 6.1 combinators are computable.

Proposition77.lean + Combinators77.lean, ns Domain.Neighborhood.Proposition77, green, wired. $\lambda x.$

Lean 4 source

```
theorem primrec_dsharpStep (hfcons : Nat.Primrec fcons) (hfeq : Nat.Primrec feq)
```

Example 7.8

the powerset PN is effectively given.

Example78.lean, ns Domain.Neighborhood.Example78, green, wired. Scott's finite-set enumeration $E_n = \{k \mid \exists i, j.$

Lean 4 source

```
def PN : NeighborhoodSystem Nat where
```

Definition 7.9

the power domain PD.

Definition79.lean, ns Domain.Neighborhood.NeighborhoodSystem, green, wired. The **Smyth power domain** $\mathbb{PD}$: Scott's down-set $\downarrow X = \{Y \in \mathcal{D} \mid Y \subseteq X\}$ is *exactly* Exercise 1.20's `upSet`, and the preparation $\mathcal{D}^\dagger = \{\downarrow X \mid X \in \mathcal{D}\}$ is *exactly* Ex 1.20's `powerSystem` (aliased `dagger`; `dagger_isomorphic :  $\mathcal{D} \cong \mathcal{D}^\dagger$` reuses `isomorphic_powerSystem`).

Lean 4 source

```
abbrev dagger : NeighborhoodSystem (Set a)
```

Proposition 7.10

PD is a neighbourhood system, effectively given if D is.

Proposition710.lean, ns Domain.Neighborhood.NeighborhoodSystem, green, wired. **Part A** `PowerDomain : NeighborhoodSystem (Set  $\alpha$ )` (`mem := PDmem`, `master :=  $\downarrow \Delta$`).

Lean 4 source

```
theorem PDmem_inter {W_1 W_2 : Set (Set a)} (h_1 : V.PDmem W_1) (h_2 : V.PDmem W_2) :
```

Definition 7.11

finite-element joins $\{x_0, \dots, x_{n-1}\}$ in the power domain.

Definition711.lean, ns Domain.Neighborhood.NeighborhoodSystem, green, wired. Scott's $\{x_0, \dots, x_{n-1}\} = \{z \in \mathbb{PD} \mid \exists X_i \in x_i.$

Lean 4 source

```
def PDfinJoinSucc {m : Nat} (xs : Fin (m + 1) -> V.Element) : V.PowerDomain.Element where
```

Proposition 7.12

the union mapping on the power domain.

Pass (**A/B/D proved; $C = D \leq \mathbb{PD}$ REFUTED with formalized counterexample**) — Proposition712.lean, ns Domain.Neighborhood.NeighborhoodSystem (+ counterexample ns Domain.Neighborhood.Counterexample712C), green, wired, zero sorry. **Part A:** `PDsingletonApproxMap` (`ofMono` on $\uparrow X \mapsto \{\uparrow X\}$)

Lean 4 source

```
def PDsingletonApproxMap : ApproximableMap V V.PowerDomain :=
```

Exercise 7.13

effectively given domain $\leftrightarrow$ an `INCL(n,m)` relation on integers.

Exercise713.lean, ns Domain.Neighborhood.Exercise713, green, wired, zero sorry.

Lean 4 source

```
structure InclStructure where
  INCL : Nat -> Nat -> Prop
  meetIdx : Nat -> Nat -> Nat
```

```

topIdx : Nat
incl_dec : RecDecidable_2 INCL
cons_dec : RecDecidable_2 (fun n m => exists k, INCL k n /\ INCL k m)
meet_dec : RecDecidable_3 (fun n m k => forall j, INCL j k <=> (INCL j n /\ INCL j m))
meetIdx_primrec : Nat.Primrec (fun t => meetIdx t.unpair.1 t.unpair.2)
incl_refl : forall n, INCL n n
incl_trans : forall {n m k}, INCL n m -> INCL m k -> INCL n k
topIdx_spec : forall n, INCL n topIdx
meetIdx_spec : forall {n m}, (exists k, INCL k n /\ INCL k m) ->

```

Exercise 7.14

(recursion theorists) r.e. facts after Def 7.2; computable elements.

Exercise714.lean, ns Domain.Neighborhood.Exercise714, green, wired, zero sorry. ****Half**
1 — “non-empty r.e.

Lean 4 source

```

theorem reprod_range_primrec {r : Nat -> Nat} (hr : Nat.Primrec r) :

```

Exercise 7.15

finish 7.4 for $D_0 \otimes D_1$, $D_0 \oplus D_1$, D_∞ .

Exercise715.lean, ns Domain.Neighborhood, green, wired, zero sorry. All three constructs effectively given.

Lean 4 source

```

structure ScottPresentation (V : NeighborhoodSystem a) where
  X : Nat -> Set a
  mem_X : forall n, V.mem (X n)
  surj : forall {Y : Set a}, V.mem Y -> exists n, X n = Y
  interEq_computable : RecDecidable_3 (fun n m k => X n ∩ X m = X k)
  cons_computable : RecDecidable_2 (fun n m => exists k, X k ⊆ X n ∩ X m)

```

Exercise 7.16

curry as a neighbourhood relation: recursive or r.e.?

Exercise716.lean, ns Domain.Neighborhood, green, wired, zero sorry, **fully choice-free**
 $\subseteq \{\text{proptext}, \text{Quot.sound}\}$ (**data and proofs**). ****Answer:** curry is a *recursive* (recursively decidable)

Lean 4 source

```

theorem curryComb_rel {G : Set (ApproximableMap (prod V_0 V_1) V_2)}
  {H : Set (ApproximableMap V_0 (funSpace V_1 V_2))} :

```

Exercise 7.17

finish 7.7 for $D\S$; strict $g:D\S \rightarrow E$.

Exercise717.lean (Part 1) + Exercise717Part2.lean (Part 2), ns Domain.Neighborhood/...Exercise717
green, wired, zero sorry.

Lean 4 source

```

def proj1Map : ApproximableMap (Dsharp D hD) (Dsharp D hD) where

```

Exercise 7.18

define *effective isomorphism*; effective $D_\infty \cong (D_\infty)_\infty$.

Exercise718.lean, ns Domain.Neighborhood.Exercise718, green, wired, zero sorry. **Part**
1 — “complete the sentence”: EffectiveIso $P \ Q =$ a pair of mutually inverse approximable maps $\text{toMap}:D \rightarrow E$, $\text{invMap}:E \rightarrow D$, **both computable** (IsComputableMap, Def 7.2), with

$\text{invMap} \circ \text{toMap} = \text{I_D}$, $\text{toMap} \circ \text{invMap} = \text{I_E}$; $\text{EffectivelyIsomorphic } P \ Q := \text{Nonempty } (\text{EffectiveIso } P \ Q)$.

Lean 4 source

```
def Fmap (V : NeighborhoodSystem a) : ApproximableMap (iterSys V) (iterSys (iterSys V)) where
```

Exercise 7.19

$D \rightarrow PD$ is a functor.

`Exercise719.lean`, `ns Domain.Neighborhood`, `green`, `wired`, `zero sorry`. Defines `PFmap`
 $f : PD \rightarrow PE$ (rep-independent $\text{rel } A \ B := PD\text{mem } A \wedge PD\text{mem } B \wedge \forall X \in A, \exists Y \in B, X \ f \ Y$), full
`ApproximableMap` (`master_rel/inter_right` via `f.inter_right + downward-closure PDmem_down/mono`).

Lean 4 source

```
theorem PDmem_down {A : Set (Set a)} (hA : V.PDmem A) {Z X : Set a}
```

Exercise 7.20

a combinator of given type.

`Exercise720.lean`, `ns Domain.Neighborhood`, `green`, `wired`, `zero sorry`. The **flattening combinator** `union : P(PD) → PD` (Smyth power-domain monad multiplication μ).

Lean 4 source

```
theorem unionMap_rel_fin (V : NeighborhoodSystem a) {LS : List (List (Set a))} {LY : List (Set a)}  

(hLS : forall l in LS, forall X in l, V.mem X) (hLY : forall Y in LY, V.mem Y) :
```

Exercise 7.21

Scott's **Exercise 7.22** is split below into sub-rows **7.22a–h**, **7.22i(a)–i(b)**, **7.22j–l**. Composer sessions **C1–C8**, **C11**, **C12**, **C9a**, **C9b1–C9b8**, **C10**, and **C7b** delivered **7.22a–h**, **7.22i(a)**, **7.22i(b)1–8**, **7.22j**, and **7.22k**; **7.22l** (Scott's infinite-word equations, as genuine domain least fixed points) is also **Pass**, closing the inventory. Palomar compares `Ssys_embeds : Ssys ≤ U` (`PalomarExamples.lean`).

Lean 4 source

```
theorem papplyEval_step_witness {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}  

{X_0 : Set a} {Y_0 : Set b} (hX_0 : V.mem X_0) (hY_0 : W.mem Y_0) :
```

Exercise 7.22a

least-fixed-point family S over $\{0,1\}^* (\text{InS})$.

Inductive `InS` with generators `univ` ($\Sigma = \text{Set.univ}$), `singleton` σ , `mul` (`concat X Y`), `inter` (`non-empty` $\cap$). Bespoke `concat X Y = {a++b | a ∈ X, b ∈ Y}` with `concat_mono/concat_assoc/concat_singleton/concat_nonempty`.

Exercise 7.22b

S is a positive neighbourhood system.

`InS.nonempty` $\implies$ `Ssys : NeighborhoodSystem (List Bool)` via `ofPositive (master Δ = Σ = univ)`; `Ssys_isPositive` proved directly. `Green`, `zero sorry`, `choice-free`.

Exercise 7.22c

semigroup on $|S|$ and embedding of the free monoid.

`mulElem` ($xy = \{Z \in S \mid \exists X \in x \exists Y \in y, XY \subseteq Z\}$) a filter; `mulElem_assoc`.

Exercise 7.22d

regular-event syntax; decidable membership (Scott's hint).

`SExpr` (`sigma/single/cat/cap` — `no` $\cup$ `compl` $*$); `denote`; `matchesB/matchesB_iff` $\implies$ `decidableMemDenote`. `inS_iff_exists_denote`, `inS_eq_range_denote`.

Exercise 7.22e

automata recognition — every `SExpr` language is a `Fintype` automaton language.

Route A leaf DFAs (`sigmaDFA`, `singleDFA`, `inter/compl` choice-free); `catEps` + `catEps_accepts`; `toNFA`, `toNFA_accepts` : `(toNFA e).accepts = denote e. denote_eq_empty_iff` (reachability).

Exercise 7.22f

executable emptiness and consistency deciders (Def 7.1 (ii) on syntax).

on `SsysX` indices). Audit: `decideEmptyB_iff/consistentB_iff` $\subseteq$ **{propext, Classical.choice, Quot.sound}** (choice Prop-level only; Bool functions choice-free).

Exercise 7.22g

Gödel enumeration `SsysX` : $\mathbb{N} \rightarrow \text{Set (List Bool)}$ of S-members.

`SExpr.encode/decode`, `SsysX`, `SsysX_mem`, `SsysX_surj`. Green, zero sorry.

Exercise 7.22h

infinite-word equations (Scott's investigatory questions).

`streamElem` (`w-vec` as $\{Z \mid \text{InS } Z \wedge \forall n, w^n \in Z\}$), `powerLang`, `streamElem_powers_of_mul`, `streamElem_idempotent` (`w-vec` $\cdot$ `w-vec` = `w-vec` when `InS` (`powerLang w`)). example checks: empty word idempotent; triple product via `mulElem_assoc`; $\sigma^{++}[\text{true}]$ and 01 four-fold cases conditional on `InS` (`powerLang ...`).

Exercise 7.22i(a)

generic primitive-recursive closure for Bool/char deciders (Composer **C9a**).

`isBinDigit`, `allBinDigitsChar`, `primrec_isBinDigit`, `primrec_allBinDigitsChar` — $\{0,1\}$ validation over `decodeList` via existing `allListChar/**foldCode**`. Reused by `decodeFuel0kChar` tag-1 (**7.22i(b)1(b)**).

Exercise 7.22i(b)1(a)

$\{0,1\}$ AND — `mulBit`, `mulBit_eq_one_iff`, `primrec_mulBit`.

Used by `.cat/.cap` branches of `decodeFuel0kCharBody`. Green; `primrec_mulBit` $\subseteq$ **{propext, Quot.sound}**.

Exercise 7.22i(b)1(b)

fuel-bounded decode ok char — `decodeFuel0kChar`, `decodeFuel0kCharBody`, `primrec_decodeFuel0kChar`.

Tag dispatch via `primrec_tagCase4`; tag-0 uses `isOne` (`1 - u`) (matches `decodeFuel`); tag-1 reuses **7.22i(a)** `allBinDigitsChar`; tags 2/3 use **7.22i(b)1(a)** `mulBit`. Fuel induction on first argument.

Exercise 7.22i(b)1(c)

tag-dispatch infrastructure for correctness link — `decodeFuel0kCharBody_eq`, `selectFn_isOne_one_sub_sigma`, `isOne_one/isOne_zero/isOne_of_ne_one`.

@[simp] `isOne_one/isOne_zero + isOne_of_ne_one` (no global simp [isOne]). `decodeFuel0kCharBody_eq`: match `c.unpair.1` with concrete `Nat.sub` per tag ($0 \rightarrow \sigma\text{-flag}$, $1 \rightarrow \text{allBinDigitsChar}$, $2/3 \rightarrow \text{mulBit}$, else 0).

Exercise 7.22i(b)1(d)

list decode ok $\leftrightarrow$ bin-digit char — `decodeListBool_isSome_iff` (`(decodeListBool n).isSome = true` $\leftrightarrow$ `allBinDigitsChar n = 1`).

`mapM_natBool_isSome_iff`: induction on coded list with `List.mapM_cons` + case split on `natBool/**mapM**`. Links via `allBinDigitsChar_eq_one_iff`.

Exercise 7.22i(b)1(e)

shallow decode link — `decodeFuel0kChar_eq_one_iff` (`decodeFuel0kChar fuel c = 1` $\leftrightarrow$ (`decodeFuel fuel c`).`isSome = true`).

Fuel induction; `decodeFuel0kCharBody_eq` + match `c.unpair.1`. Tag 0: `selectFn_isOne_one_sub_sigma`; tag 1: **(d)** + `Option.isSome_map`; tags 2/3: `mulBit_eq_one_iff` + `decodeFuel_pair*_isSome_iff`; tag ≥ 4 : both false.

Exercise 7.22i(b)1

fuel-bounded decode ok flag — `decodeFuelOkChar + primrec_decodeFuelOkChar`, then shallow link $\leftrightarrow$ `decodeFuel.isSome`.

All sub-rows (a–e) Pass. `Char + primrec` in `Recursive.lean`; shallow link `decodeFuelOkChar_eq_one_iff` in `Exercise722Presentation.lean` (fuel induction + tag dispatch via (c), tag-1 via (d), tags 2/3 via `mulBit_eq_one_iff` + pair-isSome lemmas).

Exercise 7.22i(b)2

coded list length — `listLenChar + primrec_listLenChar`.

`listLenStp` increments accumulator; `listLenChar = foldCode listLenStp 0 0. listLenChar_eq: (decodeList c).length`.

Exercise 7.22i(b)3

coded list equality — `listEqChar + primrec_listEqChar`.

`natEqChar` (factored $\{0,1\}$ nat equality); synchronized `foldCode` over `c1` threading remainder-code of `c2` via `listEqStp/listEqStpNonzero` (no `reForallChar/tabCode` witness search). `listEqChar_eq_one_iff: foldCode_eq' + listEq_foldl_end_iff` (structural induction, same idiom as `allList_foldl_eq_one_iff`).

Exercise 7.22i(b)4

list append / take / drop on codes — `appendListCode, takeCode, dropCode + primrec`.

`tabCode/**nthCode**/**listLenChar**` (no `snoc/reverse fold`): `appendListTabFn` uses `isZero ((i+1)-len1)` branch + `nthCode`; `takeCode/dropCode` tabulate at `min n len/len-n`. Correctness via `tabCode_nth_lt/nthCode_eq` + choice-free `getD_take_cf/getD_drop_cf` (inductive).

Exercise 7.22i(b)5

numeric matchesB / state-card bound — `autStateCardFuelChar, matchesBChar + primrec`.

`primrec_tagCase4` fuel dispatch (tags 0–3); `autStateCardFuelChar` ($\sigma \rightarrow 1$, $\text{single} \rightarrow \text{listLenChar} + 2$, $\text{cat} \rightarrow \text{add}$, $\text{cap} \rightarrow \text{mul}$); `matchesBChar` ($\sigma \rightarrow 1$; $\text{single} \rightarrow \text{listEqChar}$; $\text{cat} \rightarrow \text{bExistsFn}$ over `takeCode/dropCode` cut points via `matchesBCatG+mulBit`; $\text{cap} \rightarrow \text{mulBit}$ on packed subcode+word); local `c9b5_sexprGodelEncode/c9b5_sexprDepth` (Presentation link deferred). Correctness `autStateCardFuelChar_eq_autStateCard, matchesBChar_eq_one_iff` by `SExpr` induction.

Exercise 7.22i(b)6

emptiness / cap consistency chars — `decideNonemptyBChar, consistentBChar + primrec`.

Bounded *index* search, not a materialized `wordsUpToCode` (avoids the `map/flatMap`-over-coded-list combinator that sank **C9b3**’s first attempt). `codeBound n (0  $\mapsto$  1,  $n+1 \mapsto \text{pair } 1 (\text{codeBound } n)+1$ )`

Exercise 7.22i(b)7

index-level consistency char — `ssysActiveChar, ssysConsistentBChar + shallow Bool` links.

`Recursive.lean`’s `C9b5/C9b6` correctness theorems are stated against a **private-file-local** Gödel mirror (`c9b5_sexprGodelEncode/c9b5_sexprDepth`), not literally `SExpr.encode/sexprDepth` (`Recursive.lean` cannot import `Exercise722Presentation.lean` — would cycle via `Presentation  $\rightarrow$  Definition71  $\rightarrow$  Recursive`). **un-privates** the four small mirror defs (`c9b5_boolNat/c9b5_encodeListBool/c9b5_sexprDepth/c9b5_sexprGodelEncode`)

Exercise 7.22i(b)8

- `fuelTable/fuelTableStep` (generic): tabulates a fuel-recursive $\{0,1\}$ -family’s values on $[0, \text{bound}]$ as a coded list (`tabCode/nthCode`), iterated via `Nat.rec` on `fuel` — mirrors `tabCode`’s own `Nat.Primrec.prec` packaging (`C9b4`). `fuelTable_eq_of_recursion`: correctness given (a) a table-lookup-based `bodyLookup` faithfully implementing the recursive step, and (b) a **locality** hypothesis (the step’s own recursive calls at code `c` never exceed `c`). `primrec_fuelTable`: joint `Nat.Primrec` via `Nat.Primrec.prec`.

- Instantiated for `decodeFuelOkChar` and `autStateCardFuelChar` directly (`decodeFuelOkCharBody/autStateCardFuelCharBody` only recursive calls are `Nat.unpair` sub-projections, always $\leq c$ — new lemma `unpair_left_le`, paired with existing `unpair_snd_le`).

- `matchesBChar` was harder: its cat-branch recursive calls are at `pair a (takeCode i cw)/pair b (dropCode i cw)` — the word half is a *derived* code, not a raw `Nat.unpair` projection. New lemmas `encodeList_take_le/encodeList_drop_le` (prefix/suffix codes never exceed the full code, via `Nat.pair`’s monotonicity in the second argument) give `takeCode_le/dropCode_le`; combined with new `pair_le_pair/pair_le_pair_left/pair_le_pair_right` (weak monotonicity, both/either argument) for the locality hypothesis, plus `bExistsFn_congr` (`bExistsFn` depends on `g` only via its values on the search range) and `eq_of_le_one_iff_one` (two $\{0,1\}$ -bounded naturals agreeing on $=1$ are equal, to bridge two *differently-packed* but pointwise-equal `bExistsFn` calls).
- `decideNonemptyBChar/consistentBChar` needed no new course-of-values work (built from the now-joint `matchesBChar/autStateCardFuelChar` via `bExistsFn/codeBound`) — just a new `primrec_bExistsFn_param` (parametrized `bExistsFn`: `g` may depend on an external `fuel` held fixed throughout the search, packed alongside `bExistsFn`’s own `n`) to thread `fuel` through without needing `decideNonemptyBChar`’s C9b6 definition to change.
- With all five jointly primitive recursive, `primrec_ssysActiveChar/primrec_ssysConsistentBChar` compose directly (C9b7’s definitions, unchanged); `ssysConsChar_eq_ssysConsistentBChar` (via `eq_of_le_one_iff_one` + the C9b7/C9b8 `_eq_one_iff/_le_one` facts) bridges `ssysConsChar` (built from the real `ssysConsistentB`) to `ssysConsistentBChar`, giving `primrec_ssysConsChar` via `.of_eq`; `Ssys_cons_computable := Ssys_cons_computable_of_primrec_ssysConsChar primrec_ssysConsChar` closes C9.
- Two pre-existing-name collisions surfaced once the new lemmas were made public and reachable via `open Domain.Recursive` elsewhere (`unpair_fst_le/pair_le_pair_right` already existed independently in `Proposition77.lean/Exercise717Part2.lean`); renamed to `unpair_left_le/pair_le_pair_right` to disambiguate.
- $\subseteq \{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$ (choice inherited from the list-extensionality layer, same as every other C9b slice). Depends on 7.22i(b)7.

Exercise 7.22i(b)

primitive-recursive certification of consistency — `RecDecidable2` for Def 7.1 (ii) (Composer C9b umbrella).

Umbrella closes: sub-rows 7.22i(b)1–8 are all **Pass**. `Ssys_cons_computable : RecDecidable2` (`fun n m => ∃ k, SsysX k ⊆ SsysX n ∩ SsysX m`) — Scott’s Definition 7.1 (ii) consistency relation on the `SsysX` enumeration is recursively decidable, choice-free save for the inherited list-extensionality `Classical.choice`.

Exercise 7.22j

`ComputablePresentation Ssys / Ssys.IsEffectivelyGiven` (Def 7.1 packaging).

— $X_n \cap X_m = X_k$, i.e. whether two *different* syntactic caps denote the *same* language.

Exercise 7.22k

relation (i) — `interEq` / regular-language equivalence decider on indices.

C7a documented the gap: emptiness insufficient (`sigma_ne_containsZero`); (i) = language equivalence.

Exercise 7.22l

Scott’s infinite-word equations (`sigma-vecsigma-vec=sigma-vec, sigma-vecsigma-vecsigma-vec=sigma-vecsigma-vec1-vecsigma-vec1-vec=sigma-vec1-vec, 01-vec4=01-vec2`), `sigma-vec` defined by least fixed point `sigma-vec=σsigma-vec`.

The earlier framing (power-filter `streamElem w := {Z | InS Z ∧ ∀ n, wn ∈ Z}`, conditional on the side-question `InS (powerLang w)` — is $\{w^n\}$ itself in S ?) turned that side-question into a genuinely open combinatorics-on-words problem (kept, unresolved, as `streamElem/powerLang`, but it is **not** part of Scott’s actual question — an artefact of that proxy construction).

Exercise 7.23

finish PN (Ex 7.8): `fun/graph` (Ex 5.14) computable; $\cap/\cup/+$ computable; characterize the computable elements of PN.

All four parts done and audited choice-free. $\text{nbhd } n \subseteq \text{nbhd } k \leftrightarrow \text{myLor } n \ k = n$ (`nbhd_subset_iff_myLor_eq`) is the master reduction: every binary combinator here tests $E_k \subseteq h(E_n, E_m)$, i.e.

Lean 4 source

```
theorem nbhd_subset_iff_myLor_eq (n k : Nat) : nbhd n subseteq nbhd k <-> myLor n k = n
```

Exercise 7.24

define Γ (finite/infinite sequences of naturals) and its neighbourhood system L ; show L effectively given; identify $|L|$ with Γ ; relate L to B ; show LUCID (Ashcroft–Wadge) combinators are computable maps of type $(L \rightarrow T) \rightarrow (L \rightarrow T)$ / $(L \rightarrow T) \times (L \rightarrow T) \rightarrow (L \rightarrow T)$; conclude LUCID programs define computable maps.

All four claims done. (i) effectively given. $\text{Gamma} := \text{List } \mathbb{N} \oplus (\mathbb{N} \rightarrow \mathbb{N})$ (`star/cons, Gamma_cases`); L 's neighbourhoods are cone sets $\text{nbhd } l := \{z \mid \exists \gamma, z = \text{cons-chain } l \ \gamma\}$ indexed by finite lists (`nbhd_subset_iff` : $\text{nbhd } l \subseteq \text{nbhd } l' \leftrightarrow l' <+ : l$, i.e.

Lean 4 source

```
abbrev L : NeighborhoodSystem Gamma :=
```

3.8 Lecture VIII: Retracts of the Universal Domain

Lecture VIII covers retractions, projections, and the construction of the universal domain U . Scott prints Theorem 8.8 as one unlettered three-sentence theorem; for inventory and Lean-name purposes only, this narrative calls its sentences 8.8(a), 8.8(b), and 8.8(c). Those labels are editorial, not Scott's. The retraction/projection spine (Definitions 8.1/8.3, Proposition 8.2, Example 8.4(a)/(b), Theorem 8.5 in full, **Theorem 8.6 in full — (a)/(b)(i)/(b)(ii)/(c) all Pass**) is formalized below, **Definition 8.7's U itself is now built and verified as a genuine NeighborhoodSystem $\mathbb{Q}$, Pass, and the first sentence of Theorem 8.8 (U 's general/non-effective universality) is now Pass; the second sentence (the effective refinement) is now fully Pass, all internal sub-items (i)–(viii) done; the third sentence is now fully Pass, all 6 internal parts (the diagonal fixed-point predicate `DiagFixed` is r.e. given a computable map — `Theorem88h.lean`; a `qChar`-gated primitive-recursive fold whose output is always `DiagFixed` — `Theorem88i.lean`; the induced enumeration `D_X` covers `fixedNbhd a` exactly — `Theorem88j.lean`; `D_X`'s `interEq/cons` relations are recursively decidable — `Theorem88k.lean`; a primitive-recursive `.inter` for `D_X` with its `inter_spec` — `Theorem88l.lean`; and the final assembly `fixedNbhd_isEffectivelyGiven/theorem_8_8_c` — `Theorem88m.lean`); a few other hard/large items remain deferred.**

Definition 8.1

a *retraction* $a : E \rightarrow E$ with $a \circ a = a$.

`IsRetraction a` := `a.comp a = a`, verbatim. `idMap E` is trivially a retraction (`isRetraction_idMap`, one line from `idMap_comp`).

Lean 4 source

```
def IsRetraction (a : ApproximableMap E E) : Prop
```

Proposition 8.2

$D \triangleleft E$ induces a retraction $a: E \rightarrow E$.

`retractionOfSubsystem h := iøj` for the `Subsystem.inj/Subsystem.proj` pair of Prop 6.12 ($h : D \triangleleft E$); `retractionOfSubsystem_rel : (iøj).rel X Z  $\leftrightarrow$  E.mem X  $\wedge$  E.mem Z  $\wedge$   $\exists Y, D.mem Y \wedge X \subseteq Y \subseteq Z$  unfolds comp_rel/inj_rel/proj_rel. isRetraction_retractionOfSubsystem from jøj=i_D (Prop 6.12)`

Lean 4 source

```
theorem isRetraction_retractionOfSubsystem (h : D ? E) :
```

Definition 8.3

a *projection* (retraction with $a \subseteq I$); a *finitary* retraction (fixed-point set isomorphic to a domain).

`IsProjection a := IsRetraction a  $\wedge$  a  $\leq$  idMap E`; `IsFinitary a :=  $\exists \beta F$ , Nonempty (Fix(a)  $\simeq_o$  F.Element)` (explicit `universe u` to keep β and the ambient α in the same universe, avoiding a metavariable).

Lean 4 source

```
def IsProjection (a : ApproximableMap E E) : Prop
```

Example 8.4(a)

the two-element system $0 = \{\{0\}, \{0, 1\}\}$ arises from a retraction on any non-trivial D : with `check:  $D \rightarrow 0$  ( $X \text{ check } Y \leftrightarrow Y = \{0, 1\} \vee X \neq \Delta$ )`, `fade:  $0 \times D \rightarrow D$  ( $\text{fade}(t, x) = \perp_D$  if  $t = \perp_0$  else  $x$ )`, and any $u \in |D|$ with $u \neq \perp$, `a(x) := fade(check(x), u)` is a retraction whose range is isomorphic to 0 .

Strategy. 0 is a literal `NeighborhoodSystem (Fin 2)` with `mem =  $\{\{0\}, \{0, 1\}\}$` . `check` is a direct `ApproximableMap D 0` from Scott's formula (Prop-valued relation, no `ite`/decidability).

Lean 4 source

```
def 0 : NeighborhoodSystem (Fin 2) where
```

Example 8.4(b)

Scott's remarks after 8.4(a), same section ("EXAMPLES 8.4" is plural — these are two more worked uses of `check/fade`, not a mere aside, hence formalized as part of the same numbered item rather than skipped as a follow-up): (i) `strict:  $(D \rightarrow E) \rightarrow (D \rightarrow E)$` , `strict(f) =  $\lambda x. \text{fade}(\text{check}(x), f(x))$` (`fade:  $0 \times E \rightarrow E$` this time), is a *projection* whose range is exactly the strict functions and is itself a domain; (ii) `smash(x, y) = fade(check(x), fade(check(y),  $\langle x, y \rangle$ ))` is a projection on $D \times E$ with range isomorphic to the smash product $D \otimes E$.

Both combinators reuse `check/fade` **verbatim**, reinstantiated at a second neighbourhood system (they were already generic over the ambient system in `Example84.lean`) — no new relation-level combinator is defined; the whole file is `comp/paired/proj/curry` bookkeeping plus closed-form calculations.

Lean 4 source

```
theorem smashRetraction_eq_retractionOfSubsystem :
```

Theorem 8.5

for $a: E \rightarrow E$, TFAE: (i) a is a finitary projection; (ii) $a(x) = \{Y \in E \mid \exists X \in x, X \subseteq Y \wedge XaX\}$ for all $x \in |E|$.

Both directions proved, assembled as `finitaryProjection_iff_formula`. (ii) $\implies$ (i) (`isFinitaryProjection_of_formula`): `fixedNbhd a :=  $\{X \in E \mid XaX\}$` is a genuine subsystem $\triangleleft E$ for *any* a (`fixedNbhd_subsystem`, needs only `mono/inter_right`); formula (ii)

Lean 4 source

```
theorem finitaryProjection_iff_formula (a : ApproximableMap E E) :
  IsFinitaryProjection a <-> forall (x : E.Element) {Y : Set a}, (a.toElementMap x).mem Y
<->
```

Theorem 8.6(a)

the `sub` combinator on $E \rightarrow E$: `sub f := retractionOfSubsystem (fixedNbhd f)` with $X \text{ sub}(f) Z \leftrightarrow \exists Y \in E, X \subseteq Y \wedge fY \wedge Y \subseteq Z$; **range(sub) = finitary projections on E**.

Scott’s formula *is* Prop 8.2 applied to Thm 8.5’s `fixedNbhd f = {Y ∈ E | Y f Y}` (a genuine subsystem $\triangleleft E$ for *any* f , no hypotheses). Core per-token lemmas: `sub_rel` (unfolding); `sub_le` : `sub f ≤ f` (bare monotonicity: $X \subseteq Y, fY \subseteq Z \implies XfZ$); `fixedNbhd_sub` : `fixedNbhd(sub f) = fixedNbhd f` (witness $Y \subseteq Y' \subseteq Y \implies Y = Y'$); sharper idempotency `sub_sub` : `sub(sub f) = sub f` (equality, not just Scott’s stated $\sqsubseteq$); `sub_mono`.

Lean 4 source

```
def sub (f : ApproximableMap E E) : ApproximableMap E E :=
```

Theorem 8.6(b)(i)

`sub` is itself approximable, and a **projection**, on $(E \rightarrow E)$ — Scott’s remark that “ $f \mapsto \text{sub}(f)$ preserves directed unions of f ’s, thus `sub` is itself approximable”.

`subFilter := toFilter ∘ sub ∘ toApproxMap` transports per-token `sub` along `funSpaceEquiv`; `subApprox` : `ApproximableMap (funSpace E E) (funSpace E E)` built via Exercise 2.13’s `ofContinuous`. Needed new general lemma `continuous_of_monotone_iSupDirected` in `Exercise213.lean` (monotone + directed-sup-preserving $\implies$ topologically continuous), proved from algebraicity (`eq_iSupDirected_principal`, kept local to avoid importing Thm 8.5).

Theorem 8.6(b)(ii)

`sub` is **finitary** on $(E \rightarrow E)$ — `Fix(subApprox)` (the finitary projections on E) is itself isomorphic to a domain.

The originally-flagged “circularity” (writing `subApprox = retractionOfSubsystem h` needs Thm 8.5 applied to `subApprox` itself) turned out to be avoidable: no witness of *that* shape is needed.

Theorem 8.6(c)

if E is effectively given, then `sub` is **computable**.

Mirrors Theorem 7.6’s `fixMap_isComputable` template, but is shorter since `sub`’s formula has a single existential (no iteration/chain). `subApprox := ofContinuous subFilter ...` unfolds via `ofMono/toFilter` to `subApprox.rel F G ↔ (funSpace E E).mem G ∧ sub(toApproxMap ↑F)`

Lean 4 source

```
theorem subApprox_rel_iff {F G : Set (ApproximableMap E E)} (hF : (funSpace E E).mem F) :
```

Definition 8.7

the neighbourhood system U over $[0,1) \subseteq \mathbb{Q}$: non-empty finite unions of rational intervals $[r,s)$ with $0 \leq r < s \leq 1$.

Encoding. A finite union of intervals is coded by $L : \text{List}(\mathbb{Q} \times \mathbb{Q})$ (`presentedIntervals L :=  $\bigcup_{p \in L, \text{Ico } p.1 \text{ } p.2}$`); rather than force the per-pair bounds $0 \leq r < s \leq 1$ into every list operation, $U.\text{mem } X := (\exists L, X = \text{presentedIntervals } L) \wedge X.\text{Nonempty} \wedge X \subseteq \text{Ico } 0 \text{ } 1$ — presentability plus the two set-level facts Scott’s family actually needs. **Closure under $\cap$ is bookkeeping-free**: pairwise-combining two lists’ endpoints via $p.1 \sqcup q.1, p.2 \sqcap q.2$ (`combineIntervals`)

Lean 4 source

```
def U : NeighborhoodSystem ? where
```

Theorem 8.8, first sentence (editorial label 8.8(a))

for every countable neighbourhood system D , $D \trianglelefteq U$ (general/non-effective case: $\exists D' : \text{NeighborhoodSystem } \mathbb{Q}, D \cong^D D' \wedge D' \triangleleft U$).

Scott’s back-and-forth construction, fully assembled. **Atom apparatus** (Theorem88.lean): generic `genAtom` $Z \ M \ \delta \ n/\text{atomD}$, a totalized `splitChoice` packaging `exists_split` (built from Definition 8.7’s `U_no_minimal`, no interval-difference-closure lemma needed — the three cases are handled entirely by $\emptyset$, B itself, or `U_no_minimal`’s output), the recursive `atomU`, and the combined invariant `atomU_invariant` (emptiness-match/`U.mem-or- $\emptyset$` /pairwise disjointness).

Theorem 8.8(b)

if D is effectively given, the projection pair witnessing $D \trianglelefteq U$ (Theorem 8.8(a)) can be taken computable.

the effective refinement of 8.8(a)’s construction, broken into an 8-part plan for tractability, tracked as sub-rows **8.8(b)(i)–8.8(b)(viii)**. ******(i)–(vi)

Theorem 8.8(b)(i)

Part 1 of 8 — choice-free Gödel numbering of $\mathbb{Z}/\mathbb{Q}$ plus primitive-recursive comparison arithmetic, for use throughout the rest of the effective refinement.

a choice-free Gödel numbering of $\mathbb{Z}$ (difference-pairs `encodeInt/decodeInt`, exact round trip for *every* z , no canonicity side-condition) and $\mathbb{Q}$ (‘`encodeRat` $q := \text{pair } (\text{encodeInt } q.\text{num})$ ’)

Theorem 8.8(b)(ii)

Part 2 of 8 — `List` ($\mathbb{Q} \times \mathbb{Q}$)-code encoding of presented interval-unions, code-level `combineIntervals`/difference and subset/equality/nonemptiness decidability at the code level.

generic `Nat.Primrec` list-code combinators `crossCombine/flatMapCode` (new, in `RecursiveCross.lean`); `List` ($\mathbb{Q} \times \mathbb{Q}$) encoding (`encodeQPairList/decodeQPairList`, exact round trip, via `Recursive.lean`’s `encodeList`); code-level `combineCode` realizing `combineIntervals` (`presentedIntervals_decodeQPairList_combineCode`); code-level interval *difference* `diffCode` (built from the unconditional identity `Ico_diff_Ico : Ico a b \ Ico c d = Ico a (b  $\sqcap$  c)  $\cup$  Ico (a  $\sqcup$  d) ...`).

Theorem 8.8(b)(iii)

Part 3 of 8 — assemble a genuine `ComputablePresentation` of U (`U.IsEffectivelyGiven`).

List-level canonicalization `canonList` (clip every pair into $[0,1)$ via `qpClip`, filter degenerate pairs, fall back to `U.master`’s literal $[(0,1)]$ if empty) is *always* a valid U -presentation (`U_mem_presentedIntervals_canonList`)

Theorem 8.8(b)(iv)

Part 4 of 8 — an explicit deterministic `splitU` replacing Definition 8.7’s `U_no_minimal` existential midpoint-split.

Since Part 3 already showed every pair in `canonCode` n ’s decoded list is non-degenerate ($p.1 < p.2$), `splitU` needs no existential search: it deterministically takes the list’s *first* pair (`firstElemCode`)

Theorem 8.8(b)(v)

Part 5 of 8 — for an arbitrary `ComputablePresentation` of D , deciding whether a finite Boolean atom-constraint on D ’s neighbourhoods is satisfiable (Theorem 8.8(a)’s ($\diamond$) trick, made decidable).

For an arbitrary `ComputablePresentation` P of D , reindexes Theorem 8.8(a)’s ($\diamond$) D -atoms over $\mathbb{N}$ via `idxSet` (Theorem88a.lean): `IPos` $P \ pos := \{m \mid \forall i \in pos, P.X \ m \subseteq P.X \ i\}$ ($= \text{idxSet } i_1 \cap \dots$ via `IPos_cons`), `DAtom` $P \ pos \ neg := \text{IPos } P \ pos \cap \{m \mid \forall j \in neg, P.X \ m \not\subseteq P.X \ j\}$. The positive meet is computed by a fold (`meetStep/meetFold`)

Theorem 8.8(b)(vi)

Part 6 of 8 — the recursive `Y_n-chain` (Theorem 8.8(a)’s `Yidx`) as an effective witness/verifier construction, combining Part 4’s `splitU` with Part 5’s `DAtom_recDecidable`.

Sub-steps 6a–6c: **6a** — generalize Theorem88.lean over an abstract split. Introduced `SplitSpec` `split : Prop` (exactly `exists_split`’s conclusion, packaged as a `Prop` about a *total* function `split : Set  $\alpha \rightarrow$  Set  $\mathbb{Q} \rightarrow$  Set  $\alpha \rightarrow$  Set  $\mathbb{Q} \times$  Set  $\mathbb{Q}$`); `splitChoice_isSplitSpec`

recovers the classical instantiation. Every downstream definition/theorem from `atomU` through `Yseq_nonempty_of_mem` (`atomU`, `Yseq`, `atomU_invariant`, all `transfer_*/Yseq_*` lemmas)

Theorem 8.8(b)(vii)

Part 7 of 8 — the projection pair $i, j : \text{ApproximableMap } D \ U$ (Theorem 8.8(a)’s `domainIso`-derived pair) satisfy `IsComputableMap`.

Correction of an earlier plan: (vi)’s `DprimeUPresentation` alone does *not* unblock this part — it only shows D ’s own index relations decidable, never which U -code $\text{Yidx } e \ n$ (a `Classical.choice`-picked value, via `splitChoice`) actually sits at, so the needed cross-relation $\text{Yidx } e \ n \subseteq UX \ m$ cannot be shown r.e.

Theorem 8.8(b)(vii)(1)

The `atomUCode` invariant, mirroring `Theorem88.lean`’s `atomU_invariant` but proved fresh at the code level — **restated** (see Proof Notes for why) as: (validity) $U.\text{mem } (UX \ (\text{atomUCode } P \ n \ k))$ unconditionally; (disjointness) if $\text{deltaOf } k / \text{deltaOf } k'$ disagree at some $j < n$ and **both bit-sources are still D -side non-empty at depth n** ($\text{atomUEmpty } P \ n \ k = 0$, $\text{atomUEmpty } P \ n \ k' = 0$), then $UX \ (\text{atomUCode } P \ n \ k) \cap UX \ (\text{atomUCode } P \ n \ k') = \emptyset$.

***The originally-planned “(a) match” clause ($D\text{Atom} = \emptyset \leftrightarrow UX \ (\text{atomUCode } \dots)$*

Theorem 8.8(b)(vii)(2)

`YseqCode` — Scott’s Y_n , coded: a `Nat.Primrec` union, over the 2^n bit-sources $k < 2^n$ with bit n forced to 1 (i.e. $k + 2^n$ for $k < 2^n$), of `atomUCode` $P \ (n+1) \ (k+2^n)$, satisfying $UX \ (\text{YseqCode } P \ n) = \text{Yseq-analogue}$ (the `Set`-level closed form Scott needs).

Bit arithmetic first: `deltaOf_eq_testBit` identifies `deltaOf` with `Nat.testBit` outright, so `deltaOf_add_two_pow_of_lt/deltaOf_two_pow_add_self/deltaOf_mod_two_pow_of_lt` (how `deltaOf` reacts to $+2^n/\%2^n$) are direct transcriptions of `Nat.testBit_two_pow_add_gt/_eq/Nat.testBit_mod_two_pow`, no bespoke induction.

Theorem 8.8(b)(vii)(3)

Assemble a fresh subsystem D'' via $n \mapsto UX \ (\text{YseqCode } P \ n)$ (or reuse `Theorem88a.lean`’s `DprimeU/domainIso` shape with `Yseq` replaced by this `YseqCode` closed form); prove $D \cong^D D''$ and $D'' \triangleleft U$; build a `ComputablePresentation` D'' with master index 0 and $X \ n := UX \ (\text{YseqCode } P \ n)$ (genuinely code-driven, unlike `Yidx`).

New file (imports `Theorem88d.lean + Theorem88c.lean`), reindexing everything from `atomUCode/YseqCode` down to $Yc \ P \ n := UX \ (\text{YseqCode } P \ n)$. **Bridging step:** `hcoreIdxYc` (via a fresh `encodeBits` : $(\mathbb{N} \rightarrow \text{Bool}) \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ realizing any finite δ -prefix as some `deltaOf` k , plus `genAtom_Yc_empty_iff` from (vii)(2)’s closed form)

Theorem 8.8(b)(vii)(4)

`IsComputableMap` for D'' ’s `Subsystem.inj/Subsystem.proj` against U ’s presentation — i.e. the actual headline claim of Theorem 8.8(b)(vii).

New file (imports `Theorem88e.lean + Definition72.lean + Proposition612.lean`). Turned out not to need a bespoke `subsetUChar`-style decider at all: `Subsystem.inj_rel/Subsystem.proj_rel` unfold i/j ’s relations to a `mem`-clause on each side *plus* a raw subset test, and every `mem`-clause is automatically true once both sides are read off their own presentations ($\langle n, \text{rfl} \rangle$ for D'' via `DprimeUCodePresentation`, $U.\text{mem}_UX$ for U), so both relations collapse to a **single already-generic fact:** `ComputablePresentation.incl_computable` (Definition 7.1, proved once for *any* presentation)

Theorem 8.8(b)(viii)

Part 8 of 8 — final assembly `theorem_8_8_b` (the computable-projection-pair analogue of `theorem_8_8_a`) plus documentation update.

Assembles Parts 7(3)/7(4) into the final statement, mirroring `theorem_8_8_a`’s shape but with D' additionally presented and the witnessing projection pair additionally `IsComputableMap`: ‘`theorem_8_8_b` { D } ($P : \text{ComputablePresentation } D$)

Theorem 8.8(c)

converse correspondence — a computable, finitary projection a of U yields an effectively given domain $(\{Y \in U \mid YaY\} \triangleleft U$ is effectively given).

the naive reading of `arxiv.md`'s old one-line proof note (" a 's graph r.e. + U -equality decidable $\implies \{Y \in U \mid YaY\}$ r.e.

Theorem 8.8(c)(i)

Part 1 of 6 — the diagonal fixed-point predicate `DiagFixed P a n := a.rel Xn Xn` (i.e. `(fixedNbhd a).mem Xn`, given X_n is always a U -neighbourhood) is recursively enumerable, given a is a computable map.

stated generally over any $V : \text{NeighborhoodSystem } \alpha$ and $P : \text{ComputablePresentation } V$ (not just U ; Theorem 8.8(c) itself will specialize $V := U, P := U\text{ComputablePresentation}$).

Theorem 8.8(c)(ii)

Part 2 of 6 — a `qChar`-gated, `Nat.Primrec` list-fold (`myStep/myFoldCode`) whose accumulator is *always* a raw U -index satisfying `DiagFixed`, for any input list-code.

stated generally over any V/P (as with 8.8(c)(i)). Each list entry e codes a pair $\langle i, n \rangle = \text{Nat.pair } i \ n$: a candidate raw index n plus a *claimed* `qChar`-witness i .

Theorem 8.8(c)(iii)

Part 3 of 6 — the induced enumeration `D_X qChar cons c := P.X (myFoldCode qChar cons c)` is always a `fixedNbhd a`-neighbourhood (`mem_X`) and hits every one of them (`surj`).

`D_X P qChar cons c := P.X (myFoldCode P qChar cons c)`. `D_X_mem` is Part 2's invariant `diagFixed_myFoldCode` repackaged via `diagFixed_iff_fixedNbhd_mem` — one line.

Theorem 8.8(c)(iv)

Part 4 of 6 — `D_X`'s `interEq/cons` relations are recursively decidable.

`D_X_interEq_computable` is free — `D_X c1 ∩ D_X c2 = D_X c3` unfolds *literally* to $X_{n_1} \cap X_{n_2} = X_{n_3}$ for $n_i := \text{myFoldCode } qChar \text{ cons } c_i$, so `RecDecidable3` follows by composing `P.interEq_computable` with the primitive-recursive triple-`myFoldCode` `reindex (primrec_myFoldCode, Part 2)` — no `a/DiagFixed` apparatus needed.

Theorem 8.8(c)(v)

Part 5 of 6 — a primitive-recursive `.inter` field for `D_X` and its `inter_spec`.

`D_inter c1 c2 := appendListCode c1 c2` (`Recursive.lean`'s Exercise 7.22 combinator, reused outright): correct because `myFoldCode` is a *left* fold from `P.masterIdx`, so `List.foldl_append` decomposes folding the concatenated list as folding `c2`'s list *starting from* `myFoldCode c1` instead of from `P.masterIdx` (`D_X_inter_eq`). The one genuine lemma, `myFoldl_inter_of_le`: refolding a list l from a start $n \subseteq r$ (in $P.X$)

Theorem 8.8(c)(vi)

Part 6 of 6 — final assembly: package Parts 1–5 into a genuine `ComputablePresentation (fixedNbhd a)` and state `theorem_8_8_c`.

`fixedNbhd_isEffectivelyGiven {a} (hcomp : IsComputableMap P P a) : (fixedNbhd a).IsEffectivelyGiven` obtains concrete witnesses for the two abstract functions Parts 2–5 were stated over — `qChar` from Part 1's `diagFixed_exists_qChar hcomp`, and `cons` from `P.cons_computable` itself (unfolded/reindexed at `Nat.pair n m` via `unpair_pair_fst/unpair_pair_snd` into the exact `hcons` shape every part expects)

Definition 8.9

fixed computable projection pairs $i_+, j_+, i_-, j_-, i_-, j_-$ for U , and combinators $a+b, a \times b, a \rightarrow b$.

The gap: `theorem_8_8_b` (`Theorem88g.lean`) only gives a computable pair $D' \rightleftharpoons U$ for an *isomorphic copy* D' , leaving $D \cong^D D'$ a bare `Nonempty`; Definition 8.9 needs a direct pair $D \rightleftharpoons U$ for $D := U+U/U \times U/U \rightarrow U$ itself.

Lean 4 source

```
theorem theorem_8_8_b_strong :
  exists (i : ApproximableMap D U) (j : ApproximableMap U D),
```

Proposition 8.10(a)

if $a, b: \mathcal{U} \rightarrow \mathcal{U}$ are projections, then so are $a+b$, $a \times b$, $a \rightarrow b$.

Four generic element-level lemmas open the file: `toElementMap_of_comp_eq_idMap` ($j \circ i = I \Rightarrow D \Rightarrow j(i(v)) = v$), `toElementMap_le_of_comp_le_idMap` ($i \circ j \leq I \Rightarrow i(j(x)) \leq x$), `toElementMap_le_self_of_le_idMap` ($a \leq I \Rightarrow a(x) \leq x$), `toElementMap_idem_of_isRetraction` ($a \circ a = a \Rightarrow a(a(x)) = a(x)$). $\times$: `prodComb a b = iTimes (prodMap a b) o jTimes` *literally* (Exercise 3.19's product-functor combinator, `rfl`); both closure facts reduce to `pair_le_pair_iff`/`toElementMap_prodMap`/`toElementMap_mono` plus the generic lemmas.

Lean 4 source

```
theorem toElementMap_of_comp_eq_idMap {i : ApproximableMap D E} {j : ApproximableMap E D}
```

Proposition 8.10(b)

if a, b are finitary, then so are $a+b$, $a \times b$, $a \rightarrow b$, with $D_{\{a+b\}} \cong D_a + D_b$, $D_{\{a \times b\}} \cong D_a \times D_b$, $D_{\{a \rightarrow b\}} \cong (D_a \rightarrow D_b)$.

Setup: for a finitary projection a , Theorem 8.6's `sub_eq_self_of_isFinitaryProjection` gives $a = i_a \circ j_a$ for the *concrete* subsystem pair $i_a, j_a : D_a \rightleftarrows \mathcal{U}$ ($D_a := \text{fixedNbhd } a$, Theorem 8.5), replacing the abstract `IsFinitary` witness with something to compute against. **Generic tool** `elementIsoOfProjectionPair` (Proposition 8.2's `elementIso`, generalized off the literal $D \leq E$ requirement to *any* approximable pair $i: D \rightarrow E, j: E \rightarrow D$ with $j \circ i = I_D$, no shared token type needed, and $g := i \circ j$): gives $D.\text{Element} \simeq \text{Fix}(g)$ directly — verbatim `Subsystem.elementIso`'s proof.

Lean 4 source

```
def elementIsoOfProjectionPair (i : ApproximableMap D E) (j : ApproximableMap E D)
  (hji : j.comp i = idMap D) {g : ApproximableMap E E} (hg : g = i.comp j) :
  D.Element ?o {y : E.Element // g.toElementMap y = y} where
```

Exercise 8.11

$R = \{[0, r) \mid r \in \mathbb{Q}, 0 < r \leq 1\}$; $[0, r)$ a $[0, s)$ iff $r < s$ or $r = s = 1$ is approximable, a is a projection whose fixed-point set $\leftrightarrow [0, 1] \subseteq \mathbb{R}$ (Dedekind cuts), and a is NOT finitary.

$R := \text{ofNestedOrDisjoint}$ -style chain system $\{[0, r) \mid 0 < r \leq 1\}$ over $\mathbb{Q}$ (nested by r , so no disjoint case ever fires).

Lean 4 source

```
def a : ApproximableMap R R where
```

Exercise 8.12

Generalize $2X+1$ to $2^k X + \ell$ ($\ell < 2^k$); $V =$ non-empty finite unions of $2^k \mathbb{N} + \ell$; show $U \cong V$ effectively, another presentation of U .

Broken into a **7-part plan** (mirroring how Theorem 8.8 was split into 15 files/~4,470 lines across (a)/(b)/(c)), tracked as sub-rows **8.12(a)–8.12(g)**, **all now Pass**: V 's construction/-closure/faithfulness (a), its `ComputablePresentation` (b), the general two-sided back-and-forth lemma (c), its effective refinement (d), U/V each satisfying the extension property relative to the other (e)/(f), and the final assembly (g) — zero sorry project-wide.

Lean 4 source

```
def V : NeighborhoodSystem Nat where
```

Exercise 8.12(a)

Part 1 of 7 — define V : the neighbourhood system over $\mathbb{N}$ of non-empty finite unions of $2^k \mathbb{N} + \ell$ ($\ell < 2^k$), with its basic structural properties.

$\text{affine } k \ell := \{n \mid n \% 2^k = \ell\}$ (Scott’s $2^k \mathbb{N} + \ell$, auto- $\emptyset$ for $\ell \geq 2^k$). Bookkeeping-free encoding $\text{levelSet } k \ m := \{n \mid m.\text{testBit}(n \% 2^k)\}$ (a single bitmask replaces `Definition87.lean`’s `List`, since residue classes at a fixed level partition $\mathbb{N}$, unlike arbitrary rational intervals).

Lean 4 source

```
def V : NeighborhoodSystem Nat where
```

Exercise 8.12(b)

Part 2 of 7 — V ’s effective presentation (`ComputablePresentation V`), mirroring `UComputablePresentation` built the missing `Nat.Primrec` bit-manipulation infrastructure from scratch (choice-free base combinators only, no `Mathlib Primrec/Nat.testBit/Nat.land`): `myDivPow2/myTestBit` (iterate “halve” via `Nat.Primrec.prec`, matching `Nat.testBit_eq_decide_div_mod_eq`), `myLand` (bitwise AND, a bit-for-bit mirror of `Recursive.lean`’s `myLor`), and `myUpsample` — the key missing piece — realized *arithmetically*, not bit-by-bit: one level-step duplicates a mask’s low 2^k bits into a second copy shifted up by 2^k positions (`myUpsampleStep k m := 2^(2^k) * m' + m'` with `m' := myModPow2 m (2^k)`; truncating first, via `Nat.testBit_two_pow_mul_add`, is what makes this correct regardless of the input’s “junk” high bits), iterated $k'-k$ times via `Nat.Primrec.prec` jointly tracking `(level, mask)` (mirrors `myLor`’s joint-state iteration), with a final `myModPow2` re-truncation so the output is *always* bounded (`myUpsample_lt`)

Lean 4 source

```
def myDivPow2 (m ? : Nat) : Nat
```

Exercise 8.12(c)

Part 3 of 7 — new general (non-effective) lemma: two countable, atomless neighbourhood systems D_0, D_1 , each satisfying a mutual one-step “extension”/splitting property relative to the other, are order-isomorphic ($D_0.\text{Element} \simeq_o D_1.\text{Element}$).

the genuinely new piece of abstract theory this exercise needs, broken into a 7-part plan tracked as sub-rows **8.12(c)(i)–8.12(c)(vii)**, **all now Pass**: Boolean-closure prerequisites (i), generic hypotheses + free Boolean-atom infrastructure (ii), `Theorem88.lean`’s splitting lemma generalized to an abstract atomless target (iii), the interleaved two-sided atom construction + its core invariant (iv), pairwise disjointness of that construction across sign sequences (v, itself split into 5 sub-sub-parts — see 8.12(c)(v)’s own sub-rows), bidirectional transfer lemmas + the genuine-neighbourhood glue (vi, 7 sub-parts), and the final `DomainIso D_0 D_1` assembly (vii, 6 sub-parts — `domainIso812c/isomorphic_812c`).

Lean 4 source

```
theorem isomorphic_812c : D_0 ~=? D_1 :=
```

Exercise 8.12(c)(i)

Part 1 of 7 — U and V are Boolean-closed (closed under set-*difference*, not just consistent intersection) and Positive (Exercise 1.19).

`U_diff_mem` is a two-line corollary of the *already-existing* `IntervalPrimrec.lean`’s `diffLists/presentedIn diffLists` (built long ago for the computable-presentation work, never previously used for this purpose): only non-emptiness of the raw difference can obstruct `U.mem`, since presentability and $\subseteq [0, 1)$ transfer automatically.

Exercise 8.12(c)(ii)

Part 2 of 7 — generic `NoMinimal/DiffClosed` hypotheses, and the payoff: finite Boolean combinations (`Theorem88.lean`’s `genAtom`) are automatically `mem-or- $\emptyset$` , no choice needed.

`NeighborhoodSystem.NoMinimal/.DiffClosed` generalize `U_no_minimal/V_no_minimal/(i)`’s difference-closure as abstract hypotheses — `NoMinimal` phrased with explicit `Y.Nonempty/Z.Nonempty` clauses rather than `U_no_minimal`’s $Y \neq X/Z \neq X$, since an abstract `D.mem` need not itself carry

nonemptiness the way `U.mem/V.mem` do (logically equivalent given $Y \cap Z = \emptyset / Y \cup Z = X$, but directly what `exists_split'` needs).

Exercise 8.12(c)(iii)

Part 3 of 7 — generalize `Theorem88.lean`’s `exists_split/SplitSpec/splitChoice` from the hardcoded target `U` to an abstract atomless system `E`.

`exists_split'/SplitSpec'/splitChoice'` are verbatim proof transcriptions of `Theorem88.lean`’s originals with `U/U.mem/U_no_minimal` replaced by an abstract `E/E.mem/hEnomin : E.NoMinimal`, so the same lemma can later be instantiated with `E:=D1` (splitting `D1`’s side against a `D0`-enumerated target) or `E:=D0` (symmetrically)

Exercise 8.12(c)(iv)

Part 4 of 7 — the interleaved two-sided atom construction `atomPair` and its core invariant (matched emptiness, `mem-or- $\emptyset$` on both sides at every depth).

Fix `D0,D1` (Positive, difference-closed, atomless) with enumerations `X,Y` covering their neighbourhoods.

Exercise 8.12(c)(v)

Part 5 of 7 — pairwise disjointness of `atomPair` across sign sequences disagreeing before depth `n`, on *both* sides simultaneously (the two-sided analogue of `Theorem88.lean`’s `atomU_invariant`’s third clause).

needed so that a `Yseq`-style union over depth-`n` atoms recovers a well-defined, non-overlapping decomposition on *both* the `D0`- and `D1`-sides at once (unlike Theorem 8.8(a), where only the `U`-side ever needed this). Broken into 5 sub-sub-parts, mirroring how Theorem 8.8(b)(vii)

Exercise 8.12(c)(v)(1)

the generic single-sub-step abstraction `xyStep split A B Xn b` — “intersect/subtract `A` by `Xn` directly per sign `b`, and correspondingly split `B` via an abstract `split` function” as one ordinary (non-recursive) function — plus its local pairwise-disjointness fact `xyStep_disjoint_of_ne` (two applications at *different* sign bits land in disjoint sets, on *both* output components).

two elementary, fully generic set-theory facts drive it: `if_swap_disjoint` (if $P \cap Q = \emptyset$ then if `b` then `P` else `Q` and if `b'` then `P` else `Q` are disjoint whenever $b \neq b'$, by case-splitting both booleans) and `inter_diff_self_eq_empty` ($(P \cap Q)$

Exercise 8.12(c)(v)(2)

the two named sub-steps of `atomPair` as instances of (v)(1)’s `xyStep`: `xStep` (split `D1`’s side via `splitChoice'`, directly refine `D0`’s side) and `yStep` (symmetric, with a `.swap` to restore (α, β) -side ordering) — together with their subset lemmas (`xStepfst_subset/xStepsnd_subset/yStepfst_subset/yStepynd_subset`) and disjointness corollaries (`xStep_disjoint_of_ne/yStep_disjoint_of_ne`, direct instances of (v)(1)’s `xyStep_disjoint_of_ne`).

stated and proved fully generically — `xStep` only ever needs `D1/hD1nomin`, `yStep` only `D0/hD0nomin`, never the full two-sided `AtomPair` context — and placed *before* section `AtomPair` for exactly that reason: an earlier draft defined them *inside* the section and hit an `Application type mismatch` (Lean’s `include` directive silently prepended every section variable, e.g. `hD0pos`, onto their signatures even though unused, so a call like `xStep_disjoint_of_ne hD1nomin bound hD1nomin` to the wrong parameter slot).

Exercise 8.12(c)(v)(3)

the glue connecting `atomPair`’s own recursive `let-chain` to the `xStep/yStep` decomposition: `atomPair_succ_eq (atomPair  $\delta$  (n+1) = yStep D0 hD0nomin (xStep D1 hD1nomin A B (X n) ( $\delta$  n).1).1 (xStep ...).2 (Y n) ( $\delta$  n).2), xStep_spec` (the `xStep` output satisfies exactly the `SplitSpec'` preconditions `yStep` needs to fire its own lemmas), and `atomPair_congr` (sign sequences agreeing below `n` give an identical depth-`n` pair).

`atomPair_succ_eq` is proved by `rfl` outright — both sides unfold to the identical $(A2, B2)$ pair, since `xStep/yStep` are themselves just `xyStep` unfolded to the same `if-then-else` shape `atomPair`’s own definition already uses; this is what lets later lemmas manipulate the sub-steps algebraically instead of re-deriving the unfolding by hand each time. `xStep_spec` mirrors the

corresponding step inside (iv)’s `atomPair_invariant` induction (Boolean-closure for the direct side via `inter_mem_or_empty/diff_mem_or_empty`, `splitChoice'_isSplitSpec` for the split side).

Exercise 8.12(c)(v)(4)

monotonicity of `atomPair` on both sides: `atomPair_fst_subset/atomPair_snd_subset`, i.e. $\text{atomPair } \delta \ (n+1) \subseteq \text{atomPair } \delta \ n$ componentwise.

direct consequence of (v)(2)/(v)(3)’s lemmas chained through `atomPair_succ_eq`: the α -side goes through `yStep_fst_subset` (needs (v)(3)’s `xStep_spec` for its `SplitSpec'` preconditions) and then `xStep_fst_subset` (unconditional); the β -side through `yStep_snd_subset` (unconditional)

Exercise 8.12(c)(v)(5)

the headline theorem `atomPair_disjoint`: for sign sequences δ, δ' disagreeing somewhere below depth n , `atomPair` $\delta \ n$ and `atomPair` $\delta' \ n$ are disjoint on *both* sides at once.

induction on n , mirroring `Theorem88.lean`’s `atomU_invariant` disjointness clause but two-sided throughout. “Disagree below n ”: recurses via `ih` and shrinks via (v)(4)’s `atomPair_fst_subset/atomPair_snd_subset`.

Exercise 8.12(c)(vi)

Part 6 of 7 — bidirectional `Yseq`-analogue closed forms and transfer lemmas: $X \ n / Y \ n$ are each recoverable as a finite union of matched `atomPair` atoms, with `subset/inter-empty/inter_eq` transfer in both directions (mirroring `Theorem88.lean`’s `transfer_subset_iff/transfer_inter_eq_iff`, but two-way).

the bulk of the remaining size estimate — comparable to the rest of `Theorem88.lean` (`Yseq` onward, ~350 lines: `extendTrue/restrictFin` bookkeeping, `Yseq/split_fst_eq_inter_Yseq/atomU_succ_eq/atomU_eq_genAtom` closed form, `transfer_dir/transfer_empty_iff/transfer_subset_iff/transfer_inter_empty_iff/transfer_double_subset_iff/transfer_inter_eq_iff`, `Yseq_empty_or_mem/Yseq_nonempty_of_mem`), done twice (once per direction) plus inter-leaving glue.

Exercise 8.12(c)(vi)(1)

Generalize `Theorem88.lean`’s core `Yseq` closed-form apparatus (`Yseq`, `subset_Yseq`, `split_fst_eq_inter_Yseq`, `atomU_subset_master`, `atomU_succ_eq`, `atomU_eq_genAtom`) from the hard-coded target U to an abstract atomless E (`SplitSpec' E split`, `E.master`, `E.mem`) — the `Yseq`-analogue of (iii)’s `exists_split → exists_split'` generalization, but for the closed-form layer built on top.

as anticipated, `extendTrue/restrictFin` and the generic `genAtom` lemmas (`genAtom_subset/genAtom_congr/genAtom_forward/genAtom_self`) were reused **verbatim** from `Theorem88.lean`, no re-proving needed.

Exercise 8.12(c)(vi)(2)

Generalize the finite-constraint transfer lemma and its corollaries (`transfer_dir`, `transfer_empty_iff`, `transfer_subset_iff`, `transfer_inter_empty_iff`, `transfer_double_subset_iff`, `transfer_inter_eq_iff`) to the same abstract atomless E from (vi)(1).

confirmed as anticipated — `transfer_dir` needed **zero** re-proof, only its `private` modifier removed (it was already stated fully generically over two independent carrier types $\beta_1 \beta_2$, connected only by a `genAtom`-emptiness correspondence `hcore`; `private` blocks cross-file reuse regardless of genericity). The five corollaries (`transfer_empty_iffE/transfer_subset_iffE/transfer_inter_empty_iffE/transfer_double_subset_iffE/transfer_inter_eq_iffE`)

Exercise 8.12(c)(vi)(3)

Generalize the nonemptiness/membership facts (`Yseq_subset_master`, `Yseq_zero_eq_master`, `Yseq_empty_or_mem`, `Yseq_nonempty_of_mem`) to the abstract atomless E from (vi)(1).

completes “(vi)(1)–(vi)(3): the abstract, single-family, one-sided `Yseq`-apparatus over any atomless E ” — the piece that, once done, can be instantiated twice (§(vi)(5)/(vi)(6)) without re-deriving any of this general theory a second time.

Exercise 8.12(c)(vi)(4)

Identify `atomPair`'s per-side trajectory with an instance of (vi)(1)–(vi)(3)'s single-family `atomE` (**corrected**): recover $X\ n/Y\ n$ as unions of `atomPair`-derived “half-step” pieces on the *other* side, built directly against `atomPair`'s own already-proven invariants — umbrella, see sub-items (vi)(4)(a)–(vi)(4)(d) below.

the original pre-plan's literal “bridge to `atomE`” is false, not just difficult — discovered by direct calculation, not just difficulty in formalizing. `atomE`'s testing family `genAtom` $X\ \Delta\ \delta\ n$ is a *free* Boolean combination, only ever intersected/subtracted directly, never itself split; but `atomPair`'s A-component *is* itself choice-split at every Y-sub-step (via `D0.NoMinimal`, `exists_split`'s genuine-split case), so whenever that case fires (generically), the real `An` is a **proper subset** of `genAtom` $X\ D_0.master\ \delta_1\ n\ (\delta_1\ k := (\delta\ k).1)$

Exercise 8.12(c)(vi)(4)(a)

The $X\ n$ -side half-step closed form: recover $X\ n$ as a union of `atomPair`'s D_1 -side half-step pieces (the X -sub-step's “+” branch, *before* the following Y-sub-step refines it further).

new generic two-sided sign-sequence padding `extendTruePair/restrictFinPair` (componentwise `extendTrue/restrictFin`, `Theorem88.lean`) with `extendTruePair_restrictFinPair_agree`, plus `XPseq n : Set  $\beta$` (union over depth- n histories of the D_1 -piece chosen by the X -sub-step's “+” branch, via `xStep` directly with argument `true`)

Exercise 8.12(c)(vi)(4)(b)

Define the $Y\ n$ -side half-step closed form `YPseq n : Set  $\alpha$` , together with its trivial monotonicity fact.

unlike $X\ n$'s side, `yStep`'s own inputs (`A1`, `B1`, the *post-X*-sub-step values) already depend on $(\delta\ n).1$ itself, not just history strictly below n — so `YPseq`'s union needs an *extra* free `Bool` parameter `bx` (for position n 's X -sub-step bit)

Exercise 8.12(c)(vi)(4)(c)

A reusable bridging lemma (`xStep_spec_bit`): the `SplitSpec`' preconditions `yStep` needs, transported across the X -sub-step, at an *arbitrary* bit `bx` (not just $\delta\ n$'s own first component) — connecting an arbitrary (δ', bx) witness of `YPseq n` back to a concrete history.

mirrors `Theorem88.lean`'s `Function.update`-based δ_2/δ_3 device inside `split_fst_eq_inter_Yseq` (needed there because `atomU`'s recursion threads $\delta\ n$'s own value into its depth- $n+1$ step): set `$\delta'' := Function.update (extendTruePair (restrictFinPair  $\delta\ n))\ n\ (bx, true)$$` (second component of the update irrelevant/arbitrary, unused elsewhere)

Exercise 8.12(c)(vi)(4)(d)

The $Y\ n$ -side “I-formula” `yStep_fst_eq_inter_YPseq`: for any history δ and depth n , the Y -sub-step's “+” branch (applied to the post- X -sub-step values at δ 's own bit $(\delta\ n).1$) equals $(xStep_fst_output_at\ (\delta\ n).1) \cap YPseq\ n$.

assembles (vi)(4)(b)+(vi)(4)(c) with `xStep_spec_bit/yStep_fst_subset/xStep_disjoint_of_ne/xStep_fst_subset/atomPair_disjoint/atomPair_invariant` (all already `Pass`), by the same two-sided antisymmetry argument as `xStep_snd_eq_inter_XPseq`, but the $\supseteq$ direction now needs a genuine 3-way case split on the union witness (δ', bx) (`XPseq`'s proof only needed 2): (i) history agrees below n and $bx = (\delta\ n).1$ — literally the same J after two rewrites; (ii) history agrees below n but $bx \neq (\delta\ n).1$ — `xStep_disjoint_of_ne` (at δ 's own A, B)

Exercise 8.12(c)(vi)(5)

Scoping pass (2026-07-04), before any further code: re-examined `atomPair`'s own recursive step (`atomPair_succ_eq = yStep  $\circ$  xStep`) at the level of individual *half*-steps rather than full steps. Key observation: the X -sub-step's direct α -output ($A \cap X_n / A \setminus X_n$) is *already* a literal `genAtom`-style formula against the family X , and (by (vi)(4)(d)'s `yStep_fst_eq_inter_YPseq`) the Y -sub-step's choice-driven α -output is, on its “+” branch, *also* a literal `genAtom`-style formula against the family `YPseq` — so $(atomPair\ \delta\ n).1$, unrolled one half-step at a time, is *exactly* `genAtom` over the **interleaved** family `combinedX :  $\mathbb{N} \rightarrow Set\ \alpha$` with `combinedX (2k) := X k`, `combinedX (2k+1) := YPseq k`, tested against the interleaved sign sequence `combined $\delta\ \delta$`

: $\mathbb{N} \rightarrow \text{Bool}$ ($\text{combined}\delta \delta (2k) := (\delta k).1$, $\text{combined}\delta \delta (2k+1) := (\delta k).2$). Symmetrically, $(\text{atomPair } \delta n).2$ is genAtom over $\text{combinedY } (2k) := \text{XPseq } k$, $\text{combinedY } (2k+1) := Y k$, against the *same* $\text{combined}\delta \delta$. Once both sides are literal genAtoms over a shared sign sequence, `Theorem88.lean`’s fully generic `transfer_dir/transfer_empty_iff/transfer_subset_iff/transfer_inter_empty_iff/transfer_double_subset_iff/transfer_inter_eq_iff` become directly reusable (none are `private`, confirmed) exactly as (vi)(2) reused `transfer_dir` — specializing the reused lemmas to even/even or odd/odd index pairs $(2i, 2j$ or $2i+1, 2j+1)$ unwinds `combinedX/combinedY` back to literal $X i \subseteq X j / \text{XPseq } i \subseteq \text{XPseq } j$ -style statements. This is a materially smaller plan than the initial “substantial, unscoped new work” estimate, because it needs **no** new disjointness or invariant proof: the two ingredients `transfer_dir` needs (matching-emptiness `hcore`, and — internally, already consumed by (vi)(4) — pairwise disjointness) are *already available* half-step-by-half-step as `atomPair_invariant` (full-step/even case) and `xStep_spec_bit` (half-step/odd case, already proved in (vi)(4)(c)). The only genuinely new content is (a) below (completing (vi)(4)’s one-branch I-formulas into full closed forms) and the bookkeeping of (b)–(d). Broken into 4 sub-sub-sub-parts, mirroring (vi)(4)’s own post-hoc (a)–(d) split: * **(a)** the “else-branch” closed-form algebra completing (vi)(4)(a)/(d)’s one-branch (`true-only`) I-formulas into full two-branch closed forms (mirroring `Theorem88.lean`’s `atomU_succ_eq`, which derives its `false` branch from its `true` branch plus `SplitSpec`’s $I \cup J = B/I \cap J = \emptyset$) — needed so `xStep`’s/`yStep`’s outputs match `genAtom`’s own if δ then Z else $M \setminus Z$ recursive shape at *both* signs, not just `true`. * **(b)** the interleaved families `combinedX/combinedY` and shared sign-interleaving map `combined`, plus the closed-form identification theorems ($(\text{atomPair } \delta n).1 = \text{genAtom } \text{combinedX } D_0.\text{master } (\text{combined}\delta \delta) (2*n)$, and the odd-depth half-step analogue; symmetrically for `combinedY/.2`) — proved by a single two-half-steps-at-a-time induction using (a). * **(c)** assembling the `hcore` fact $\forall \delta' n$, $\text{genAtom } \text{combinedX } D_0.\text{master } \delta' n = \emptyset \leftrightarrow \text{genAtom } \text{combinedY } D_1.\text{master } \delta' n = \emptyset$ for an *arbitrary* $\delta' : \mathbb{N} \rightarrow \text{Bool}$ (via de-interleaving δ' into a $\mathbb{N} \rightarrow \text{Bool} \times \text{Bool}$ history and re-applying (b)) from `atomPair_invariant` (even depths) and `xStep_spec_bit` (odd depths) — pure assembly, no new mathematical content. * **(d)** instantiating `Theorem88.lean`’s `transfer_dir/transfer_empty_iff/transfer_subset_iff/transfer_inter_empty_iff/transfer_double_subset_iff/transfer_inter_eq_iff` with $Z1 := \text{combinedX}$, $M1 := D_0.\text{master}$, $Z2 := \text{combinedY}$, $M2 := D_1.\text{master}$, `hcore` from (c) (verbatim reuse, mirroring (vi)(2)’s precedent), then specializing to even/even and odd/odd index pairs to extract and state the headline bidirectional facts: $X i \subseteq X j \leftrightarrow \text{XPseq } i \subseteq \text{XPseq } j$, $Y i \subseteq Y j \leftrightarrow \text{YPseq } i \subseteq \text{YPseq } j$, and their inter-empty/inter-eq analogues — the actual deliverable this sub-part exists for.

Already Pass (unaffected by the above scoping, kept in this row for continuity): reusing (vi)(3)’s fully generic `iUnion_mem_or_empty` (no new theory needed): `atomPair_fst_subset_master/atomPair_snd_subset_master` (`atomPair`’s sides are always $\subseteq D_0.\text{master}/D_1.\text{master}$, by induction from the base case and `atomPair_fst_subset/atomPair_snd_subset`, (v)(3)); then `XPseq_subset_master/XPseq_empty_or_mem` (via `xStep_snd_subset+atomPair_invariant`, and `iUnion_mem_or_empty` fed each `xStep` “+”-branch’s own `SplitSpec`’ `mem-or- $\emptyset$` guarantee) and the symmetric `YPseq_subset_master/YPseq_empty_or_mem` (via `yStep_fst_subset/xStep_fst_subset+xStep_spec_bit`, `iUnion_mem_or_empty` applied twice for `YPseq`’s doubly-indexed union). Zero sorry; whole-project lake build (3163 jobs) green; `#print axioms` on all six theorems give $\subseteq \{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$, matching the baseline.

Exercise 8.12(c)(vi)(5)(a)

Complete (vi)(4)(a)/(d)’s one-branch (`true-only`) I-formulas (`xStep_snd_eq_inter_XPseq`, `yStep_fst_eq_inter_YPseq`) into full two-branch closed forms giving `xStep`’s/`yStep`’s output at *either* sign directly in terms of `XPseq n/YPseq n` — the two-sided, half-step analogue of `Theorem88.lean`’s `atomU_succ_eq`.

`xStep_snd_succ_eq`: $(\text{xStep } D_1 \text{ hD}_1\text{nomin } (\text{atomPair } \delta n).1 (\text{atomPair } \delta n).2 (X n) b).2 = (\text{atomPair } \delta n).2 \cap (\text{if } b \text{ then } \text{XPseq } n \text{ else } D_1.\text{master} \setminus \text{XPseq } n)$, and `yStep_fst_`

`succ_eq`: the symmetric `YPseq/α`-side statement, at an arbitrary bit `b` for the `Y`-sub-step itself but with the `X`-sub-step's own bit kept fixed at `(δ n).1` (exactly as `yStep_fst_eq_inter_YPseq` already does)

Exercise 8.12(c)(vi)(5)(b)

Debugging note (real, not hypothetical) — a second instance of the (vi)(5)(a)-documented include/omit pitfall, one level subtler: `combinedδ_even/combinedδ_odd`'s own `omit hD0pos hD0diff hD1pos hD1diff hXmem hYmem hD0mne hD1mne` in annotation (copied from the neighboring `combinedX_even/combinedY_even`, which *do* need `D0 D1 hD0nomin hD1nomin`) omitted only 8 of the section's 10 blanket-included hypotheses, leaving `hD0nomin/hD1nomin` (and transitively `D0/D1`, since `hD0nomin : D0.NoMinimal` depends on `D0`) force-included as *unused* leading parameters — even though `combinedδ_even`'s statement (`combinedδ δ (2*k) = (δ k).1`) never mentions any of `D0/D1/hD0nomin/hD1nomin`. Bare `rw [combinedδ_even]` cannot unify these phantom parameters from the rewrite pattern (they don't occur in it), leaving them as dangling metavariables that Lean reports as bizarre leftover goals (`⊢ NeighborhoodSystem ?m, ⊢ NoMinimal ?D0, ⊢ Type ?u`) attached to the *enclosing* theorem's `:=` by position — easily mistaken for an elaborator bug in the surrounding proof, especially since the goals cascade to every caller (`genAtom_combinedX_succ_eq/Y`, then `atomPair_fst/snd_eq_genAtom`) and persist even with the caller's body replaced by `sorry` (since the phantom metavariables are created by elaborating the `rw` lemma itself, before the tactic block even runs). Root-caused by explicitly supplying `combinedδ_even`'s “hidden” args (`combinedδ_even D0 D1 hD0nomin hD1nomin δ n`) and observing the resulting *genuine* type mismatch (`δ` provided where `D0 : NeighborhoodSystem ?m` was expected) — confirming the extra params existed and were simply never constrained. **Fix:** extend both `omit` lines to the *full* 10-name list (adding `hD0nomin hD1nomin`), matching `combinedδ`'s own definition (which has no such annotation at all and correctly stays a bare 2-argument function, since a `def`'s equation-compiler body, unlike a `theorem`'s included context, only picks up variables actually referenced). **Lesson for future omit lines in this file:** the omitted list must be checked against the *lemma's actual conclusion*, not copy-pasted from a neighboring lemma that happens to need a different subset of the section's variables. Zero `sorry`; whole-project lake build green; `#print axioms on atomPair_fst_eq_genAtom/atomPair_snd_eq_genAtom` gives `⊆{propext, Classical.choice, Quot.sound}`, matching the baseline (choice inherited from `splitChoice'`, 8.12(c)(iii); no new taint).

Exercise 8.12(c)(vi)(5)(c)

The matching-emptiness fact $\forall (\delta' : \mathbb{N} \rightarrow \text{Bool}) \ n, \text{genAtom combinedX } D_0.\text{master } \delta' \ n = \emptyset \leftrightarrow \text{genAtom combinedY } D_1.\text{master } \delta' \ n = \emptyset$ — the `hcore` hypothesis `Theorem88.lean`'s `transfer_dir` needs, for the interleaved families.

pure assembly overall, no new mathematical content, but genuinely multiple Lean steps: (1) define the de-interleaving map and prove the round-trip identity feeding both later cases; (2) the even-index case, via (b) + `atomPair_invariant`; (3) the odd-index case, via (b)

Exercise 8.12(c)(vi)(5)(c)(1)

Given arbitrary $\delta' : \mathbb{N} \rightarrow \text{Bool}$, de-interleave it into $\delta \ k := (\delta' (2*k), \delta' (2*k+1)) : \mathbb{N} \rightarrow \text{Bool} \times \text{Bool}$, and prove that re-interleaving recovers δ' exactly: `combinedδ δ = δ'`.

`'deinterleavedδ (δ' : ℕ → Bool) (k : ℕ)`

Exercise 8.12(c)(vi)(5)(c)(2)

The even-index case of `hcore`: `genAtom combinedX D0.master δ' (2*n) = ∅ ↔ genAtom combinedY D1.master δ' (2*n) = ∅`.

`hcore_even (δ' : ℕ → Bool) (n : ℕ): 'rw [← combinedδ_deinterleavedδ δ', ← atomPair_fst_eq_genAtom ... (deinterleavedδ δ']`

Exercise 8.12(c)(vi)(5)(c)(3)

The odd-index case of `hcore`: `genAtom combinedX D0.master δ' (2*n+1) = ∅ ↔ genAtom combinedY D1.master δ' (2*n+1) = ∅`.

`hcore_odd (δ' : ℕ → Bool) (n : ℕ): 'rw [← combinedδ_deinterleavedδ δ', genAtom_`

combinedX_succ_eq ... (deinterleaved δ')

Exercise 8.12(c)(vi)(5)(c)(4)

Final assembly: $\forall (\delta' : \mathbb{N} \rightarrow \text{Bool}) \ n, \text{genAtom combinedX } D_0.\text{master } \delta' \ n = \emptyset \leftrightarrow \text{genAtom combinedY } D_1.\text{master } \delta' \ n = \emptyset$, the actual hcore deliverable of (c).

hcore $(\delta' : \mathbb{N} \rightarrow \text{Bool}) \ (n : \mathbb{N})$: 'rcases (by omega : $n \% 2 = 0 \vee n \% 2 = 1$)

Exercise 8.12(c)(vi)(5)(d)

The headline bidirectional transfer facts: $X \ i \subseteq X \ j \leftrightarrow XPseq \ i \subseteq XPseq \ j, Y \ i \subseteq Y \ j \leftrightarrow YPseq \ i \subseteq YPseq \ j$, and their inter-empty/inter-eq analogues (mirroring Theorem88.lean's transfer_subset_iff/transfer_inter_empty_iff/transfer_inter_eq_iff, doubled for both directions) — the actual deliverable of 8.12(c)(vi)(5), needed for the eventual DomainIso assembly in (vii).

one correction to the pre-plan: Theorem88.lean's transfer_subset_iff/transfer_inter_empty_iff/transfer_double_subset_iff/transfer_inter_eq_iff (unlike transfer_dir, which *is* fully generic and reused verbatim)

Exercise 8.12(c)(vi)(6)

literal mirror of (vi)(5) with the two sides' roles swapped (**superseded**): folded into (vi)(5) above once it became clear XPseq/YPseq (not a fresh $E := D_1$ instantiation) are the right pair of objects, and both sides' mem-or- $\emptyset$ /subset-of-master facts were proved together in one pass.

kept as a placeholder row (rather than deleted) for traceability with earlier HANDOFF.md checkpoints that still referred to a separate (vi)(6); see (vi)(5)'s row for the actual content and status.

Exercise 8.12(c)(vi)(7)

Bidirectional glue: whatever additional bridging (vii)'s final assembly needs to combine (vi)(5)'s and (vi)(6)'s two one-sided closed forms into a single, mutually-consistent correspondence between D_0 - and D_1 -neighbourhoods — resolved to: $X \ n/Y \ n$ and their atomPair-images $XPseq \ n/YPseq \ n$ are *always genuine neighbourhoods* (never merely “mem-or- $\emptyset$ ”), on both sides, unconditionally.

the vagueness resolved cleanly, with **no separate “mutual extension” hypothesis needed beyond NoMinimal on both sides** — a genuine (if small) new fact, not just transcription.

Exercise 8.12(c)(vii)

Part 7 of 7 — final assembly of (iv)/(v)/(vi)'s ingredients into the headline DomainIso $D_0 \ (D_0.\text{Element} \simeq_o D_1.\text{Element})$, completing Exercise 8.12(c).

scoped into 6 sub-parts 2026-07-04 (**8.12(c)(vii)(1)–(6)**), mirroring how (vi)(5)(c) was scoped before executing, then executed sub-part by sub-part, **all now Pass**.

Exercise 8.12(c)(vii)(1)

Add the covering/surjectivity hypotheses $hXcover : \forall S, D_0.\text{mem } S \leftrightarrow \exists n, S = X \ n$ and $hYcover : \forall S, D_1.\text{mem } S \leftrightarrow \exists n, S = Y \ n$ to the construction (mirroring Theorem88a.lean's hcover), plus Scott's zero-convention hypotheses $hX0 : X \ 0 = D_0.\text{master}/hY0 : Y \ 0 = D_1.\text{master}$ (mirroring he0) — the missing hypotheses section AtomPair's docstring already assumed but never formally declared.

new section Iso opened, re-declaring the full D_0/D_1 /positivity/DiffClosed/NoMinimal/ $X/Y/hXmem/hYmem$ /variable list from section AtomPair plus the four new hypotheses $hXcover, hYcover, hX0, hY0$ as variables. No proof content; pure declaration.

Exercise 8.12(c)(vii)(2)

$XPseq \ 0 = D_1.\text{master}$ and $YPseq \ 0 = D_0.\text{master}$, given (1)'s $hX0/hY0$.

mirrors Theorem88.lean's Yseq_zero_eq_master argument via splitChoice'_isSplitSpec: at $n = 0$, atomPair's pairing is $(D_0.\text{master}, D_1.\text{master})$ regardless of sign sequence, so $hX0 : X \ 0 = D_0.\text{master}$ makes the X-sub-step's $D_0.\text{master} \setminus X \ 0 = \emptyset$, forcing (by SplitSpec's uniqueness-of-empty-difference clause) the split's “-”-branch empty and its “+”-branch to swallow all of $D_1.\text{master}$; $XPseq_zero$ then follows by Set.Subset.antisymm against the already-proved $XPseq_subset_master$ using subset_XPseq at the empty history (Fin.elim0).

Exercise 8.12(c)(vii)(3)

The cross-parity specializations of `transfer_subset_combined/transfer_inter_eq_combined` needed for `up_mem/inter_mem`'s mixed cases: $X \text{ } i \subseteq Y \text{ } pseq \text{ } j \leftrightarrow X \text{ } pseq \text{ } i \subseteq Y \text{ } j$ (order) and an `exists_inter_index`-style pair of lemmas (mirroring `Theorem88a.lean`'s `exists_inter_index_of_dmem/_of_nonempty`) built from `X_inter_eq_iff_XPseq_inter_eq/YPseq_inter_eq_iff_Y_inter_eq` ((5)(d)) plus (1)'s `hXcover/hYcover`, for both the $D_0 \rightarrow D_1$ and $D_1 \rightarrow D_0$ directions.

`X_subset_YPseq_iff_XPseq_subset_Y/YPseq_subset_X_iff_Y_subset_XPseq` are direct specializations of `transfer_subset_combined` at the mixed indices $(2i, 2j+1)/(2i+1, 2j)$, using the same `Set.inter_eq_self_of_subset_right` bookkeeping as (5)(d)'s same-parity headline facts — zero new proof machinery, exactly as scoped.

Exercise 8.12(c)(vii)(4)

`toD1 : D0.Element → D1.Element`, the pushforward filter $\{T \mid \exists n, T = X \text{ } pseq \text{ } n \wedge x \text{ } .mem (X \text{ } n)\}$, proved to satisfy all of `Element`'s axioms (`sub`, `master_mem`, `inter_mem`, `up_mem`).

`sub/master_mem/inter_mem` are immediate from `XPseq_mem/XPseq_zero/exists_inter_index_X`, direct transcription of `Theorem88a.lean`'s `toDprimeU`. `up_mem` needed **one genuinely new lemma beyond (3)'s scoping**: `'XPseq_eq_Y_iff_X_eq_YPseq (j k)`

Exercise 8.12(c)(vii)(5)

`toD0 : D1.Element → D0.Element`, the symmetric pullback filter $\{S \mid \exists n, S = Y \text{ } pseq \text{ } n \wedge y \text{ } .mem (Y \text{ } n)\}$, proved to satisfy all of `Element`'s axioms.

exact mirror of (4) with the two sides' roles swapped (`YPseq_mem/YPseq_zero/exists_inter_index_Y` for `sub/master_mem/inter_mem`; `YPseq_subset_X_iff_Y_subset_XPseq/hXcover/hYcover/(4) XPseq_eq_Y_iff_X_eq_YPseq` for `up_mem`, now used in its `.mp` direction rather than `.mpr/.symm` since the roles are swapped).

Exercise 8.12(c)(vii)(6)

`toD1/toD0` are mutually inverse and order-preserving/-reflecting, assembling `domainIso812c : DomainIso D0 D1` and the headline `isomorphic_812c : D0 ≅D D1`, completing Exercise 8.12(c) in full.

direct transcription of `Theorem88a.lean`'s `domainIso`'s `left_inv/right_inv/map_rel_iff'` fields (renamed `domainIso812c/isomorphic_812c` — plain `domainIso` already exists in `Theorem88a.lean` within the same `Scott1980.Neighborhood` namespace, a name clash only surfaced by the whole-project `lake build`, not the single-file build). `left_inv/right_inv` use (4)'s `XPseq_eq_Y_iff_X_eq_YPseq` at each of its two directions (`.mp/.mpr`, with an extra outer `.symm` chain on one side)

Exercise 8.12(d)

- *Harder*: `Theorem 8.8(b)`'s effective refinement only ever needs to split `U`'s side (`Theorem 8.8(a)`'s `atomU/Yidx` construction only ever calls `U_no_minimal`, never anything on the arbitrary `D`-side — `D` only needs its *index relations* decidable, via the fully generic `DAtomDecidable.lean`, already reusable as-is for **both** `D0` and `D1` here). Exercise 8.12(c)'s `xStep/yStep`, by contrast, split **both** sides alternately (`splitChoice' D1/splitChoice' D0`), so an effective refinement needs an *effective, computable replacement for NoMinimal's splitting existential on both D₀ and D₁* — and unlike `U`, `D0/D1` are *arbitrary* effectively-given systems with no special concrete structure (no interval/dyadic representation) to build an explicit `splitU`-style deterministic split from.

- *More tractable*: `exists_split'`'s proof (`Exercise812c.lean` lines 226–258) shows the split witness `I, J` **never needs to relate to the probe set `Xn` beyond a nonemptiness match** ($A \cap Xn = \emptyset \leftrightarrow I = \emptyset, A \setminus Xn = \emptyset \leftrightarrow J = \emptyset$) — in the genuine-split case it is *any* `NoMinimal`-witnessed disjoint nonempty pair `Y, Z` of the target `B`, with no further constraint. This means an effective split doesn't need to track *which* piece of `B` “belongs to” which piece of `A` — all of that cross-side bookkeeping is already

handled entirely by the *transfer* lemmas (`transfer_dir/transfer*_combined`), independently of the split’s specific choice. So the needed new theorem is comparatively clean to *state*: “given a `ComputablePresentation` P of a `NoMinimal` system E and (the index of) some P .`mem`-witnessed B , computably/effectively produce indices m, m' with $P.X\ m, P.X\ m'$ a valid `NoMinimal`-split of B ” — plausible via **unbounded search** (`Nat.rfind`-style) over pairs (m, m') , checking the *decidable* predicate “ $P.X\ m \cap P.X\ m' = \emptyset \wedge P.X\ m \cup P.X\ m' = B$ ” (built from `interEq_computable`/a union-decider — nonemptiness is free, since `NoMinimal.mem_ne_empty` (8.12(c)(vi)(7)) already shows no P .`mem` set is ever $\emptyset$), **guaranteed to halt** because `NoMinimal`’s `Prop`-level existence guarantees at least one such pair is enumerated eventually. This is exactly the “unbounded search inside an r.e. predicate” idiom already used pervasively in Chapter 7–8 (e.g. Theorem 7.6’s `fixMap_isComputable`), and `Definition71.lean`’s own `ComputablePresentation.inter` docstring already flags this exact search ($\mu k. X_n \cap X_m = X_k$) as the *non-primitive-recursive* fallback its `inter` field is designed to avoid needing generically.

- **Design decision needed before executing (flagged, not resolved, pending user input):** should (d) attempt the **fully generic** “`NoMinimal` + effectively given $\implies$ effective split via search” theorem above (real new math, extra effort, but then applies automatically to *any* effectively-given D_0/D_1 , potentially even letting (e)/(f) skip building bespoke concrete splits for U/V), or take the **safer, Theorem88.lean-style route**: generalize `xStep/yStep/atomPair/.../domainIso812c` over an **abstract hypothesis** “`splitX/splitY` are computable and satisfy `SplitSpec`” (mirroring `Theorem88b.lean`’s “generalize over abstract `split`”), *deferring* “does such a split exist” to whoever instantiates the theorem — i.e. to (e)/(f), which would then need to build bespoke computable splits for U (largely reusable from Theorem 8.8(b)’s existing `SplitU.lean`) and V (new, but expected to be comparably easy given V ’s neighbourhoods are already coarse dyadic-residue unions). **The safer route is recommended**: it mirrors this project’s established pattern of parametrizing over abstract hypotheses first and instantiating later, and avoids committing to the generic search theorem’s added risk/effort inside (d) itself (that theorem, if wanted, could always be added as a *separate*, later strengthening). Sub-parts below assume the safer route.

Exercise 8.12(d)(1)

Generalize `Exercise812c.lean`’s `xStep/yStep/atomPair` (through `atomPair_disjoint/atomPair_fst_subset_master/atomPair_snd_subset_master`) over abstract `splitX : Set  $\alpha \rightarrow$  Set  $\beta \rightarrow$  Set  $\alpha \rightarrow$  Set  $\beta \times$  Set  $\beta$ /splitY : Set  $\beta \rightarrow$  Set  $\alpha \rightarrow$  Set  $\beta \rightarrow$  Set  $\alpha \times$  Set  $\alpha$` hypotheses satisfying `SplitSpec`’ (mirroring `Theorem88b.lean`’s “6a”).

Scope adjustment, discovered during execution: the original wording also listed `XPseq/YPseq/combinedX/combinedY/toD1/toD0/domainIso812c` as needing a parallel classical-abstract-split generalization here. This turns out to be unnecessary extra work — those are all downstream consequences of `atomPair`’s invariant/disjointness/subset facts alone (never touching the split function or its `SplitSpec`’ proof directly), so (d)(3)–(d)(6) build the *code-level* analogues (`atomPairCode`, `XPseqCode/YPseqCode`, computability, final `EffectiveIso`)

Exercise 8.12(d)(2)

Define what “`splitX/splitY` computable relative to presentations P_0, P_1 ” means.

‘`IsComputableSplit {V W} (P : ComputablePresentation V) (Q : ComputablePresentation W)`

Exercise 8.12(d)(3)

1. **A missing prerequisite.** `Theorem88d.lean`’s central design trick — never build a `Set`-valued split/step function; track an explicit presentation-index as state from the very first

step, since there is no way to effectively recover “the” canonical index of an arbitrary **Set** — means **atomPairG**’s recursion needs **both** intersection *and* set-difference to stay effectively indexed at every step. But Scott’s **ComputablePresentation** (Definition 7.1) only requires the **intersection** relation/index (**inter/cons_computable**) to be computable — there is no analogous “diff index” primitive for $\setminus$. **Theorem88d.lean** never needed one because **U/V** have bespoke, concrete diff constructions (**SplitU.lean**, bitmask XOR); for an *arbitrary* effectively-given D_0/D_1 there is no such guarantee, so a new prerequisite hypothesis (mirroring **inter/cons_computable**’s shape, but for $\setminus$) is needed before **atomPairCode** can be built at all.

2. **The scale of the state machine itself.** The bulk of **Theorem88d.lean**’s ~1300 lines is not the recursion’s “happy path” but its **junk/decidable-emptiness tracking** (**atomUEmpty**, **datumDec**, freezing at a junk sentinel once a branch goes empty, **atomUCode_disjoint**’s restriction to non-junk branches) — needed because **SplitSpec**’s “direct” sub-step ($A \cap X_n / A \setminus X_n$) can genuinely be empty, unlike the “split” sub-step’s two outputs (never literally $\emptyset$, by **NoMinimal.mem_ne_empty**, (c)(vi)(7)). **atomPairG** alternates **two** interleaved sides (**xStep**’s direct-then-split on D_1 , **yStep**’s direct-then-split on D_0), so a faithful **atomPairCode** needs this whole apparatus built — and kept mutually in sync — **twice**, not once.
3. **A simplification found while analyzing the above (partial compensation):** decidability of the *split* sub-step’s own emptiness turns out to be **free**, not a separate hypothesis: **SplitSpec**’s defining clause $A \cap X_n = \emptyset \leftrightarrow (\text{split } A \ B \ X_n).1 = \emptyset$ means “is the split output empty” reduces *exactly* to “is the direct side’s intersection/difference empty” — i.e. once the direct-side deciders exist (**cons_computable** for $\cap$, the new hypothesis’s decider for $\setminus$), the split side’s decidability comes along for free via this **iff**, with no separate decidability field needed on **IsComputableSplit**.

• **Tentative sub-part breakdown (6 parts, pending confirmation):**

1. **(d)(3)(a)** — **IsComputableDiff {V} (P : ComputablePresentation V)**: the missing prerequisite from finding 1 above, mirroring **ComputablePresentation**’s own **inter/cons_computable** pair but for $\setminus$ — a primitive-recursive **diffIdx** : $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ with **diffIdx_spec** : $(\exists k, X\ k = X\ n \setminus X\ m) \rightarrow X\ (\text{diffIdx } n\ m) = X\ n \setminus X\ m$ (mirroring **inter_spec**’s shape exactly), plus a **RecDecidable₂**-style decider for “ $X\ n \setminus X\ m$ is a genuine neighbourhood” (mirroring **cons_computable**, needed per finding 3 to get the split-side’s decidability for free). One structure serves both P_0 and P_1 symmetrically, same design as **IsComputableSplit**.
2. **(d)(3)(b)** — the per-depth state representation: a **Nat**-code packing (in **Theorem88d.lean**’s **packState** style) both sides’ current presentation-index (D_0 -side index into P_0 , D_1 -side index into P_1) plus a junk/non-junk flag per side, and the **X**-sub-step’s state transition (intersect/diff the D_0 -side index directly by **n** via $P_0.\text{inter}/(a)$ ’s **diffIdx**, split the D_1 -side index via (2)’s **IsComputableSplit**) as a single **Nat.Primrec** step function.
3. **(d)(3)(c)** — the **Y**-sub-step’s state transition (symmetric: intersect/diff the D_1 -side index directly by **n**, split the D_0 -side index), combined with (b) into the full $n \rightarrow n + 1$ transition **atomPairCodeState**, mirroring **Theorem88d.lean**’s **atomUCodeState/atomStep**.
4. **(d)(3)(d)** — per-step correctness: whenever the recorded state is non-junk, the packed indices’ $P_0.X/P_1.X$ values literally equal **atomPairG**’s corresponding component at that depth, mirroring **genAtom_atomUCode**.
5. **(d)(3)(e)** — the junk invariant: once junk, a side stays junk forever (mirroring **atomUEmpty_mono**); non-junk states are always genuine D_0/D_1 -neighbourhoods (mirroring **atomUCode_mem**).

6. **(d)(3)(f)** — disjointness: for two sign-sequences disagreeing below depth n with *both* still non-junk at depth n , the corresponding indexed sets are disjoint on both sides, mirroring `atomUCode_disjoint`.

Exercise 8.12(d)(3)(a)

`IsComputableDiff`: a computable “diff index” hypothesis mirroring `ComputablePresentation`’s `inter/cons_computable` pair, but for $\setminus$ (the missing prerequisite Scott’s Definition 7.1 doesn’t provide).

`diffIdx` : $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$, `Nat.Primrec (diffIdx_primrec)`, with ‘`diffIdx_spec` : $(\exists k, X \text{ k} = X \text{ n } X \text{ m}) \rightarrow X \text{ (diffIdx n m)}$

Exercise 8.12(d)(3)(b)

The X -sub-step’s code-level state transition.

Scope simplification, discovered during execution: the per-depth state packs both sides’ current presentation-index plus a **single shared** junk/non-junk flag (`packState2 idx0 idx1 junk`) — *not* “one per side” as originally tentatively scoped, since (d)(1)’s own `atomPairG_invariant (ihAB)`

Exercise 8.12(d)(3)(c)

The Y -sub-step’s code-level state transition, combined into the full `atomPairCodeState`.

`ySubStep`, symmetric to (b)’s `xSubStep` (intersect/diff the D_1 -side index directly against $P_1.X \text{ n}$, split the D_0 -side index via `hSplitY` : `IsComputableSplit P1 P0 splitY`), reusing the *same* packed-argument projections `xwN/xwB1/xwS` unchanged (pure $\mathbb{N}$ -arithmetic, not tied to X). Composed with `xSubStep` into the full $n \rightarrow n + 1$ transition `atomPairStep` (one `xSubStep` with `bit rem % 2`, then one `ySubStep` with `bit (rem / 2)`)

Exercise 8.12(d)(3)(d)

Per-step correctness of `atomPairCodeState` against `atomPairG`.

whenever the recorded state is non-junk, the packed indices’ $P_0.X/P_1.X$ values literally equal `atomPairG`’s corresponding component at that depth, mirroring `genAtom_atomUCode`. `deltaPair k` : $\mathbb{N} \rightarrow \text{Bool} \times \text{Bool}$ reads the two-bits-per-depth sign sequence off bit-source k (`(k / 4 ^ i)`)

Exercise 8.12(d)(3)(e)

The junk invariant and validity of `atomPairCodeState`.

once junk, a side stays junk forever (mirroring `atomUEmpty_mono`); non-junk states are always genuine D_0/D_1 -neighbourhoods (mirroring `atomUCode_mem`). Turned out much smaller than (d)(3)(d): the hard direction (“junk propagates *backward*”)

Exercise 8.12(d)(3)(f)

Disjointness of `atomPairCodeState` across disagreeing, non-junk sign-sequences.

for two sign-sequences disagreeing below depth n with *both* still non-junk at depth n , the corresponding indexed sets are disjoint on both sides, mirroring `atomUCode_disjoint`. Completes 8.12(d)(3).

Exercise 8.12(d)(4)

1. **A missing prerequisite, structurally different from (d)(3)(a)’s.** `Theorem88d.lean`’s `unionUX n m := appendCode (canonCode n) (canonCode m)` is **bespoke to U**: its codes are literal lists of rational intervals, freely concatenable, and `U_mem_union_UX` shows the union of *any* two U -neighbourhoods is unconditionally again a U -neighbourhood. Neither fact generalizes: Scott’s `ComputablePresentation` (Definition 7.1) gives an intersection index (`inter`) because `NeighborhoodSystem.inter_mem` makes intersection a *primitive* closure property — there is no analogous union primitive, because `V.mem` is **not** assumed closed under union at all. `Exercise812c.lean`’s `XPseq_mem/YPseq_mem` (already `Pass`, (c)(vii)) show the specific growing unions built by this construction *are* always genuine D_1/D_0 -neighbourhoods, but only via a bespoke `NoMinimal/SplitSpec`’ argument giving mere `Prop`-level existence (`P.surj`) of an index — not an effective way to compute one.

So, exactly as (d)(3)(a) needed a fresh `IsComputableDiff` because Definition 7.1 has no diff-index primitive, (d)(4) needs a fresh prerequisite because Definition 7.1 has no union-index primitive either.

2. **The union sizes match `deltaPair`'s existing base-4 convention.** $\text{XPseq } n = \bigcup \delta' : \text{Fin } n \rightarrow \text{Bool} \times \text{Bool}, \dots$ unions over 4^n sign-pair-sequences (not 2^n as in `Theorem88d.lean`'s single-sided `YseqCode`), and $\text{YPseq } n$ unions over an *extra* $\text{bx} : \text{Bool}$ factor on top, i.e. $2 \cdot 4^n$. Conveniently, $i < 4^n \leftrightarrow$ a length- n prefix of bit-pairs is exactly (d)(3)(d)'s already-built `deltaPair` convention $((k / 4^i) \% 2 / (k / 4^{i/2} \% 2))$ — no new bit-source encoding needed, and (d)(1)'s `atomPairG_congr` already shows the depth- n value never depends on bits beyond position n , so extending a length- n prefix $i < 4^n$ to a full bit-source (e.g. literally $k := i$, padding implicitly with `(false, false)`) is immaterial — no fresh *code*-level congruence lemma is needed beyond what (d)(1) already proved at the classical `atomPairG` level.
3. **A classical generalization (d)(1)'s `docstring` explicitly deferred into this sub-part.** `Exercise812c.lean`'s `XPseq/YPseq` are hardcoded to `xStep/atomPair` (the `splitChoice'`-based, non-abstract construction) — (d)(1)'s `docstring` flags this exactly (“the original scoping listed `XPseq/YPseq/...` as also needing a parallel classical-abstract-split generalization ... (d)(3)–(d)(6) can build the code-level analogues ... directly on top of `atomPairG`”). Mirroring `Theorem88d.lean`'s own two-layer pattern (code-level `atomUCode/YseqCode` and the classical anchor `genAtom`, linked by `atomUCode_eq_genAtomYseqCode`), the cleanest route is a light, mechanical `XPseqG/YPseqG` (verbatim transcriptions of `XPseq/YPseq` over abstract `splitX/splitY`, exactly as (d)(1) transcribed `atomPair` into `atomPairG`) with their own `mem/zero/subset` facts transcribed from `XPseq_mem/XPseq_zero/etc.` — giving (d)(3)–(d)(6) a `Set`-level anchor to connect `XPseqCode` back to, without needing the original `splitChoice'`-hardcoded `XPseq` at all.
4. **A downstream simplification found while investigating (d)(6) (does not reduce (d)(4)'s own scope, but bounds it and affects sub-part boundaries below):** `Approximable.lean`'s `ofIso` (**Theorem 2.7** — *any* domain isomorphism $e : V_0.\text{Element} \simeq_o V_1.\text{Element}$ induces an `ApproximableMap` $V_0 \rightarrow V_1$ choice-free) plus `Theorem88n.lean`'s already-proven precedent (`isoInj := ofIso (domainIsoCode P)`, reducing `IsComputableMap` to a raw-index inclusion test) strongly suggest (d)(6)'s `EffectiveIso.toMap/.invMap` should be `ofIso domainIso812c/ofIso domainIso812c.symm` directly, rather than re-deriving bespoke `ApproximableMaps` that duplicate `toD1/toD0`'s `up_mem` reasoning from scratch. **Crucial difference from `Theorem88n.lean`:** there, D/D' share *one* index family via a literal primitive-recursive involution `eIdx`, collapsing `isoInj`'s relation to a single reindexed `incl_computable` query with no search at all. Here, D_0/D_1 have genuinely *independent* index families (P_0, P_1) , so $(\text{ofIso domainIso812c}).\text{rel } (P_0.X \ n) \ (P_1.X \ m)$ unfolds (via `toD1`'s definition and `V.principal`) to $\exists n', P_1.X \ m = \text{XPseq } n' \wedge P_0.X \ n \subseteq P_0.X \ n'$ — genuinely needing `XPseq n'`'s P_1 -index (i.e. `XPseqCode n'`) to reduce “ $P_1.X \ m = \text{XPseq } n'$ ” to something checkable. So `ofIso` simplifies (d)(6)'s *assembly* (no bespoke `ApproximableMap` axioms to re-prove) but does **not** remove (d)(4)'s core deliverable. **Correction (2026-07-05, (d)(5)'s scoping):** the specific claim above that `ofIso domainIso812c` is the right map to make computable is **wrong** — `domainIso812c`'s `XPseq/YPseq` are hardcoded to the classical, non-computable `splitChoice'`, disconnected from this track's effective `splitX/splitY`; (d)(5) needs a *fresh* order-iso built directly on `atomPairG/XPseqCode/YPseqCode` instead (see (d)(5)'s row, finding 1, for the full correction). The $\exists n', P_1.X \ m = [\dots] n' \wedge P_0.X \ n \subseteq P_0.X \ n'$ unfolding shape and “`XPseqCode/YPseqCode` get consumed” punchline above are otherwise directionally correct, once `XPseq n'` is replaced by $P_1.X \ (\text{XPseqCode } n')$.

- **Design decision needed before executing (flagged, not resolved, pending user input — same fork as (d)’s own docstring, now recurring for $\cup$ instead of the effective split):** mirroring (d)(3)(a)’s `IsComputableDiff`, should the new union prerequisite be (a) an **abstract hypothesis** `IsComputableUnion` (mirroring `IsComputableDiff`’s shape: $\text{unionIdx} : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$, Nat.Primrec , $\text{unionIdx_spec} : (\exists k, X\ k = X\ n \cup X\ m) \rightarrow X\ (\text{unionIdx}\ n\ m) = X\ n \cup X\ m$, plus a RecDecidable_2 existence-decider), deferred to instantiation exactly like `IsComputableSplit/IsComputableDiff` were; or (b) derived via **unbounded search** (Nat.rfind), since the *specific* unions arising in this fold are already known non-empty/genuine by `NoMinimal/XPseq_mem`, mirroring (d)’s own flagged-but-declined generic-search alternative. **The abstract-hypothesis route is recommended**, for the same reasons (d)’s docstring gave: consistent with this project’s established pattern, and defers “does U/V actually admit an effective union index” to whoever instantiates at (e)/(f).

- **Tentative sub-part breakdown (4 parts, pending confirmation):**

1. (d)(4)(a) — `IsComputableUnion {V} (P : ComputablePresentation V)`: the missing prerequisite from finding 1, mirroring (d)(3)(a)’s `IsComputableDiff` shape exactly but for $\cup$ (one generic structure, reused for both P_0 and P_1 , same design as `IsComputableSplit/IsComputableDiff`).
2. (d)(4)(b) — `XPseqG/YPseqG`: the classical `Set`-level generalization of `Exercise812c.lean`’s `XPseq/YPseq` over abstract `splitX/splitY` (finding 3), transcribing `XPseq_mem/XPseq_zero/YPseq_mem/YPseq_zero` verbatim onto the abstracted definitions — expected genuinely light/mechanical, per (d)(1)’s own docstring assessment, so bundled as one sub-part covering both sides.
3. (d)(4)(c) — `XPseqCode`: the X-side fold (`XFoldStep/XFold`, mirroring `yFoldStep/yFold` over $i < 4^n$ using `deltaPair/(a)’s unionIdx` instead of $2^n/\text{unionUX}$) plus its closed-form membership characterization (mirroring `mem_UX_YseqCode_iff`) and the link back to (b)’s `XPseqG` (mirroring `atomUCode_eq_genAtomYseqCode`).
4. (d)(4)(d) — `YPseqCode`: the Y-side fold, symmetric to (c) but with the extra `bx : Bool` union layer ($2 \cdot 4^n$ total, an outer 2-way union of two inner 4^n -folds) plus its closed-form characterization and link to (b)’s `YPseqG`.

Exercise 8.12(d)(4)(a)

`IsComputableUnion`: a computable “union index” hypothesis mirroring `ComputablePresentation`’s `inter/cons_computable` pair, but for $\cup$ (the missing prerequisite Scott’s Definition 7.1 doesn’t provide, and `Theorem88d.lean`’s `unionUX` doesn’t generalize past).

direct structural mirror of (d)(3)(a)’s `IsComputableDiff`, but for $\cup$ instead of $\setminus$: `IsComputableUnion` P bundles a primitive-recursive $\text{unionIdx} : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ with ‘ $\text{unionIdx_spec} : (\exists k, X\ k = X\ n \cup X\ m) \rightarrow X\ (\text{unionIdx}\ n\ m)$

Exercise 8.12(d)(4)(b)

originally scoped as `XPseqG/YPseqG` (a classical `Set`-level generalization of `XPseq/YPseq` over abstract `splitX/splitY`); **re-scoped during execution** (see below) to `NeighborhoodSystem.mem_union_of_mem`, the actual prerequisite (d)(4)(c)/(d)’s folds need.

Scope simplification, discovered during execution (documented in-file as a docstring, mirroring (d)(1)’s own precedent for flagging such findings): the originally-planned `XPseqG/YPseqG` turns out to be both unnecessary *and* the wrong shape. `XPseq_mem` (`Exercise812c.lean`)

Exercise 8.12(d)(4)(c)

Exercise 8.12(d)(4)(c)(i): one-step 4-way classical reunion

Exercise 8.12(d)(4)(c)(ii): classical covering induction

Exercise 8.12(d)(4)(c)(iii): encode sign-histories as deltaPair-matching bit-sources

Exercise 8.12(d)(4)(c)(iv): non-trivial intersection with $P_0.X\ n$, still classical

Exercise 8.12(d)(4)(c)(v): the converse-biconditional — (c)’s originally-named gap, now itself a concrete induction

Exercise 8.12(d)(4)(c)(vi): assemble into the unconditional “found” fact Exercise 8.12(d)(4)(d)

Exercise 8.12(d)(4)(d)(i): one-step 4-way classical reunion, D_1 -side

Exercise 8.12(d)(4)(d)(ii): classical covering induction, D_1 -side

Exercise 8.12(d)(4)(d)(iii): encode sign-histories as deltaPair-matching bit-sources, D_1 -side

Exercise 8.12(d)(4)(d)(iv): non-trivial intersection with $P_1.X\ n$, still classical

Exercise 8.12(d)(4)(d)(v): the converse-biconditional, D_1 -side

Exercise 8.12(d)(4)(d)(vi): assemble into the unconditional “found” fact, doubled over bx Exercise 8.12(d)(5)

1. **Correction to the (d)(4)/(d)(6) cross-reference notes below (flagged 2026-07-04, found wrong on inspection 2026-07-05):** `domainIso812c` cannot be reused as-is. Those notes proposed `toMap := ofIso domainIso812c`, betting that `(ofIso domainIso812c).rel`’s unfolding would land on `XPseqCode/YPseqCode`. It does not: `domainIso812c`’s `toD1/toD0` are defined via `XPseq/YPseq (Exercise812c.lean)`, which are hardcoded (through `atomPair`) to the *classical, choice-derived* `split splitChoice' D1 hD1nomin / splitChoice' D0 hD0nomin` — a **specific, non-constructive** witness of `NoMinimal`’s splitting existential, fixed once `hD0nomin/hD1nomin` are given, with **no reason to coincide** with whatever effectively-computable `splitX/splitY` (satisfying `SplitSpec'`) this sub-part is handed. `SplitSpec'` does not pin down a *unique* split of a given set — different valid splits generally carve out different disjoint nonempty pieces — so `atomPairG`’s (hence `XPseqCode`’s/`YPseqCode`’s) recovered neighbourhoods are, in general, literally different sets from `XPseq`’s/`YPseq`’s. Reusing `domainIso812c` would leave `(ofIso domainIso812c).rel (P0.X n) (P1.X m)`’s unfolding stuck on $\exists k, P_1.X\ m = XPseq\ D_0\ D_1\ hD_0nomin\ hD_1nomin\ P_0.X\ P_1.X\ k \wedge P_0.X\ n \subseteq P_0.X\ k$ — a condition about `splitChoice'`’s *classical* recovered sets, with no bridge to the *effective* `splitX/splitY/XPseqCode/YPseqCode` apparatus this whole (d) track exists to build. (d)(4)’s and (d)(6)’s rows have been annotated with a pointer to this correction.
2. **The right fix: a fresh, generalized order-iso, built directly on `atomPairG/XPseqCode/YPseqCode`, mirroring `Exercise812c.lean`’s *entire* §`AtomPair`(vi)(5)(b)–(6) interleaving layer and §`Iso` assembly (c)(vii), generalized over `splitX/splitY`.** Unlike (d)(4)(c)/(d)(4)(d)’s code-level folds — which bypassed that classical interleaving machinery entirely (per (d)(4)’s own finding 3: `XPseq_mem`’s heavy `combinedX/combinedY/transfer_inter_empty_combined` detour is unneeded for bare genuineness) — this sub-part’s *cross-family order*

comparisons (`up_mem`'s core content) have no such shortcut found yet: they inherently compare an arbitrary raw index $P_0.X \ i$ against a recovered union $P_1.X \ (\text{YPseqCode } j)$'s "other side", which is exactly what `Exercise812c.lean`'s `combinedX/combinedY/combined $\delta$ /genAtom-interleaving/transfer_subset_combined/transfer_inter_eq_combined` machinery (~300 lines, (c)(vi)(5)(b)–(vi)(6)) was built to handle. **Grep-confirmed:** `Exercise812d.lean` currently has *zero* generalized analogues of any of this (`combinedXG/genAtomG/transfer_*G` all absent) — it is unstated, not merely deferred.

3. **A genuine simplification specific to (d) (not available to (c)(vii)'s bare-enumeration setting):** `inter_mem/exists_inter_index_X/exists_inter_index_Y` need no interleaving detour at all. (c)(vii)'s `toD1/toD0` needed `exists_inter_index_X/Y` (via `hXcover/hYcover` plus the transfer layer) *only* because their X/Y parameters are bare index-enumeration functions with no built-in intersection primitive. Here, P_0/P_1 are full `ComputablePresentations` (Definition 7.1): $P_0.inter/P_1.inter$ (with `inter_spec, inter_primrec`) already hand back a primitive-recursive intersection index directly. So the generalized `toD1Code/toD0Code`'s `inter_mem` field can cite $P_1.inter \ i \ j / inter_spec$ directly, with **no classical existence detour and no `hXcover/hYcover` hypotheses needed for this field at all** — a genuine scope reduction versus (c)(vii)'s own proof, not just a mechanical transcription.
4. **The zero/master facts likely need no interleaving either.** (c)(vi)(2)'s `XPseq_zero/YPseq_zero` are proved directly from `atomPair`'s own $n = 0$ unfolding plus `SplitSpec'` (not via `combinedX/genAtom`) — the generalized `atomPairG/XPseqCode/YPseqCode` analogues should transcribe the same direct argument, expected light.
5. **The exercise's actual literal target, once the order-iso exists, is a clean `Theorem88n.lean`-style computability proof — genuinely new content, not yet attempted.** With a generalized `domainIsoCode812d : DomainIso D0 D1` in hand (built from `toD1Code/toD0Code`), of `iso domainIsoCode812d`'s relation unfolds (via `toD1Code`'s definition and `V.principal`, mirroring `isoInj_rel_iff_incl`) to $\exists k, P_1.X \ m = P_1.X \ (\text{XPseqCode } k) \wedge P_0.X \ n \subseteq P_0.X \ k$ — now genuinely checkable: $P_1.X \ m = P_1.X \ (\text{XPseqCode } k)$ is `P1.eq_computable` (Definition 7.1, generic on any `ComputablePresentation`) reindexed by the already-`Nat.Primrec XPseqCode (primrec_XPseqCode, (d)(4)(c))`, conjoined with `P0.incl_computable`; the outer $\exists k$ over a decidable body is `REPred.of_iff-r.e.`, exactly `isoInj_isComputableMap`'s pattern. Symmetric for the $D_1 \rightarrow D_0$ direction via `YPseqCode`.
- **Design decision needed before executing (flagged, not resolved, pending user input):** should (d)(5)'s cross-family order facts (finding 2) be built by a **full, careful generalization** of `Exercise812c.lean`'s `combinedX/combinedY/genAtom-interleaving/transfer_* chain` (the *safe*, guaranteed-to-work route, directly mirroring ~300 lines of already-correct precedent line-for-line with `splitChoice' → splitX/splitY` and `atomPair → atomPairG` substitutions), or should the first executing session **first spend a bounded effort searching for a shortcut** directly from already-proven `atomPairG`-level facts (`atomPairG_disjoint, atomPairG_invariant, xStepG_snd_union, (d)(4)(c)/(d)(4)(d)`'s `mem_XPseqCode_iff_unconditional/mem_YPseqCode_iff_unconditional`) that might avoid rebuilding the interleaving layer wholesale? **The bounded-search-then-fallback route is recommended:** this project's own history ((d)(4)(b)'s `XPseqG/YPseqG` "unnecessary and not the right shape" finding) shows the heavy classical detours from (c) are sometimes avoidable once code-level closed forms are already in hand, so a short investigation is cheap insurance before committing to the larger transcription — but the full-transcription route remains the fallback of known, bounded size if no shortcut turns up.
- **Sub-part breakdown:** re-scoped into 6 dependent sub-goals, (d)(5)(a)–(f) below, each its own block ((a) → (b) → (c) → (d) → (e) → (f), strictly sequential —

every later block's target hypothesis-list is a strict superset of the ones before it, exactly as (d)(4)(c)'s (c)(i)–(vi) chain was).

Exercise 8.12(d)(5)(a): zero/master facts for XPseqCode/YPseqCode

- $hX0 : P_0.X \ 0 = D_0.master$, $hY0 : P_1.X \ 0 = D_1.master$ (new section hypotheses, threaded through every later sub-part).

- theorem $XPseqCode_zero : P_1.X \ (XPseqCode \ P_0 \ P_1 \ hDiff0 \ hDiff1 \ splitX \ hSplitX \ splitY \ hSplitY \ hUnion1 \ 0) = D_1.master$
- theorem $YPseqCode_zero : P_0.X \ (YPseqCode \ P_0 \ P_1 \ hDiff0 \ hDiff1 \ splitX \ hSplitX \ splitY \ hSplitY \ hUnion0 \ 0) = D_0.master$

1. **New generic helper** $atomPairJunk_zero \ (k) : atomPairJunk \ \dots \ 0 \ k = 0$ ($atomPairCodeState$ base clause $stateBase2$ hardcodes $junk \ 0$ regardless of bit-source k ; proved by `unfold atomPairJunk; simp [atomPairCodeState, atomPairBase, stateBase2]`), feeding $atomPairCodeState_correct \ \dots \ 0 \ 0 \ (atomPairJunk_zero \ \dots)$ unconditionally to get $P_0.X \ (atomPairIdx0 \ \dots \ 0 \ 0) = D_0.master / P_1.X \ (atomPairIdx1 \ \dots \ 0 \ 0) = D_1.master$ ($atomPairG$'s own $n = 0$ clause is $defeq \ (D_0.master, D_1.master)$, so $hcs.1/hcs.2$ typecheck directly against those ascribed types with no further unfolding needed).
2. $xPseqAtomJunk \ \dots \ 0 \ 0 = 0$ and $P_1.X \ (xPseqAtomIdx \ \dots \ 0 \ 0) = D_1.master$ exactly by $XPseq_zero$'s own $SplitSpec'$ argument, transcribed line-for-line: $hxSplit$ applied at $A = D_0.master$, $B = D_1.master$, $Xn = D_0.master$ (using $hX0$ to identify Xn with $P_0.X \ 0$) gives the “–”-branch $= \emptyset$ ($Set.diff_self$ on $D_0.master \setminus D_0.master$) forcing the “+”-branch/posIdx — via $IsComputableSplit.posIdx_spec$ — to the full union $D_1.master$; the junk-freeness itself falls out of $emptyInterDec_eq_one_iff$'s contrapositive ($D_0.master \cap D_0.master = D_0.master \neq \emptyset$ by $hDomne$).
3. **New generic helper** $xPseqAtomIdx0_eq$ (the $stateIdx0$ /direct-refine twin of the pre-existing $xPseqAtomIdx_eq$, same proof shape via $xSubStep_idx0_eq$ instead of $xSubStep_idx1_eq$) plus $ComputablePresentation.inter_spec$ (witness $k := atomPairIdx0 \ \dots \ 0 \ 0$ itself, since $P_0.X \ k \subseteq P_0.X \ k \cap P_0.X \ 0$ is an equality once both sides read $D_0.master$) gives the companion fact $P_0.X \ (stateIdx0 \ (xPseqAtomState \ \dots \ 0 \ 0)) = D_0.master$ — the “B”-input Y-side's own split needs.
4. **Key shortcut (avoids rebuilding ySubStep/xSubStep unfolding machinery for the Y-side from scratch):** $yPseqAtomState$'s inner $xSubStep$ call at bit $bx = 1$ is *definitionally* $xPseqAtomState \ (Nat.pair \ n \ (Nat.pair \ 1 \ (packState2 \ (atomPairIdx0 \ n \ i) \ (atomPairIdx1 \ n \ i) \ (atomPairJunk \ n \ i))))$ is literally the same term whether the 1 comes from $xPseqAtomState$'s hardcoded bit or $yPseqAtomState$'s bx substituted with 1) — so $stateJunk/stateIdx1$ of that inner call are rfl -equal to $xPseqAtomJunk/xPseqAtomIdx \ \dots \ 0 \ 0$, letting $ySubStep_junk_eq/ySubStep_idx0_eq$ at $bx = 1$ reuse finding 2/3's already-established facts directly (via `show/type-ascription defeq`, exactly $xPseqAtomIdx_eq$'s own `have h' : ... := h` technique) rather than re-deriving them. The same $SplitSpec'$ argument ($hySplit$ at $A = D_1.master$, $B = D_0.master$, $Xn = D_1.master$) then gives $yPseqAtomJunk \ \dots \ 0 \ 0 \ 1 = 0$ and $P_0.X \ (yPseqAtomIdx \ \dots \ 0 \ 0 \ 1) = D_0.master$.
5. **The $\subseteq D_1.master / \subseteq D_0.master$ half of each equality needs no witness/case-analysis on i at all** (a further simplification over the original plan's “unique bit-source $i = 0$ ” framing): every half-step atom is `mem-genuine` *unconditionally*, regardless of junk status or bit-source ($xPseqAtomIdx_mem/yPseqAtomIdx_mem$, both pre-existing), so $D_1.sub_master/D_0.sub_master$ closes that direction immediately from

`mem_XPseqCode_iff_unconditional/mem_YPseqCode_iff_unconditional`'s raw existential with no need to pin $i = 0$ — only the *reverse* ($\supseteq$) direction needs the explicit $i = 0$ witness with $0 < 4^0$. Both headline theorems assembled via `Set.Subset.antisymm`. Zero sorry; lake build (3165 jobs) and lake env lean `Exercise812d.lean` both clean (no new warnings, no unused-variable/unused-include lint hits). `#print axioms` on both gives $\subseteq \{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$, matching this section's established baseline (e.g. `mem_XPseqCode_iff_unconditional`, `atomPairCodeState_correct` carry the identical footprint) — no new choice introduced.

Exercise 8.12(d)(5)(b): the interleaving/order layer, generalized

- theorem `X_subset_iff_XPseqCode_subset` ($i\ j : \mathbb{N}$) : $P_0.X\ i \subseteq P_0.X\ j \leftrightarrow P_1.X\ (\text{XPseqCode} \dots i) \subseteq P_1.X\ (\text{XPseqCode} \dots j)$
 - theorem `YPseqCode_subset_iff_Y_subset` ($i\ j : \mathbb{N}$) : $P_0.X\ (\text{YPseqCode} \dots i) \subseteq P_0.X\ (\text{YPseqCode} \dots j) \leftrightarrow P_1.X\ i \subseteq P_1.X\ j$
 - theorem `X_inter_eq_iff_XPseqCode_inter_eq` ($i\ j\ k : \mathbb{N}$) : $P_0.X\ i \cap P_0.X\ j = P_0.X\ k \leftrightarrow P_1.X\ (\text{XPseqCode} \dots i) \cap P_1.X\ (\text{XPseqCode} \dots j) = P_1.X\ (\text{XPseqCode} \dots k)$
 - theorem `YPseqCode_inter_eq_iff_Y_inter_eq` ($i\ j\ k : \mathbb{N}$) : $P_0.X\ (\text{YPseqCode} \dots i) \cap P_0.X\ (\text{YPseqCode} \dots j) = P_0.X\ (\text{YPseqCode} \dots k) \leftrightarrow P_1.X\ i \cap P_1.X\ j = P_1.X\ k$
- Design decision (RESOLVED 2026-07-05, after the bounded-search investigation the (d)(5) row asked for): Route 1 is necessary — `X_subset_iff_XPseqCode_subset` etc. compare $P_0.X\ i/P_0.X\ j$, *raw* mutually-unrelated enumeration indices that are never themselves outputs of any `atomPairG` recursion (they are only ever *fed into* it as the X_n/Y_n step argument), so there is no way to transport a subset/inter fact between two arbitrary such indices through `atomPairG_disjoint/atomPairG_invariant/xStepG_snd_union` alone: those facts are all *single-depth* invariants of one fixed recursion run, not a same-family relation between two different indices. Route 2's candidate shortcut does not exist; the full `combinedX/combinedY/genAtom`-interleaving apparatus (generalized) is needed, exactly as the (d)(5) row's finding 2 anticipated. **However, a genuine, non-trivial simplification survives:** `Exercise812c.lean`'s own `xStep_snd_eq_inter_XPseq/yStep fst_eq_inter_YPseq` “I-formula” lemmas (~ 270 lines, needed to seed the interleaved family's odd-depth half-steps) are hard classically only because `XPseq/YPseq` union over the *uncountable* $\delta' : \mathbb{N} \rightarrow \text{Bool} \times \text{Bool}$; at the code level `XPseqCode/YPseqCode` union over *at most* 4^n *literally distinct* bit-sources, and any two distinct bit-sources both $< 4^n$ are *automatically* distinguished somewhere below n (new lemma `exists_deltaPair_ne_of_lt_of_ne`, via `Nat.eq_of_testBit_eq` reading `deltaPair` as two `testBits` per depth) — so the classical proof's entire “does δ' agree with δ through n ?” case split never arises, collapsing ~ 60 lines of case analysis per I-formula to a single disjointness appeal (`atomPairCodeState_disjoint`). Re-scoped into 4 further sub-goals, (b)(i)–(b)(iv), strictly sequential.

Exercise 8.12(d)(5)(b)(iv): the headline transfer theorems Exercise 8.12(d)(5)(c): cross-family order and equality facts

- theorem `X_subset_YPseqCode_iff_XPseqCode_subset_Y` ($i\ j : \mathbb{N}$) : $P_0.X\ i \subseteq P_0.X\ (\text{YPseqCode} \dots j) \leftrightarrow P_1.X\ (\text{XPseqCode} \dots i) \subseteq P_1.X\ j$
 - theorem `YPseqCode_subset_X_iff_Y_subset_XPseqCode` ($i\ j : \mathbb{N}$) : $P_0.X\ (\text{YPseqCode} \dots i) \subseteq P_0.X\ j \leftrightarrow P_1.X\ i \subseteq P_1.X\ (\text{XPseqCode} \dots j)$

- theorem `XPseqCode_eq_Y_iff_X_eq_YPseqCode` ($j\ k : \mathbb{N}$) : $P_1.X\ (XPseqCode\ \dots\ j) = P_1.X\ k \leftrightarrow P_0.X\ j = P_0.X\ (YPseqCode\ \dots\ k)$

Exercise 8.12(d)(5)(c)(i): the cross-parity order fact, even/odd mix

- theorem `X_subset_YPseqCode_iff_XPseqCode_subset_Y` ($i\ j : \mathbb{N}$) : $P_0.X\ i \subseteq P_0.X\ (YPseqCode\ \dots\ j) \leftrightarrow P_1.X\ (XPseqCode\ \dots\ i) \subseteq P_1.X\ j$

Exercise 8.12(d)(5)(c)(ii): the cross-parity order fact, odd/even mix

- theorem `YPseqCode_subset_X_iff_Y_subset_XPseqCode` ($i\ j : \mathbb{N}$) : $P_0.X\ (YPseqCode\ \dots\ i) \subseteq P_0.X\ j \leftrightarrow P_1.X\ i \subseteq P_1.X\ (XPseqCode\ \dots\ j)$

Exercise 8.12(d)(5)(c)(iii): the cross-parity embed_eq_iff analogue

- theorem `XPseqCode_eq_Y_iff_X_eq_YPseqCode` ($j\ k : \mathbb{N}$) : $P_1.X\ (XPseqCode\ \dots\ j) = P_1.X\ k \leftrightarrow P_0.X\ j = P_0.X\ (YPseqCode\ \dots\ k)$

Exercise 8.12(d)(5)(d): toD1Code/toD0Code, the generalized elementwise maps

- def `toD1Code` ($x : D_0.Element$) : $D_1.Element$ where `mem T` := $\exists n, T = P_1.X\ (XPseqCode\ \dots\ n) \wedge x.mem\ (P_0.X\ n)$; ...
 - def `toD0Code` ($y : D_1.Element$) : $D_0.Element$ where `mem S` := $\exists n, S = P_0.X\ (YPseqCode\ \dots\ n) \wedge y.mem\ (P_1.X\ n)$; ...
 - **A second, sharper simplification found (2026-07-05, on close inspection before executing):** `up_mem`’s *own* body needs only *one* `surj` call, not two. `Exercise812c.lean`’s `toD1.up_mem` performs *two* renamings: first `hYcover` `T2` names the arbitrary target `T2` as some `Y k`; then, after transporting `x`’s membership across via the cross-parity order fact, `hXcover` renames the *resulting* set `YPseq k` as some fresh `X j` (needed only because `YPseq k`, a priori just some `D_0.mem` set, is not *literally* a value of the enumeration `X` — genuine search required). At the code level this second search is **not needed**: `YPseqCode`/`XPseqCode` are already $\mathbb{N}$ -valued, so $P_0.X\ (YPseqCode\ \dots\ k)$ (resp. $P_1.X\ (XPseqCode\ \dots\ k)$) is *already*, syntactically, $P_0.X\ j$ for the explicit witness $j := YPseqCode\ \dots\ k$ — no covering search is needed to “find” it, it is handed over for free. The only remaining obligation is the *equation* linking this free index back to the original target index k (i.e. $P_1.X\ k = P_1.X\ (XPseqCode\ \dots\ (YPseqCode\ \dots\ k))$, resp. $P_0.X\ k = P_0.X\ (YPseqCode\ \dots\ (XPseqCode\ \dots\ k))$), and this drops out of (c)(iii)’s `XPseqCode_eq_Y_iff_X_eq_YPseqCode` applied at the *self-referential* pair $(YPseqCode\ \dots\ k, k)$ (resp. $(k, XPseqCode\ \dots\ k)$), whose “other side” is trivially `rfl` ($P_0.X\ (YPseqCode\ \dots\ k) = P_0.X\ (YPseqCode\ \dots\ k)$, resp. $P_1.X\ (XPseqCode\ \dots\ k) = P_1.X\ (XPseqCode\ \dots\ k)$). So `toD1Code`/`toD0Code`’s `up_mem` needs exactly *one* `P_0.surj`/`P_1.surj` call each (to name the arbitrary target `T2`/`S2` as some code-level index in the first place), not two — a genuine simplification over `Exercise812c.lean`’s two-search argument, verified by hand-tracing the full proof term before writing any Lean.
- **Sub-part breakdown:** re-scoped into 4 sub-parts, (d)(5)(d)(i)–(iv) below, mirroring `Exercise812c.lean`’s own `Element`-field structure: the two easy fields (`sub`, `master_mem`) and the `inter_mem`/`up_mem` fields all being one-liners except `up_mem` (the only field needing genuinely chained reasoning), each half (`toD1Code`, `toD0Code`) is split into its own standalone `up_mem` helper lemma — stated at the literal type the `Element.up_mem` field needs, so it plugs directly into the final structure literal — followed by the full assembly `def` (which discharges `sub`/`master_mem`/`inter_mem` inline, since each really is a one-liner not worth its own row).

Exercise 8.12(d)(5)(d)(i): toD1Code's up_mem helper

- theorem toD1Code_up_mem (x : D₀.Element) {T1 T2 : Set β} (h1 : ∃ n, T1 = P₁.X (XPseqCode ... n) ∧ x.mem (P₀.X n)) (hD1T2 : D₁.mem T2) (hT1T2 : T1 ⊆ T2) : ∃ n, T2 = P₁.X (XPseqCode ... n) ∧ x.mem (P₀.X n)

Exercise 8.12(d)(5)(d)(ii): toD1Code, full assembly

- def toD1Code (x : D₀.Element) : D₁.Element where mem T := ∃ n, T = P₁.X (XPseqCode ... n) ∧ x.mem (P₀.X n); ...

Exercise 8.12(d)(5)(d)(iii): toD0Code's up_mem helper

- theorem toD0Code_up_mem (y : D₁.Element) {S1 S2 : Set α} (h1 : ∃ n, S1 = P₀.X (YPseqCode ... n) ∧ y.mem (P₁.X n)) (hD0S2 : D₀.mem S2) (hS1S2 : S1 ⊆ S2) : ∃ n, S2 = P₀.X (YPseqCode ... n) ∧ y.mem (P₁.X n)

Exercise 8.12(d)(5)(d)(iv): toD0Code, full assembly

- def toD0Code (y : D₁.Element) : D₀.Element where mem S := ∃ n, S = P₀.X (YPseqCode ... n) ∧ y.mem (P₁.X n); ...

Exercise 8.12(d)(5)(e): domainIsoCode812d, the generalized order isomorphism

- theorem X_eq_iff_XPseqCode_eq (i j : ℕ) : P₀.X i = P₀.X j ↔ P₁.X (XPseqCode ... i) = P₁.X (XPseqCode ... j) (same-family “embed_eq_iff” companion, needed by map_rel_iff', packaged from (d)(5)(b)’s subset facts via Set.Subset.antisymm in each direction, mirroring X_eq_iff_XPseq_eq)
 - noncomputable def domainIsoCode812d : DomainIso D₀ D₁ where toFun := toD1Code ...; invFun := toD0Code ...; left_inv := ...; right_inv := ...; map_rel_iff' := ...
 - theorem isomorphic_812d : D₀ ≅^D D₁ := ⟨domainIsoCode812d ...⟩
- 1. left_inv/right_inv are genuinely two-directional mutual-inverse content, not one-liners, but the same up_mem-style simplification applies to them, hand-traced below. domainIso812c.left_inv x (Exercise812c.lean lines 2054–2073) needs **two** covering searches per direction (hXcover/hYcover, mirroring toD1.up_mem’s two-search argument) to show toD0 (toD1 x) = x as Elements. At the code level, hand-tracing toD0Code (toD1Code x) = x (via Element.ext, unfolding both mem predicates to ∃ m n, S = P₀.X (YPseqCode ... m) ∧ P₁.X m = P₁.X (XPseqCode ... n) ∧ x.mem (P₀.X n) ↔ x.mem S) shows it collapses to **one** P₀.surj call exactly as toD1Code_up_mem did: given x.mem S, x.sub/P₀.surj names S as some P₀.X n outright; take m := XPseqCode ... n (handed over for free, no search); P₁.X m = P₁.X (XPseqCode ... n) is rfl; and (d)(5)(c)(iii)’s XPseqCode_eq_Y_iff_X_eq_YPseqCode n (XPseqCode ... n) |>.mp rfl : P₀.X n = P₀.X (YPseqCode ... (XPseqCode ... n)) supplies the closing S = P₀.X (YPseqCode ... m) equation for free. The converse direction (consuming an arbitrary witness ⟨m, n, hS, hmn, hxn⟩) is pure bookkeeping: XPseqCode_eq_Y_iff_X_eq_YPseqCode n m transports hmn into P₀.X n = P₀.X (YPseqCode ... m), combines with hS to get S = P₀.X n, and rewrites hxn into x.mem S. right_inv (toD1Code (toD0Code y) = y) is the exact mirror via P₁.surj/YPseqCode ... m := YPseqCode-then-XPseqCode.

2. `map_rel_iff'` needs the same one-search simplification. `domainIso812c.map_rel_iff'` (lines 2094–2108) splits into an easy `mpr` direction ($x \leq x2 \rightarrow \text{toD1 } x \leq \text{toD1 } x2$, pure unfolding, no search) and a harder `mp` direction ($\text{toD1 } x \leq \text{toD1 } x2 \rightarrow x \leq x2$) needing `hXcover` to name an arbitrary `S` with `x.mem S` as some `X n`, then `hle` at `XPseq n` plus `X_eq_iff_XPseq_eq` to transport the resulting index equality back to `S`. At the code level the `hXcover` search is again just `P0.surj`, and `X_eq_iff_XPseqCode_eq` (finding 3 below) plays `X_eq_iff_XPseq_eq`'s role — no other change needed, direct transcription otherwise.
 3. `X_eq_iff_XPseqCode_eq` itself is a clean, short packaging lemma, exactly mirroring `X_eq_iff_XPseq_eq` — `Set.Subset.antisymm` of (d)(5)(b)'s `X_subset_iff_XPseqCode_subset` applied in each direction ($i \ j/j \ i$), no new content.
- **Sub-part breakdown:** re-scoped into 4 sub-parts, (e)(i)–(iv) below, mirroring (d)(5)(d)'s helper-lemma-then-assembly pattern (applied here to `left_inv/right_inv` instead of `up_mem`, one standalone lemma per direction) plus a leading small packaging lemma and a trailing full assembly: (e)(i) the `X_eq_iff_XPseqCode_eq` packaging lemma (finding 3); (e)(ii)/(e)(iii) the `left_inv/right_inv` mutual-inverse content as standalone lemmas (finding 1), one per direction; (e)(iv) the full `domainIsoCode812d/isomorphic_812d` assembly, discharging `toFun/invFun/map_rel_iff'` inline (the latter short enough per finding 2 not to need its own row, same call as (d)(5)(d)(ii)'s `inter_mem`) and citing (e)(ii)/(e)(iii) for `left_inv/right_inv`.

Exercise 8.12(d)(5)(e)(i): `X_eq_iff_XPseqCode_eq`, the same-family `embed_eq_iff` companion

Exercise 8.12(d)(5)(e)(ii): the `left_inv` content, `toD0Code (toD1Code x) = x`

Exercise 8.12(d)(5)(e)(iii): the `right_inv` content, `toD1Code (toD0Code y) = y`

Exercise 8.12(d)(5)(e)(iv): `domainIsoCode812d/isomorphic_812d`, full assembly **Exercise 8.12(d)(5)(f):** computability — the exercise's literal target

- theorem `toD1Code_rel_iff (n m : ℕ) : (ofIso (domainIsoCode812d ...)).rel (P0.X n) (P1.X m) ↔ ∃ k, P1.X m = P1.X (XPseqCode ... k) ∧ P0.X n ⊆ P0.X k` (mirroring `isoInj_rel_iff_incl`)
 - theorem `toD0Code_rel_iff (m n : ℕ) : (ofIso (domainIsoCode812d ...)).symm).rel (P1.X m) (P0.X n) ↔ ∃ k, P0.X n = P0.X (YPseqCode ... k) ∧ P1.X m ⊆ P1.X k` (mirroring `isoProj_rel_iff_incl`)
 - theorem `domainIsoCode812d_isComputableMap : IsComputableMap P0 P1 (ofIso (domainIsoCode812d ...))`
 - theorem `domainIsoCode812d_symm_isComputableMap : IsComputableMap P1 P0 (ofIso (domainIsoCode812d ...).symm)`
- **Re-scoping investigation (2026-07-05),** done before writing any code, hand-tracing the exact unfolding against `Approximable.lean`'s `ofIso/Basic.lean`'s `mem_principal` and `Definition72.lean`'s `comp_isComputable/apply_isComputableElement` (the *actual* r.e.-existential-closure precedent — see finding 2 below, correcting this row's own original proof-note):

1. `toD1Code_rel_iff`'s unfolding confirmed by hand. $(\text{ofIso } e).\text{rel } X \ Y := \exists _ : D_0.\text{mem } X, (e (D_0.\text{principal } \langle D_0.\text{mem } X \rangle)).\text{mem } Y$ (`Approximable.lean`). At $X := P_0.X \ n$, $e := \text{domainIsoCode812d}$, $e (D_0.\text{principal } hXn) = \text{toD1Code } \dots (D_0.\text{principal } hXn)$ (since $\text{domainIsoCode812d.toFun} = \text{toD1Code } \dots$), whose `mem` predicate $((d) (5) (d) (ii))$ is $\text{fun } T \Rightarrow \exists \ k, T = P_1.X \ (\text{XPseqCode } \dots \ k) \wedge (D_0.\text{principal } hXn).\text{mem } (P_0.X \ k)$; `mem_principal` turns the inner membership into $D_0.\text{mem } (P_0.X \ k) \wedge P_0.X \ n \subseteq P_0.X \ k$, and $D_0.\text{mem } (P_0.X \ k)$ is automatically $P_0.\text{mem}_X \ k$ (drop-pable). Net: $(\text{ofIso } \text{domainIsoCode812d}).\text{rel } (P_0.X \ n) \ (P_1.X \ m) \leftrightarrow \exists \ k, P_1.X \ m = P_1.X \ (\text{XPseqCode } \dots \ k) \wedge P_0.X \ n \subseteq P_0.X \ k$, exactly the stated target — confirms the plan's claimed shape is correct, and the proof is a short, direct unfold (no search, no case split), genuinely mirroring `Theorem88n.lean`'s `isoInj_rel_iff_incl/isoProj_rel_iff_incl` proof *shape* (unfold `ofIso`/the map's `mem` field/`mem_principal`, constructor, `rintro`/exact). `toD0Code_rel_iff` is the exact mirror via $\text{domainIsoCode812d.symm.toFun} = \text{domainIsoCode812d.invFun} = \text{toD0Code } \dots$
 2. **Correction to this row's own original proof-note: the computability half does *not* mirror `isoInj_isComputableMap`'s proof shape "line-for-line".** `isoInj_isComputableMap` (`Theorem88n.lean`) has *no* unbounded existential in its target relation (`isoInj_rel_iff_incl`'s RHS is the bare inclusion $P.X \ a \subseteq P.X \ (eIdx \ P \ b)$, since `eIdx` is a deterministic involution supplying the *unique* witness directly) — its proof is a single `RecDecidable.of_iff/.re` step. Our `toD1Code_rel_iff`'s RHS, by contrast, retains a **genuine unbounded** $\exists \ k$ (`XPseqCode` is not known to be surjective/invertible the way `eIdx` is), so the correct precedent is `Definition72.lean`'s `comp_isComputable/apply_isComputableElement`: reindex the two decidable pieces ($P_1.X \ m = P_1.X \ (\text{XPseqCode } \dots \ k) \text{ — } P_1.\text{eq_computable}$ reindexed along the primitive-recursive `XPseqCode` via `RecDecidable.comp/primrec_XPseqCode`, $(d) (4) (c)$; $P_0.X \ n \subseteq P_0.X \ k \text{ — } P_0.\text{incl_computable}$ directly) up to `REPred` via `.re`, combine with `REPred.and`, and close the outer $\exists \ k$ with `REPred.proj`, then `REPred.of_iff` to match `toD1Code_rel_iff`'s exact shape — mirroring `comp_isComputable`'s $(hf'.\text{comp } hgf).\text{and } (hg'.\text{comp } hgg)).\text{proj}$ assembly line-for-line, not `isoInj_isComputableMap`'s single-step `RecDecidable.re`. Symmetric for `domainIsoCode812d_symm_isComputableMap` via `toD0Code_rel_iff/YPseqCode/P_0.eq_computable/P_1.incl_computable`.
 3. **Once this lands, (d) (5) is complete** and (d) (6) (final `EffectiveIso` assembly) becomes close to immediate, per (d) (6)'s own (corrected) proof note.
- **Sub-part breakdown:** re-scoped into 4 sub-parts, (f) (i)–(iv) below, mirroring (d) (5) (d)'s per-direction pairing (one `rel_iff` lemma then one `IsComputableMap` theorem consuming it, done twice — once per direction) rather than (d) (5) (e)'s single dependent chain, since the two directions (`toD1Code`/`toD0Code`) are otherwise fully independent of each other here: (f) (i) `toD1Code_rel_iff`; (f) (ii) `domainIsoCode812d_isComputableMap` (depends on (f) (i)); (f) (iii) `toD0Code_rel_iff` (mirror of (f) (i), independent of (f) (ii)); (f) (iv) `domainIsoCode812d_symm_isComputableMap` (depends on (f) (iii), mirror of (f) (ii)).

Exercise 8.12(d)(5)(f)(i): `toD1Code_rel_iff`

Exercise 8.12(d)(5)(f)(ii): `domainIsoCode812d_isComputableMap`

Exercise 8.12(d)(5)(f)(iii): `toD0Code_rel_iff`

Exercise 8.12(d)(5)(f)(iv): `domainIsoCode812d_symm_isComputableMap` **Exercise 8.12(d)(6)**

Re-scoping investigation (2026-07-05, now that (d)(5) is complete), done before writing any code: 1. Correction to the “further correction” above: `SplitSpec' (hxSplit/hySplit)` is *not* droppable — it is already a load-bearing hypothesis of (d)(5)’s own theorems, contradicting (e)’s finding 1. (e)’s finding 1 is about `AtomPairCode/XPseqCode/`*bare index formulas*, which indeed only cite `IsComputableSplit` directly. But grep-confirmed: `hxSplit/hySplit (SplitSpec' D1 splitX/SplitSpec' D0 splitY)` genuinely appear inside the proof terms of the (b) interleaving layer’s disjointness facts (e.g. `xStepG_disjoint_of_ne hxSplit ...` at `Exercise812d.lean:4150,4162,4600,4631`) — the “different atoms are disjoint” arguments `up_mem/toD1Code/toD0Code/domainIsoCode812d` all transitively depend on. Confirmed independently by Lean’s own `linter.unusedSectionVars`: every (d)(5)(f) theorem’s include `hxSplit ... hySplit ...` list was accepted with **zero** unused-variable warning on `hxSplit/hySplit` specifically (only `hD0nomin/hD1nomin`, in one unrelated pre-existing theorem, are ever flagged) — if `hxSplit/hySplit` were truly dead weight the same linter would have caught it, exactly as it did for `hD0nomin/hD1nomin`. **Net correction: this row’s hypothesis list is simply identical to (d)(5)(f)’s own include list** (`P0 P1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY hD0pos hD0diff hD0nomin hxSplit hD1pos hD1diff hD1nomin hySplit hD0mne hD1mne hUnion0 hUnion1 hX0 hY0`) — **no reduction, no addition, and (contra the previous note) no new “SplitSpec' relative to P₀/P₁” concept is needed; `hxSplit : SplitSpec' D1 splitX/hySplit : SplitSpec' D0 splitY` are literally the plain, already-included hypotheses.** (e)’s own finding 1 stands unaffected for *its* actual question (whether a *concrete* U/V-side `splitX/splitY` can be built satisfying literal `SplitSpec'` — it cannot, for the stated Q.X-valued-output reason) — that is a question about *constructing* a witness, orthogonal to this row’s question of what hypotheses the *abstract* (d)(6) theorem needs to be *handed*. 2. The assembly itself needs no new mathematical content beyond (d)(5)’s already-Pass lemmas — confirmed by direct analogy with `Theorem88n.lean’s isoProj_comp_isoInj/isoInj_comp_isoProj`, which are *generic* facts about `ofIso` of any `OrderIso` and its `.symm`, needing nothing about the specific `iso` beyond its bare `OrderIso` structure. With `e := domainIsoCode812d ...` in hand: `toMap := ofIso e`, `invMap := ofIso e.symm` (`ApproximableMap D0 D1/ApproximableMap D1 D0`, `Approximable.lean’s Theorem 2.7`); `toMap_computable/invMap_computable` are *literally* (d)(5)(f)’s `domainIsoCode812d_isComputableMap/domainIsoCode812d_symm_isComputableMap` (no further work); `left_inv/right_inv` (`ApproximableMap`-level, `invMap.comp toMap = idMap D0/toMap.comp invMap = idMap D1`) are `ext_of_toElementMap` reduced (via `toElementMap_comp`) to `toElementMap_ofIso` twice plus `OrderIso.symm_apply_apply/OrderIso.apply_symm_apply` (a bare `Mathlib OrderIso` fact) plus `toElementMap_idMap` — *exactly* `isoProj_comp_isoInj/isoInj_comp_isoProj`’s own four-line proof shape (`Theorem88n.lean` lines 59–75), substituting `domainIsoCode812d` for `domainIsoCode`. **P. Crucially, this route needs *none* of (d)(5)(e)’s `toD0Code_toD1Code/toD1Code_toD0Code` directly** — those were needed to construct `domainIsoCode812d` itself (as `OrderIso`’s own `left_inv/right_inv` fields), and once `domainIsoCode812d : DomainIso D0 D1` exists as a bona fide `OrderIso`, the `ApproximableMap`-level mutual-inverse facts follow generically from `ofIso/OrderIso.symm_apply_apply` alone, with zero re-derivation. 3. **Once `left_inv/right_inv` and the two `_computable` facts are in hand, `effectiveIso812d : EffectiveIso P0 P1/effectivelyIsomorphic_812d : EffectivelyIsomorphic P0 P1` are one-where-block/one-line packaging**, exactly mirroring (d)(5)(e)(iv)’s `domainIsoCode812d/isomorphic_812d` assembly pattern. * **Sub-part breakdown:** re-scoped into 3 sub-parts, (d)(6)(a)–(c) below, mirroring (d)(5)(e)’s (e)(ii)/(e)(iii)/(e)(iv) pattern (one standalone lemma per mutual-inverse direction, then a final assembly) rather than (d)(5)(f)’s per-direction-pairing (since here there is no analogous `rel_iff/_isComputableMap` split — the two `_computable` fields are direct citations, not new proofs): (d)(6)(a) `invMap_comp_toMap` (the `left_inv` content, mirroring `isoProj_comp_isoInj`); (d)(6)(b) `toMap_comp_invMap` (the `right_inv` content, mirroring `isoInj_comp_isoProj`, independent of (a)); (d)(6)(c) the full `effectiveIso812d/effectivelyIsomorphic_812d` as

sembly (depends on (a)/(b) plus (d)(5)(f)’s `domainIsoCode812d_isComputableMap/domainIsoCode812d_symm_isComputableMap`).

Exercise 8.12(d)(6)(a): `invMap_comp_toMap`, the `left_inv` content

Exercise 8.12(d)(6)(b): `toMap_comp_invMap`, the `right_inv` content

Exercise 8.12(d)(6)(c): `effectiveIso812d/effectivelyIsomorphic_812d`, final assembly

Exercise 8.12(e)

1. **Correction to (d)(6)’s note below (flagged, needs re-verification): the concrete `splitX/splitY` this track needs almost certainly do *not* need to satisfy the classical `SplitSpec'` at all — only `IsComputableSplit`.** Grep-confirmed: the `AtomPairCode/XPseqCode/YPseqCode` sections of `Exercise812d.lean` (which (d)(5)’s `domainIsoCode812d` is built on) take `hSplitX : IsComputableSplit P0 P1 splitX/hSplitY : IsComputableSplit P1 P0 splitY` as their *only* split-related hypotheses — `SplitSpec'/hxSplit/hySplit` never appear in that chain (they belong to a *different*, `SplitSpec'`-based `atomPairG`, used only by (c)’s fully classical, non-effective machinery). Moreover a literal `SplitSpec' V splitX` for a *Q.X-valued* `splitX` would be internally inconsistent: `posIdx_spec/negIdx_spec` force every output to equal some `Q.X k`, and `V.mem/U.mem` both bake in `Set.Nonempty (V.mem X := ∃ k m, X = levelSet k m ∧ X.Nonempty, similarly U.mem)`, so `Q.X k` is *never* literally `∅` — but `SplitSpec'` demands literal `∅` outputs in some genuine cases. So `SplitSpec'` and `IsComputableSplit` cannot both hold of the *same* `Q.X-valued` split function; only `IsComputableSplit` is actually consumed downstream.
2. **Tracing `xSubStep`’s body resolves *why* this is fine:** `newJunk := selectFn junk 1 directEmpty` then both indices are overwritten to the sentinel 0 whenever `newJunk = 1` (`selectFn newJunk 0 directIdx/selectFn newJunk 0 splitIdx`), where `directEmpty` is decided *purely from the prober side* via the already-generic `emptyInterDec/emptyDiffDec` ((d)(2), `Pass`). So the split’s chosen-branch index is only ever *retained* (not junk-masked) exactly when the prober-side direct refinement is already known nonempty — `IsComputableSplit`’s bare index-equation is the only obligation the code layer ever imposes; a “should be `∅`” branch’s *actual* index value is irrelevant once masked.
3. **But a fully context-blind split (ignoring `Xn/A` entirely, e.g. always bisecting `B` by a fixed rule) is ruled out**, not merely unnecessary: `XPseqCode` always forces the “+” sub-bit, so a context-blind split’s `V`-side values would form a single $\subseteq$ -chain (nested bisection depths) for *every* `n` — but `U`’s $\subseteq$ order is not a chain (two disjoint rational intervals are incomparable), so `XPseqCode` could never be order-reflecting against a chain-only image. Genuine `Xn`-dependence, via the prober’s own deciders, is unavoidable.
4. **The one piece of genuinely new combinatorics the exercise originally anticipated — a computable canonical bisection of a single `V`-neighbourhood into two disjoint (hence, since `V.mem → Nonempty`, automatically nonempty) proper `V`-pieces — is exactly `V_no_minimal`’s proof (`Exercise812.lean`), which is *already fully constructive* (refine one level finer via `upsample`, peel off one witnessing bit, its twin guarantees the rest is nonempty).** Mirrors `SplitU.lean`’s existing `splitULeft/splitURight` for `U` almost exactly; just not yet extracted into `Nat.Primrec` form. This — not the `[1/3,2/3]`-density argument the exercise’s original combinatorial framing (now superseded by (c)/(d)’s abstract back-and-forth) anticipated — is the actual new content (e) needs.

5. (f) (below) turns out to need *no* new bisection at all — it reuses `SplitU.lean` (U’s own canonical midpoint bisection, already `Pass` from Theorem 8.8(b)) directly, confirming the old “(f) expected easier” intuition, for a sharper reason than originally guessed (not V’s coarseness, but that U’s own bisection already exists).

- **Sub-part breakdown:** 4 dependent sub-goals, (e)(a)–(d) below, strictly sequential.
Policy note (2026-07-05): every sub-goal below is now stated as a genuine Lean declaration; “Pass” means it typechecks (zero sorry for theorems) — no more prose-only “scoping” rows without a corresponding Lean artifact.

Exercise 8.12(e)(a): the split’s contract, as Lean declarations

- `structure ComputableBisection {W} (Q : ComputablePresentation W) where left right : $\mathbb{N} \rightarrow \mathbb{N}$; left_primrec; right_primrec; disjoint : $\forall k, Q.X \text{ (left } k) \cap Q.X \text{ (right } k) = \emptyset$; union : $\forall k, Q.X \text{ (left } k) \cup Q.X \text{ (right } k) = Q.X \text{ } k$; left_congr : $\forall k k', Q.X \text{ } k = Q.X \text{ } k' \rightarrow Q.X \text{ (left } k) = Q.X \text{ (left } k')$; right_congr : $\forall k k', Q.X \text{ } k = Q.X \text{ } k' \rightarrow Q.X \text{ (right } k) = Q.X \text{ (right } k')$
 - noncomputable def ComputableBisection.posIdxFromBisection (n m k : $\mathbb{N}$) : $\mathbb{N}$:= selectFn (emptyInterDec P (Nat.pair n k)) m (selectFn (emptyDiffDec P hDiff (Nat.pair n k)) m (B.left m))
 - noncomputable def ComputableBisection.negIdxFromBisection (n m k : $\mathbb{N}$) : $\mathbb{N}$:= selectFn (emptyInterDec P (Nat.pair n k)) m (selectFn (emptyDiffDec P hDiff (Nat.pair n k)) m (B.right m))`

Exercise 8.12(e)(b): `SplitV.lean` — a computable canonical bisection for V

- `theorem splitV_disjoint (n :  $\mathbb{N}$ ) :  $VX \text{ (splitVLeft } n) \cap VX \text{ (splitVRight } n) = \emptyset$`
 - `theorem splitV_union (n :  $\mathbb{N}$ ) :  $VX \text{ (splitVLeft } n) \cup VX \text{ (splitVRight } n) = VX \text{ } n$`

(nonemptiness of both pieces is free from `V.mem` $\rightarrow$ `Nonempty`, unlike `SplitU.lean`’s own extra `splitU_left_ne/splitU_right_ne` properness lemmas, which nothing downstream here actually needs).

- **Sub-part breakdown:** 4 sequential sub-goals, (e)(b)(i)–(iv) below, mirroring `SplitU.lean`’s own natural separation (search/arithmetic primitives $\rightarrow$ definitions $\rightarrow$ correctness).

Exercise 8.12(e)(b)(i): `myFirstBit` — the least-set-bit search combinator

- `theorem primrec_myFirstBit : Nat.Primrec (fun t => myFirstBit t.unpair.1 t.unpair.2)`
- `theorem myFirstBit_lt {m N :  $\mathbb{N}$ } (h :  $\exists \ell < N, m.\text{testBit } \ell = \text{true}$ ) : myFirstBit m N < N`
- `theorem myFirstBit_testBit {m N :  $\mathbb{N}$ } (h :  $\exists \ell < N, m.\text{testBit } \ell = \text{true}$ ) : m.testBit (myFirstBit m N) = true`

Exercise 8.12(e)(b)(ii): a computable “clear one bit” / xor-with-power-of-2 primitive

- `theorem primrec_myClearBit : Nat.Primrec (fun t => myClearBit t.unpair.1 t.unpair.2)`
- `theorem myClearBit_eq_xor {m  $\ell$  :  $\mathbb{N}$ } (h : m.testBit  $\ell = \text{true}$ ) : myClearBit m  $\ell = m \text{ } \sim \sim \sim 2 \text{ } ^ \ell$  (or the corresponding direct testBit-level correctness spec, whichever is more convenient downstream)`

Exercise 8.12(e)(b)(iii): splitVLeft/splitVRight definitions and Nat.Primrec

- `def splitVLeft (n : ℕ) : ℕ := Nat.pair (k + 1) (2 ^ ℓ0)`
- `def splitVRight (n : ℕ) : ℕ := Nat.pair (k + 1) (myClearBit (myUpsample k (k+1) m) ℓ0)`
- `theorem primrec_splitVLeft : Nat.Primrec splitVLeft`
- `theorem primrec_splitVRight : Nat.Primrec splitVRight`

Exercise 8.12(e)(b)(iv): correctness — VX_splitVLeft/VX_splitVRight, splitV_disjoint, splitV_union

- `theorem VX_splitVLeft (n : ℕ) : VX (splitVLeft n) = levelSet (k+1) (2^ℓ0)` (i.e. = Y from V_no_minimal's proof, at code-level indices)
- `theorem VX_splitVRight (n : ℕ) : VX (splitVRight n) = levelSet (k+1) (myUpsample k (k+1) m ^ 2^ℓ0)` (i.e. = Z)
- `theorem splitV_disjoint (n : ℕ) : VX (splitVLeft n) ∩ VX (splitVRight n) = ∅`
- `theorem splitV_union (n : ℕ) : VX (splitVLeft n) ∪ VX (splitVRight n) = VX n`

Exercise 8.12(e)(c): generic decider+bisection → IsComputableSplit construction

- `noncomputable def splitFromBisection {P Q} (hpos : _.IsPositive) (hnomin : _ .NoMinimal) (hdiff : IsComputableDiff P) (B : ComputableBisection Q) : Set α → Set γ → Set α → Set γ × Set γ`, defined via `Classical.choose` inversion ($\exists n k m, A = P.X n \wedge Xn = P.X k \wedge B' = Q.X m$) plus (e)(a)'s same case split, landing on `Q.X (posIdxFromBisection ...)/Q.X (negIdxFromBisection ...)` — well-defined as a genuine function of the *sets* `A/Xn/B'` (not just of one arbitrarily-chosen representative index) precisely because `emptyInterDec/emptyDiffDec` are already set-level-invariant (`emptyInterDec_eq_one_iff` et al., (d)(2)) and `ComputableBisection.left_congr/right_congr` ((e)(a)) make `B.left/B.right` set-level-invariant too
 - `theorem isComputableSplit_ofBisection : IsComputableSplit P Q (splitFromBisection hpos hnomin hdiff B)`
- **Sub-part breakdown:** 2 sequential sub-goals, (e)(c)(i)–(ii) below.

Exercise 8.12(e)(c)(i): decider congruence and posIdxFromBisection/negIdxFromBisection well-definedness

- `theorem emptyInterDec_congr (hpos : V.IsPositive) (hnomin : V.NoMinimal) {n n' k k' : ℕ} (hn : P.X n = P.X n') (hk : P.X k = P.X k') : emptyInterDec P (Nat.pair n k) = emptyInterDec P (Nat.pair n' k')`
- `theorem emptyDiffDec_congr (hdiffClosed : V.DiffClosed) (hnomin : V.NoMinimal) {n n' k k' : ℕ} (hn : P.X n = P.X n') (hk : P.X k = P.X k') : emptyDiffDec P hDiff (Nat.pair n k) = emptyDiffDec P hDiff (Nat.pair n' k')` — note the new `hdiffClosed : V.DiffClosed` hypothesis, absent from (e)(c)'s draft signature (see this row's finding 2)
- `theorem posIdxFromBisection_congr (hpos) (hnomin) (hdiffClosed) {n n' k k' m m' : ℕ} (hn : P.X n = P.X n') (hk : P.X k = P.X k') (hm : Q.X m = Q.X m') : Q.X (posIdxFromBisection P hDiff B n m k) = Q.X (posIdxFromBisection P hDiff B n' m' k')`
- `theorem negIdxFromBisection_congr` — same shape, for `negIdxFromBisection`

Exercise 8.12(e)(c)(ii): splitFromBisection and isComputableSplit_ofBisection

- `noncomputable def splitFromBisection {P Q} (hpos) (hnomin) (hdiffClosed : V.DiffClosed) (hdiff : IsComputableDiff P) (B : ComputableBisection Q) : Set  $\alpha \rightarrow$  Set  $\gamma \rightarrow$  Set  $\alpha \rightarrow$  Set  $\gamma \times$  Set  $\gamma$`
- `theorem isComputableSplit_ofBisection : IsComputableSplit P Q (splitFromBisection hpos hnomin hdiffClosed hdiff B)`

Exercise 8.12(e)(d): instantiate for $U \leftrightarrow V$ — splitX812e, the exercise's literal target

- `theorem isComputableSplit_812e : IsComputableSplit UComputablePresentation VComputablePresentation (splitX812e, via isComputableSplit_ofBisection fed U_isPositive/U_noMinimal/U's diff-closedness)`
- **Sub-part breakdown:** 3 sequential sub-goals, (e)(d)(i)–(iii) below.

Exercise 8.12(e)(d)(i): minimal-level canonicalization for V's codes

- `def isPeriodicMask (k j m :  $\mathbb{N}$ ) :  $\mathbb{N}$  ({0,1}-valued) — decides whether levelSet k m is also presentable at level  $j \leq k$ , via the canonical witness myModPow2 m (2 ^ j) (m's own low bits)`
- `def minLevel (k m :  $\mathbb{N}$ ) :  $\mathbb{N}$  — the smallest  $j \leq k$  with isPeriodicMask k j m = 1`
- `def minMask (k m :  $\mathbb{N}$ ) :  $\mathbb{N}$  := myModPow2 m (2 ^ minLevel k m)`
- `theorem primrec_minLevel/primrec_minMask : Nat.Primrec ...`
- `theorem levelSet_minLevel (k m :  $\mathbb{N}$ ) : levelSet (minLevel k m) (minMask k m) = levelSet k m`
- `theorem minLevel_unique {k1 m1 k2 m2 :  $\mathbb{N}$ } (h : levelSet k1 m1 = levelSet k2 m2) : minLevel k1 m1 = minLevel k2 m2  $\wedge$  minMask k1 m1 = minMask k2 m2 — the crux fact that actually delivers left_congr/right_congr in (e)(d)(ii)`

Exercise 8.12(e)(d)(ii): redefine splitVLeft/splitVRight via the minimal presentation; left_congr/right_congr

- redefined `splitVLeft/splitVRight` (same shape as (e)(b)(iii), but built from `(minLevel (canonIdx n).1 (canonIdx n).2, minMask ...)` in place of `(canonIdx n).1, (canonIdx n).2` directly)
- re-derived `VX_splitVLeft/VX_splitVRight/splitV_disjoint/splitV_union` (expected light re-derivations — same shape as (e)(b)(iv), just fed the canonical presentation)
- `theorem splitVLeft_congr (hn : VX n = VX n') : VX (splitVLeft n) = VX (splitVLeft n')` (and `splitVRight_congr`), from (e)(d)(i)'s `minLevel_unique` applied to `canonIdx n/canonIdx n'`

Exercise 8.12(e)(d)(iii): package as ComputableBisection; instantiate for $U \leftrightarrow V$

- `def B812e : ComputableBisection VComputablePresentation (from redefined splitVLeft/splitVRight splitV_disjoint/splitV_union, splitVLeft_congr/splitVRight_congr)`
- `def splitX812e := splitFromBisection UComputablePresentation (U'sIsComputableDiff) B812e`

- theorem isComputableSplit_812e : IsComputableSplit UComputablePresentation VComputablePresentation splitX812e

Exercise 8.12(f)

- UBisection2 : ComputableBisection UComputablePresentation (fixed bisection, left := splitU2Left, right := splitU2Right, left_congr/right_congr now genuinely proved)
 - splitX812f := ComputableBisection.splitFromBisection VComputablePresentation V_isComputableDiff UBisection2
 - isComputableSplit_812f := ComputableBisection.isComputableSplit_ofBisection VComputablePresentation V_isComputableDiff UBisection2 V_isPositive V_noMinimal V_diffClosed

Lean 4 source

theorem hySplit812f : SplitSpec' U splitX812f :=

Exercise 8.12(g)

Part 7 of 7 — assembly: apply (c)/(d)’s general lemma to the concrete U,V using (e)/(f)’s extension-property proofs to conclude $U \cong^D V$ effectively, closing out Exercise 8.12.

Scoping investigation (2026-07-06), then executed in full across three follow-up sessions. The original “no new mathematical content, pure instantiation” guess (like the analogous guess for (f) that turned out wrong twice) was **also wrong here on first pass**: effectiveIso812d (Exercise812d.lean’s actual final-assembly theorem)

Lean 4 source

theorem U_master_nonempty : U.master.Nonempty

Exercise 8.12(g)(1)

The small assembly prerequisites effectiveIso812d needs beyond (c)/(d)/(e)/(f)’s own headline facts: U.master.Nonempty/V.master.Nonempty (hD₀mne/hD₁mne) and UX 0 = U.master/VX 0 = V.master (hX₀/hY₀).

Re-scoped 2026-07-06, verified directly against source (not guesswork) before writing any code. All four facts are confirmed short/mechanical with zero new mathematical content, but they decompose along **two genuinely independent chains, one per system** (U’s rational-interval representation vs.

Exercise 8.12(g)(1)(a)

U-side of (g)(1): U.master.Nonempty (hD₀mne) and UX 0 = U.master (hX₀).

Matched the scoping plan exactly, no surprises. U_master_nonempty : U.master.Nonempty := ⟨0, by norm_num [U]⟩ (needs [U] in the norm_num simp-set to unfold the structure projection U.master down to Set.Ico 0 1 — plain norm_num alone leaves 0 ∈ U.master unsolved, the one real gotcha, found immediately by lake build, not anticipated in the scoping note).

Exercise 8.12(g)(1)(b)

V-side of (g)(1): V.master.Nonempty (hD₁mne) and VX 0 = V.master (hY₀).

Matched the scoping plan exactly, no surprises (unlike (g)(1)(a)’s one norm_num [U] gotcha, this side had none). V_master_nonempty : V.master.Nonempty := Set.univ_nonempty (V.master = Set.univ is definitional, no unfolding lemma needed at all).

Exercise 8.12(g)(2)

IsComputableUnion UComputablePresentation and IsComputableUnion VComputablePresentation (Exercise812d.lean’s (d)(3)(a)-shaped structure for $\cup$ instead of $\setminus$) — never instantiated for either concrete system.

Matched the scoping plan closely — indeed the easiest of the four sub-parts, purely mechanical, no new mathematical content — with three small execution-time gotchas, none a real gap: (1) a naming clash — `Definition87.lean` already has a `U_union_mem` (the $= \emptyset \vee \text{mem-dichotomy}$ lemma for raw `U.mem`, unrelated), so the new unconditional UX-level fact is named `U_union_UX_mem` instead.

Exercise 8.12(g)(3)

1. `splitX812e := ComputableBisection.splitFromBisection UComputablePresentation U_isComputableDiff B812e`. Its two outputs were, by `IsComputableSplit`'s *original*, *unconditional* contract (`posIdx_spec/negIdx_spec`, `Exercise812d.lean (d)(2)`), *always* literally `Q.X (posIdxFromBisection ...) / Q.X (negIdxFromBisection ...)` where `Q := VComputablePresentation`.
2. `Q.X j` is **never** $\emptyset$, for any `j` — `Q.mem_X :  $\forall n, V.mem (Q.X n)$` and `V.mem X :=  $\exists k m, X = \text{levelSet } k m \wedge X.\text{Nonempty}$` (`Exercise812.lean` line 199) *structurally requires* nonempty; the mirror fact holds for `U/UComputablePresentation (U.mem X := ...  $\wedge X.\text{Nonempty} \wedge \dots$ , Definition87.lean line 96).`
3. `posIdxFromBisection/negIdxFromBisection` (`Exercise812e.lean`) *by design* both evaluate to `m` (i.e. literally `B` itself, not $\emptyset$) whenever **either** `emptyInterDec/emptyDiffDec` fires.
4. Concretely: whenever `A  $\cap$  Xn =  $\emptyset$` while `B  $\neq \emptyset$` (routine and unavoidable), `SplitSpec'` demands `(splitX812e A B Xn).1 =  $\emptyset$` , but by (1)–(3) the actual value was `Q.X m = B  $\neq \emptyset$` . A genuine contradiction under the *original* unconditional `posIdx_spec/negIdx_spec`.
5. Not sidesteppable by supplying a *different* already-available split: `effectiveIso812d/domainIsoCode812d` (`Exercise812d.lean (d)(6)`) bind `hxSplit : SplitSpec' D1 splitX` to the **same** `splitX` as `hSplitX : IsComputableSplit P0 P1 splitX`, and the choice-based `splitChoice'` (`Exercise812c.lean`) satisfies `SplitSpec'` but cannot satisfy `IsComputableSplit` (not primitive recursive).
6. **The repair (part 2):** weakened `IsComputableSplit.posIdx_spec/negIdx_spec` from unconditional `(split ...).1 = Q.X (posIdx ...)` to conditional `(split ...).1  $\neq \emptyset \rightarrow$  (split ...).1 = Q.X (posIdx ...)` (symmetrically for `negIdx_spec/.2`) — exactly the fix sketched in part 1's root-cause note. Propagated through `Exercise812d.lean`: `posIdx_mem/negIdx_mem` corollaries gained the same `hne` hypothesis; `xSubStep_correct/ySubStep_correct ((d)(3)(b))` now derive the needed non-emptiness from `SplitSpec'`'s own “empty iff empty” conjunct plus `emptyInterDec_eq_one_iff/emptyDiffDec_eq_one_iff` (needing `atomPairG_invariant`-style `hAB/hBmem` preconditions threaded in from every call site) before invoking `posIdx_spec/negIdx_spec`, rather than getting the equality for free; the same pattern (derive $\neq \emptyset$ from the junk-flag being 0 via `selectFn_one_eq_zero_iff` + the relevant `*Dec_eq_one_iff`, then feed `SplitSpec'`'s `.2.2.1/.2.2.2.1` conjunct) recurs through `atomPairCodeState_correct`, `xPseqAtomIdx_subset_atomPairIdx1`, `xPseqAtomIdx_eq_inter_XPseqCode`, `yPseqAtomIdx_subset_xStepGFst`, `xStepG_snd_eq_inter_XPseqCode`, and `yStepG_fst_eq_inter_YPseqCode`'s closed forms ((d)(5)) — a genuinely mechanical but *wide* ripple (every call site of the three theorems needed the extra hypotheses threaded through explicitly, since Lean's include-based implicit-argument mechanism only auto-supplies section variables *within* the section that declares them, not across section boundaries at a call site). `Exercise812e.lean`'s `isComputableSplit_ofBisection` updated to match (`posIdx_spec/negIdx_spec` fields now take an unused `_hne` parameter).
7. **The concrete construction (part 3, 2026-07-06):** `splitFromBisection` itself **re-designed**. Key realization: `posIdxFromBisection/negIdxFromBisection` (the *index*-level $\mathbb{N}$ -valued functions) needed **no change at all** — they already compute exactly the

right index in the “genuinely nonempty” branch of each field and an unread junk placeholder (m) in the “should be $\emptyset$ ” branch. The bug was purely that `splitFromBisection` (the *set*-level function) wrapped **both** branches in $Q.X (\dots)$, which is never literally $\emptyset$. Fix: route the same two deciders through a three-way `if/else if/else` directly at the `Set  $\gamma \times \text{Set } \gamma$` level — `emptyInterDec` fires $\implies (\emptyset, B')$; else `emptyDiffDec` fires $\implies (B', \emptyset)$; else the genuine bisection ($Q.X (B.\text{left } m)$, $Q.X (B.\text{right } m)$) — proved well-defined (independent of which witness `Classical.choose` picks for the “presented” existential) via a new bridging lemma `splitFromBisection_eq` (built from `emptyInterDec_congr/emptyDiffDec_congr/B.left_congr/B.right_congr`, the same congruence lemmas (e)(c)(i) already supplied). Also generalized the previous “not presented $\rightarrow$ junk (B', B')” fallback (provably wrong in general: $.1 \cap .2 = B' \cap B' = B'$, not $\emptyset$, unless $B' = \emptyset$) to the classical `splitChoice'` (`Exercise812c.lean`) — needed since `SplitSpec'` is stated for **every** $A/Xn : \text{Set } \alpha$, not just presented ones, but only free to change since `IsComputableSplit`’s own `posIdx_spec/negIdx_spec` only ever constrain the presented case. `splitFromBisection` gained one new parameter, `hWnomin` : $W.\text{NoMinimal}$ (needed to build the `splitChoice'` fallback), threaded through `isComputableSplit_ofBisection` and the concrete instantiations (`Exercise812eD.lean`’s `V_noMinimal`, `Exercise812f.lean`’s `U_noMinimal`). The new `splitFromBisection_isSplitSpec'` theorem needs exactly one hypothesis beyond `isComputableSplit_ofBisection`’s own: $hQne : \forall j, Q.X j \neq \emptyset$, used **only** in the “neither decider fires” branch (to show the bisection’s two halves are each individually non-empty, matching $A \cap Xn \neq \emptyset \leftrightarrow .1 \neq \emptyset / A \setminus Xn \neq \emptyset \leftrightarrow .2 \neq \emptyset$) — not derivable from `ComputableBisection`’s abstract fields alone (nothing there rules out $Q.X (B.\text{left } m) = \emptyset$ for an arbitrary W), but immediate for both concrete systems from step 2’s “ $Q.X j$ never $\emptyset$ ” fact (`fun j => (VX_nonempty j).ne_empty` for V , `fun j => (U_mem_UX j).2.1.ne_empty` for U). The two decider-fires branches need no such hypothesis at all — $.1/.2$ there are literally $\emptyset/B'$, and the requisite $\leftrightarrow$ s reduce to hAB via the algebraic facts $A \cap Xn = \emptyset \rightarrow A \setminus Xn = A$ (resp. $A \setminus Xn = \emptyset \rightarrow A \cap Xn = A$). Concrete instances: `hxSplit812e : SplitSpec' V splitX812e` (`Exercise812eD.lean`) and `hySplit812f : SplitSpec' U splitX812f` (`Exercise812f.lean`), each a direct application of `splitFromBisection_isSplitSpec'` fed the relevant `NoMinimal/IsPositive/DiffClosed` facts plus the nonemptiness closure. `lake build (3175 jobs) clean, zero sorry` project-wide; `#print axioms on hxSplit812e/hySplit812f/splitFromBisection_isSplitSpec'` all give $\subseteq \{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$ (`Classical.choice` pre-existing throughout this file’s open `scoped Classical` — not a new leak).

Exercise 8.12(g)(4)

Final assembly — instantiate `Exercise812d.lean`’s `effectiveIso812d` with $D_0 := U, D_1 := V$ and (g)(1)–(3)’s facts, concluding $U\text{ComputablePresentation} \cong^e V\text{ComputablePresentation}$ and closing out Exercise 8.12 in full.

Fully mechanical, exactly as scoped — a single field-by-field instantiation of `effectiveIso812d` (22 positional arguments: `UComputablePresentation VComputablePresentation U_isComputableDiff V_isComputableDiff splitX812e isComputableSplit_812e splitX812f isComputableSplit_812f U_isPositive U_diffClosed U_noMinimal hxSplit812e V_isPositive V_diffClosed V_noMinimal hySplit812f U_master_nonempty V_master_nonempty U_isComputableUnion V_isComputableUnion UX_zero VX_zero`), no new proof content of its own — every subgoal `effectiveIso812d` demands was already discharged by (c)–(g)(3).

Exercise 8.13

Open scoping decision (needs user input before starting code): how far to push the “= Lindenbaum algebra of propositional calculus” identification literally. **Option A (semantic, smaller):** define the Lindenbaum algebra *directly* as (the quotient of) `Formula  $\mathbb{N}$` (a small inductive AST — `var n/ $\top$ / $\perp$ / $\neg$ / $\wedge$ / $\vee$` — over countably many propositional variables) by **semantic** equivalence ($\varphi \sim \psi := \forall v : \mathbb{N} \rightarrow \text{Bool}, \text{eval } v \varphi = \text{eval } v \psi$), justified by

soundness+completeness of classical propositional logic (a standard fact, not reproved) identifying semantic equivalence with provable equivalence — then show $[\varphi] \mapsto \{v : \text{eval } v \varphi = \text{true}\}$ is a Boolean-algebra isomorphism onto Lindenbaum’s underlying algebra. Modest addition (Formula/eval is a short inductive + recursive Bool-eval, no proof calculus needed). **Option B (syntactic, much larger):** build an actual Hilbert-style (or natural-deduction) proof system for propositional logic with $\vdash$, then prove soundness *and* completeness w.r.t. eval, *then* define the Lindenbaum algebra as the genuine syntactic quotient by $\vdash$ -provable equivalence. This is effectively a full propositional-logic metatheory formalization — likely bigger than 8.12 on its own — and Option A already delivers the *same* final algebra (by the completeness theorem Option B would have to prove anyway) with none of the added machinery. **Recommend Option A** unless the user specifically wants the proof-theoretic development as its own exercise. **Topologists’ half — a genuine finding, not yet a plan:** the literal reading “Lindenbaum.Element (proper filters) $\simeq$ non-empty open subsets of $2^{\mathbb{N}}$ ” is **false** as stated, for a structural reason worth recording: every Element (Scott filter) contains V.master/Lindenbaum.master (Element.master_mem is a required field, Basic.lean line 216), so the naive map $x \mapsto \bigcup \{X : x.\text{mem } X\}$ is constant (always the whole space) — filters do **not** correspond to open sets via *union*. Working out the correct correspondence (via the *opposite* direction, $x \mapsto \bigcap \{X : x.\text{mem } X\}$, using that basic neighbourhoods are clopen in a *compact* space): this **does** biject Element with non-empty **closed** sets that are intersections of clopens (all of them, in a zero-dimensional compact space) — order-reversing ($x \leq y \leftrightarrow Z(y) \subseteq Z(x)$), proved via a genuine finite-intersection-property/compactness argument ($Z(x) \subseteq X \rightarrow X \in x$, for X a basic neighbourhood, via extracting a finite subcover of X ’s complement from x ’s “witnessing” family and closing under x ’s existing finite-meet/upward closure — a real two-line-of-math but not-yet-formalized argument). This exactly reflects 8.12’s own aside (“compact = principal”): the **compact/principal** elements are precisely the ones whose $Z(x)$ is clopen (hence open), matching Definition 1.7’s principal $X \mapsto X$ on the nose; *general* (non-compact) elements correspond to closed, not open, sets. Two live options for what to actually formalize: (i) the precise closed-set duality above (mathematically the “real” content, but doesn’t literally match “open subsets” as worded — would need dualizing via complementation, which turns “non-empty” into “proper” and vice-versa, so *still* doesn’t literally match); (ii) a much more modest (and easily achievable) reading: show Lindenbaum’s basic neighbourhoods (cylinderSet) are exactly the clopen basis generating Mathlib’s standard product topology on $\mathbb{N} \rightarrow \text{Bool}$ (Pi.topologicalSpace/discrete Bool factors) — i.e. literally exhibit $\{X \mid \text{Lindenbaum.mem } X\} \cup \{\emptyset\}$ as IsTopologicalBasis for that topology — connecting the algebraic ($\mathcal{D}$) and topological (open sets = arbitrary unions of $\mathcal{D}$) pictures without a false isomorphism claim. **Needs a user decision before any code is written** (see HANDOFF.md’s matching checkpoint / the chat where this was scoped).

Exercise 8.13(a)

V (Exercise812.lean) is the domain of proper filters of the free Boolean algebra on $\aleph_0$ (independent) generators; hence $U \cong^{\mathcal{D}}$ that domain.

Landed **simpler than even the already-optimistic scoping estimate**: no new NeighborhoodSystem was needed at all — V itself (on $\mathbb{N}$, not $\mathbb{N} \rightarrow \text{Bool}$) already *is* the algebra, avoiding every bit of cylinderSet/Cantor-space bookkeeping the scoping row anticipated.

Lean 4 source

```
theorem levelSet_compl (k m : Nat) :
```

Exercise 8.13(b)

Justify the “= Lindenbaum algebra of propositional calculus” identification: a Formula/propositional syntax bridge from 8.13(a)’s concrete $\{X \mid \text{GeneratedBy generator } X\}$ to an actual quotient of propositional formulas.

Implemented “Option A” (semantic-equivalence quotient, no full proof calculus/soundness/-completeness needed). **Formula** (a minimal **var**/**bot**/**top**/**neg**/**and**/**or** AST over $\mathbb{N}$ -indexed variables)

Lean 4 source

```
theorem generatedBy_iff_exists_evalSet {X : Set Nat} :
```

Exercise 8.13(c)

(topologists) Connect 8.13(a)’s filter-domain representation of $\mathcal{U}$ with the non-empty open subsets of Cantor space $2^{\mathbb{N}}$ ($\mathbb{N} \rightarrow \text{Bool}$ with the product topology).

The literal “proper filters $\simeq$ non-empty opens” reading is **false** (every filter contains **master**, so $\bigcup$ -of-filter is constant; filters correspond, order-*reversingly*, to non-empty **closed** sets via $x \mapsto \bigcap \{X : x.\text{mem } X\}$, using compactness). The reading that actually lands on “opens” (matching Scott’s literal word)

Lean 4 source

```
def genPoint (i : Nat) : Set (Nat -> Bool)
```

Exercise 8.13(c1)

Cantor space $2^{\mathbb{N}}$ ’s clopen algebra is exactly **GeneratedBy** **genPoint** (**genPoint** $i := \{x : \mathbb{N} \rightarrow \text{Bool} \mid x\ i = \text{true}\}$), and every open set is a union of such clopens.

genPoint $i := (\text{fun } x \Rightarrow x\ i)^{-1} \{ \text{true} \}$ — literal transcription of 8.13(a)’s generator i onto carrier $\mathbb{N} \rightarrow \text{Bool}$; **isClopen_genPoint** via **isClopen_discrete** $\{ \text{true} \}$ (**Bool**’s **DiscreteTopology**)

Lean 4 source

```
theorem isClopen_genPoint (i : Nat) : IsClopen (genPoint i) :=
  (isClopen_discrete ({true} : Set Bool)).preimage (continuous_apply i)
```

Exercise 8.13(c2)

generator($\mathbb{N}$)/**genPoint**($\mathbb{N} \rightarrow \text{Bool}$) realize the same abstract free Boolean algebra on $\aleph_0$ generators — an explicit order-isomorphism $\{X \mid \text{GeneratedBy } \text{generator } X\} \simeq \{Y \mid \text{GeneratedBy } \text{genPoint } Y\}$, mediated by 8.13(b)’s Lindenbaum quotient (a second evaluation **evalSet'**).

evalSet' : **Formula** $\rightarrow$ **Set** ($\mathbb{N} \rightarrow \text{Bool}$) (8.13(b)’s **Formula**, evaluated via **genPoint** instead of **generator**) — *simpler* than **evalSet**: **mem_evalSet'** **_iff** needs no bit-encoding step ($x : \mathbb{N} \rightarrow \text{Bool}$ already *is* a valuation, contrast **evalSet**’s $n \mapsto \text{fun } i \Rightarrow n.\text{testBit } i$), so **semanticEquiv_iff_evalSet'_eq/entails_iff_evalSet'_subset** need no finitary agreement argument either (both direct **rw**/case-split, no **exists_bitsOf_agree**-style lemma).

Lean 4 source

```
def genPoint (i : Nat) : Set (Nat -> Bool)
```

Exercise 8.13(c3)

For a Boolean subalgebra of sets closed under complement, I is a proper ideal iff $\{Y^c : Y \in I\}$ is a proper filter.

A **g**-generic toolkit (not Cantor-specific — works for any $\{\alpha : \text{Type}^*\}/g : \mathbb{N} \rightarrow \text{Set } \alpha$): **dualFilter** $x := \{Y \mid Y^c \in x\}$ ($Y \mapsto Y^c$ applied to a whole family; **dualFilter_dualFilter** records it as its own inverse).

Lean 4 source

```
theorem generatedBy_of_mem_dualFilter {a : Type*} {g : Nat -> Set a} {x : Set (Set a)}
```

Exercise 8.13(c4)

Assembly: non-empty open $0 \subseteq 2^{\mathbb{N}} \mapsto \text{idealOf } 0 \mapsto (\text{c3}) \text{ dual filter} \mapsto (\text{c2}) \text{ filter of } \mathbb{N}\text{-clopens} = \text{V.Element} \mapsto (8.13a) \text{ element of } U$.

$\text{idealOf } 0 := \{Y \mid \text{GeneratedBy } \text{genPoint } Y \wedge Y \subseteq 0\}$ (the clopens inside 0) is a proper ideal exactly when $0 \neq \text{Set.univ}$ (empty_mem_idealOf , $\text{univ_not_mem_idealOf}$, union_mem_idealOf , down_mem_idealOf); dualizing (8.13(c3))’s toolkit)

Lean 4 source

```
theorem empty_mem_idealOf (0 : Set (Nat -> Bool)) : ({ } : Set (Nat -> Bool)) in idealOf 0
:=
```

Exercise 8.14

closure operators ($I \sqsubseteq a$); fixed-point set finitary; what are the finite elements.

$\text{IsClosureOperator } a := \text{IsRetraction } a \wedge \text{idMap } E \leq a$ ($I_E \sqsubseteq a$, i.e. a inflationary — $\text{le_toElementMap_of_isClosureOperator}$).

Lean 4 source

```
theorem le_toElementMap_of_isClosureOperator (h : IsClosureOperator a) (y : E.Element) :
```

Exercise 8.15

is. (Hint: the finite elements of the domain correspond exactly to the finite systems $X \triangleleft D$.) For $D = \mathcal{U}$, show the computable elements correspond exactly to the effectively presented domains (up to effective isomorphism). modulo the pre-existing UX artifact; 6g deliberately out of scope, documented in-file) For a finite index list $js : \text{List } \mathbb{N}$, the *finitely generated subsystem* $\text{Fin}(js) := \{Y \mid \exists \text{sub} \sqsubseteq js, D.\text{mem } Y \wedge Y = \text{interFrom } P \ D.\text{master } \text{sub}\}$ (reusing `Theorem75.lean`’s interFrom) is itself a subsystem $\text{finGenSys}(js) \triangleleft D$ (closure under consistent intersection: combine two sub-selections by `list-filter`, no bitmask needed at this layer). The **key algebraic fact** (Fin_concat_le_iff): for any $D' \triangleleft D$, $\text{Fin}(js_1 ++ js_2) \subseteq D'.\text{mem} \leftrightarrow \text{Fin}(js_1) \subseteq D'.\text{mem} \wedge \text{Fin}(js_2) \subseteq D'.\text{mem}$ — proved via $\text{interFrom_mem_of_witness}$ (any intermediate partial meet along a consistent chain is itself consistent, using the *full* meet as the witness) + $D'.\text{inter_closed}$. Iterating over singletons gives $\text{Fin_le_iff_forall_mem}$: $\text{Fin}(js) \subseteq D'.\text{mem} \leftrightarrow \forall i \in js, D'.\text{mem } (P.X \ i)$ — the workhorse for everything downstream.

The domain. Token type $\text{List } \mathbb{N}$; $\text{SubD.mem } N := \exists js, N = \text{nbhd}(js)$ where $\text{nbhd}(js) := \{js' \mid \text{Fin}(js) \subseteq \text{Fin}(js')\}$ (anti-monotone principal up-set, mirroring `Exercise222.nbhd`). $\text{nbhd}(js_1) \cap \text{nbhd}(js_2) = \text{nbhd}(js_1 ++ js_2)$ is exactly Fin_concat_le_iff instantiated at $D' := \text{finGenSys}(js')$. $\text{nbhd}(a) = \text{nbhd}(b) \leftrightarrow \text{Fin}(a) = \text{Fin}(b)$ (antisymmetry, $a \in \text{nbhd}(a)$ reflexive).

Lean 4 source

```
theorem master_mem_finGen (js : List Nat) : D.master in finGen P js :=
```

Exercise 8.16

for finitary projections $a, b: E \rightarrow E$, $a \sqsubseteq b \leftrightarrow D_a \triangleleft D_b$ where $D_a = \{X \in E \mid X \sqsubseteq aX\}$; if E is effectively given and $a: E \rightarrow E$ is computable, then D_a is effectively given.

Both halves were already (nearly) proved by existing machinery, so this is a thin assembly.

Lean 4 source

```
theorem isFinitaryProjection_le_iff_fixedNbhd_subsystem {a b : ApproximableMap E E}
(ha : IsFinitaryProjection a) (hb : IsFinitaryProjection b) :
```

Exercise 8.17

find explicit projection pairs for $\mathcal{U} + \mathcal{U}$, $\mathcal{U} \times \mathcal{U}$, $\mathcal{U} \rightarrow \mathcal{U}$ needed for 8.9; are any of these isomorphic to $\mathcal{U}$?; find a universal domain $V \neq \mathcal{U}$.

Part 1 (projection pairs) was already answered by `Definition89.lean`’s `iPlus/jPlus`, `iTimes/jTimes`, `iArrow/jArrow` (via `theorem_8_8_b_strong`); this file restates them under 8.17’s name and adds the free bonus corollary `sumUU_triangleleftteq_U/prodUU_triangleleftteq_U/‘funSpaceUU_triangleleftteq_U : (·)`

Lean 4 source

```
theorem sumUU_triangleleftteq_U : sum U U U_mem_nonempty U_mem_nonempty ? U :=
```

Exercise 8.18

establish the unproved cases of 8.10.

Every case of Proposition 8.10 — all three combinators $+/ \times / \rightarrow$, both halves (projection-preservation and finitary-preservation-with-isomorphism) — was already fully established by `Proposition810.lean` (`isProjection_sumComb/isProjection_prodComb/isProjection_arrowComb/isProjection_combinators`)

Lean 4 source

```
theorem exercise_8_18 {a b : ApproximableMap U U} (ha : IsProjection a) (hb : IsProjection b) :
```

Exercise 8.19

consequences of two known facts.

Reading “we know T ” as “ $T \trianglelefteq E$ ” (the natural companion to $E \rightarrow E \trianglelefteq E$), the answer is **yes**, both $E+E \trianglelefteq E$ and $E \times E \trianglelefteq E$ follow. **Products:** $T \trianglelefteq E$ gives $E \times E \trianglelefteq (T \rightarrow E)$ via pairing-up-as-a- T -indexed-function (`pairToFun/funToPair`, using Exercise 3.26’s `cond`), and $T \trianglelefteq E$ gives ‘ $(T \rightarrow E)$

Lean 4 source

```
theorem expMap_mono_left {V0 : NeighborhoodSystem a} {V1 : NeighborhoodSystem b}
  {V0' : NeighborhoodSystem ?} {V1' : NeighborhoodSystem ?}
  {h h' : ApproximableMap V0' V0} (hh : h <= h') (k : ApproximableMap V1 V1') :
```

Exercise 8.20

$D \trianglelefteq D+D$; what about other constructs?.

$D \trianglelefteq D \times D$ and $D \trianglelefteq (D \rightarrow D)$ both hold **unconditionally, for every domain D** (no hypotheses needed, unlike Exercise 8.19’s $E \times E \trianglelefteq E$), by the simplest possible combinator recipes, formalized slightly generally over two domains A, B : **product** — embed $a \mapsto \langle a, \perp \rangle / b \mapsto \langle \perp, b \rangle$ (paired (`idMap _`) (`constMap _ bot`)), retract with `proj0/proj1` (`triangleleftteq_prod_fst/triangleleftteq_prod_snd`, Proposition 3.2(i)’s `pair_le_pair_iff` reduces the inequality half to $\perp \leq z.snd$); **function space** — embed ‘ $b \mapsto (\lambda _.$

Lean 4 source

```
theorem triangleleftteq_prod_fst : A ? prod A B :=
```

Exercise 8.21

a computable operator $\lambda a. a\&$ on finitary projections, with $D_{\{a\}} \cong (D_a)\&$.

split into **8.21(a)** (construction + computability + half of projection-closure — **Pass**), **8.21(b)** (the other half of projection-closure, idempotence — **Pass**), and **8.21(c)** (the headline isomorphism $D_{\{a\}} \cong (D_a)\&$ — **Pass**). See sub-rows for full detail.

Exercise 8.21(a)

construct $\lambda a. a\&$, show it is computable, and prove $a \leq I \implies a\& \leq I$ (half of “ a a projection $\implies a\&$ a projection”).

The construction of $a\&$ and its tractable properties. $U\& \trianglelefteq U$ (`dsharpU_triangleleftteq_U`): `dsharpPresentation` (Prop 7.7)

Lean 4 source

```
theorem dsharpU_trianglelefteq_U : Dsharp U U_mem_nonempty ? U :=
```

Exercise 8.21(b)

the other half of “ a a projection $\implies a\mathfrak{s}$ a projection” — idempotence $a=a\circ a \implies a\mathfrak{s}=a\mathfrak{s}\circ a\mathfrak{s}$, completing `isProjection_aSharp` (Proposition 8.10’s recipe, transcribed to $\mathfrak{s}$, for the retraction half rather than the $\leq I$ half already done in 8.21(a)).

Unlike $\times/\rightarrow/+$ in `Proposition810.lean`, there is no elementwise closed form for `gMap`’s action on a *general* element of $D\mathfrak{s}$ (its elements can be infinite trees, unlike `pair/fst/snd` or `curry/eval`, which have one-step defining equations valid everywhere), so this needs a genuine catamorphism-uniqueness lemma. All four planned sub-steps closed: ****8.21(b)(1)**

Lean 4 source

```
theorem isProjection_aSharp {a : ApproximableMap U U} (ha : IsProjection a) :
```

Exercise 8.21(c)

the headline isomorphism $D_{\{a\mathfrak{s}\}} \cong (D_a)\mathfrak{s}$ for a a finitary projection of U .

The originally-planned route (via `fixedNbhd`-level transport plus a Theorem-6.14-style initial-algebra uniqueness argument on the sub-tree-algebra cut out by a ’s fixed points) turned out to be avoidable entirely, by noticing $D \mapsto D\mathfrak{s}$ is *functorial on morphisms*, not just on domains — reducing 8.21(c)

Lean 4 source

```
theorem dsharpMap_self_eq_aSharpInner (a : ApproximableMap U U) :
```

Exercise 8.22

which of $B \trianglelefteq C/C \trianglelefteq B$ holds for Example 6.2’s $B \cong B+B, C \cong \mathbb{K}+C+C$; the general $D=T(D)+S(D), E=T(E)$ case; the projections visible in 6.2.

Both directions hold. $B \trianglelefteq C$ (`B_trianglelefteq_C`): B/C share tokens `Str` with $B.\text{mem} \subseteq C.\text{mem}$ (every B -cone is already a C -neighbourhood), so B is a *literal* subsystem $B \triangleleft C$ (Definition 6.10) — `inter_closed` follows from `cone_trichotomy` plus `C_nonempty` ruling out the disjoint case — and `Lemma615.Subsystem.trianglelefteq` hands us $B \trianglelefteq C$ directly; the projection pair is the identity inclusion $i:B\rightarrow C$ and the “collapse completions back onto their cone” retraction $j:C\rightarrow B$, exactly Scott’s hinted projection.

Lean 4 source

```
theorem B_trianglelefteq_C : B ? C :=
```

Exercise 8.23

if $t:(U\rightarrow U)\rightarrow(U\rightarrow U)$ is computable, sends finitary projections a to finitary projections $t(a)$ with $D_{\{t(a)\}} \cong T(D_a)$, does $\|t\|=\text{fix}(t)$ solve $D_{\{\|t\|\}} \cong T(D_{\{\|t\|\}})$ initially w.r.t. projections, effectively?.

Yes to all three, two formalized in full, one in prose. t is modelled as an approximable self-map of the function space $t : \text{ApproximableMap} (\text{funSpace } E \ E) (\text{funSpace } E \ E)$ (general E , not just U), with `tOp t a := toApproxMap (t.toElementMap (toFilter a))` the induced operator on actual maps and `fixOp t := toApproxMap t.fixElement` (Theorem 4.1)

Lean 4 source

```
theorem isFinitaryProjection_fixOp (t : ApproximableMap (funSpace E E) (funSpace E E))
  (ht : forall a, IsFinitaryProjection a -> IsFinitaryProjection (tOp t a)) :
```

Exercise 8.24

binary constructs $S, T \implies$ a pair of effectively presented domains.

The two-variable generalization of Exercise 8.23, reduced to a single instance of Exercise 8.23’s own machinery run on the product function space. Scott’s $s, t : (E \rightarrow E) \times (E \rightarrow E) \rightarrow (E \rightarrow E)$ are modelled as approximable maps $s \ t : \text{ApproximableMap PairSpace (funSpace E E)}$ out of the pair space ‘PairSpace := prod (funSpace E E)

Lean 4 source

```
theorem isFinitaryProjection_aStar_bStar (s t : ApproximableMap (PairSpace (E
  (hst : forall a b, IsFinitaryProjection a -> IsFinitaryProjection b ->
```

Exercise 8.25

a non-trivial domain D ($D \not\cong \mathbb{K}$) with $D \cong D \rightarrow D$; is D effectively given?

Follows Scott’s own hint chain exactly, six steps. (1) the “obvious” solution is trivial (Exercise825Unit.lean): ‘funSpace_unitSys_isomorphic : $(\mathbb{K} \rightarrow \mathbb{K})$

Lean 4 source

```
theorem exercise_8_25 :
```

Exercise 8.26

discuss the “pay-off” for U — untyped λ -calculus translated into U via the end-of-Definition-8.9 equations, and the whole of typed λ -calculus retranslated back into U “with the aid of projections” (hint: $f : D_a \rightarrow D_b$ becomes $f = b \circ f \circ a$; $\lambda x \{D_a\}. \sigma$ becomes $\lambda x. b(\sigma'[a(x)/x])$, checking this “has the right type”).

A genuinely expository exercise (“discuss in more detail”), unlike 8.17–8.25’s crisp existence claims — formalized as the three checkable pieces of Scott’s already-sketched scheme. ******(1)

Lean 4 source

```
theorem translateAbs_sandwich {a b body : ApproximableMap E E}
  (ha : IsRetraction a) (hb : IsRetraction b) :
```

Exercise 8.27

(Suggested by James Donahue.) Define computable *infinite* cartesian products over U . Regard sub (Theorem 8.6) as a finitary projection of U whose fixed points are exactly all the finitary projections; a map $d = \text{sub} \circ d \circ \text{sub}$ is a “polymorphic type” (since t a finitary projection $\implies d(t)$ one too). The *continuous product* of all these types is $\{x \mid \forall t, x(t) = d(t)(x(t))\}$. Define $\Pi = \lambda d. \lambda x. \lambda t. \text{sub}(d(\text{sub}(t)))(x(\text{sub}(t)))$. Show that for d a polymorphic type, $\Pi(d)$ is a type (hint: $\Pi(d)$ is easily a projection; the hard part is showing it is *finitary*). This is the **last exercise in the book** (the text ends immediately after it).

Fully closed, and in fact **strictly stronger than asked**: `isFinitaryProjection_piU` proves $\Pi(d)$ is a type for *every* $d : \mathcal{U} \rightarrow \mathcal{U}$, not only polymorphic d — `IsPolymorphicType/polymorphicType_apply_mem_fix` are formalized (Scott’s own justification for the exercise’s setup) but never needed by the finitary-ness proof itself.

Lean 4 source

```
theorem isFinitaryProjection_piU (d : ApproximableMap U U) : IsFinitaryProjection (piU d) :=
```

Exercise 8.27(a)

“It is easy to check that $\Pi(d)$ is a projection” (Scott’s own hint) — established *unconditionally* for *every* $d : \mathcal{U} \rightarrow \mathcal{U}$, not only polymorphic d ; includes the exercise’s opening sentence (`sub`

regarded as a finitary projection of $\mathcal{U}$ itself, fixed points = exactly the finitary projections) and the construction of Π as a combinator.

(1) *sub* regarded as a combinator on $\mathcal{U}$: $\text{subU} := i_{\rightarrow} \circ \text{subApprox} \circ j_{\rightarrow}$ (conjugating Thm 8.6(b)’s $\text{subApprox} : (\mathcal{U} \rightarrow \mathcal{U}) \rightarrow (\mathcal{U} \rightarrow \mathcal{U})$ through the fixed pair $i_{\rightarrow}, j_{\rightarrow}$ of Def 8.9, the *same* recipe Def 8.9 itself uses for $a \rightarrow b$).

Lean 4 source

```
theorem isProjection_conjArrow {H : ApproximableMap (funSpace U U) (funSpace U U)}
  (hH : IsProjection H) : IsProjection (iArrow.comp (H.comp jArrow)) :=
```

Exercise 8.27(b)

“The problem is to show it is *finitary*” (Scott’s own hint) — for d a polymorphic type, $\Pi(d)$ (equivalently $\text{piU } d$) is finitary, i.e. $\text{IsFinitary } (\text{piU } d)$; combined with 8.27(a)’s isProjection_piU , this completes the exercise’s statement “ $\Pi(d)$ is a type.”

Closed via the strategic pivot described below, **without ever needing a fresh dependent-product NeighborhoodSystem** and, remarkably, **without ever needing d ’s polymorphism**: $\text{isFinitaryProjection_piD}/\text{isFinitaryProjection_piU}$ hold for *every* $d : \mathcal{U} \rightarrow \mathcal{U}$. Also addresses, in `Exercise827.lean`’s module docstring (no separate lemma needed), Scott’s parenthetical “why does this equation mean x is in the product?” — $x(t) = d(t)(x(t))$ unwinds via Ex 8.26’s Uapply to $x(t) \in \text{Fix}(D_{\{d(t)\}})$, i.e.

Lean 4 source

```
theorem isProjection_piU (d : ApproximableMap U U) : IsProjection (piU d) :=
```

Exercise 8.27(b)(0)

$\text{Fix}(\text{piU } d) \simeq_o \text{Fix}(\text{piD } d)$ — reduces “ $\text{piU } d$ is finitary” to “ $\text{piD } d$ is finitary.”

Cheap, mechanical: piUFixIso copies Step 1’s subUFixIso recipe verbatim, substituting $\text{piD } d/\text{piU } d$ for $\text{subApprox}/\text{subU}$ (both built by the identical $i_{\rightarrow} \circ (-) \circ j_{\rightarrow}$ conjugation, so every step transfers unchanged) — $\text{toElementMap_piU}/\text{piD_fix_of_piU_fix}/\text{piU_fix_of_piD_fix}$ mirror $\text{toElementMap_subU}/\text{subApprox_fix_of_subU_fix}/\text{subU_fix_of_subApprox_fix}$ line-for-line.

Exercise 8.27(b)(1)

Reduce “ $\text{piD } d$ is finitary” to Scott’s Theorem 8.5 step-closure formula (ii): $(\text{piD } d).\text{toElementMap } x = \{Y \in (\text{funSpace } U \text{ } U).\text{mem} \mid \exists X, x.\text{mem } X \wedge X \subseteq Y \wedge X(\text{piD } d)X\}$, for every x .

A direct specialization of `Theorem85.lean`’s general $\text{isFinitaryProjection_of_formula}$ ($E := \text{funSpace } U \text{ } U$, $a := \text{piD } d$) — no new proof needed beyond the specialization itself.

Exercise 8.27(b)(2)

Unwind $\text{piD } d = \text{curry } (\text{piDUncurried } d)$ ’s neighbourhood-level relation $X (\text{piD } d) Y$ between $\text{funSpace } U \text{ } U$ -nbhds, down to a formula in terms of subU ’s and d ’s own relations plus evalMap ’s abstract defining relation.

(2a) curry_rel (`FunctionSpace.lean`, pre-existing — the abstract layer, not the Ex 7.16 coded one) already reduces $(\text{piD } d).\text{rel } X \text{ } W$ to a literal set-membership test ‘ $\text{gSection } (\text{piDUncurried } d)$ ’

Exercise 8.27(b)(3)

Prove Scott’s formula (ii) for $\text{piD } d$ (per (b)(1)), using (b)(2)’s .rel formula.

Split into (a) the reduction + the unconditional $\Leftarrow$ half and (b)

Exercise 8.27(b)(3)(a)

Reduce hii ’s LHS to a literal membership test, and close the $\Leftarrow$ half of formula (ii) for $\text{piD } d$ unconditionally (for any d , not just polymorphic).

$\text{piD_toElementMap_mem_iff}$ reduces hii ’s LHS to a literal membership test $\text{piDApply } d (\text{toApproxMap } x) \in Y$, no abstract element-reasoning left, via $\text{Sub8_6.toFilter_toApproxMap}/\text{mem_toFilter}$ + the already-proven $\text{toApproxMap_toElementMap_piD}$.

Exercise 8.27(b)(3)(b)

Close the $\implies$ half of formula (ii) for $\text{piD } d$: given $\text{piDApply } d \text{ (toApproxMap } x) \in Y$, construct X with $x.\text{mem } X$, $X \subseteq Y$, $(\text{piD } d).\text{rel } X \ X$.

This is the actual mathematical content of “the problem” — closed, for every d . The obstruction diagnosed in the prior checkpoint (chasing a *single* domain witness $T_i \supseteq Y_i$ forces the wrong, strictly weaker type-projection when testing X ’s self-relation) is resolved by a compactness-descent argument mirroring `Theorem85.lean`’s own (i) $\implies$ (ii) hard direction: $t := \text{subU}(\uparrow Y_0)$ is *itself* the directed sup of its own `subU`-self-consistent approximants $\text{scPrincipal} : \text{scFamily } t \rightarrow \text{U.Element (eq_iSupDirected_scPrincipal, cofinal by exists_rel_self_subset_of_mem — formula (ii))}$

Exercise 8.27(b)(4)

Assemble (b)(0)–(b)(3) into `IsFinitaryProjection (piU d)` for d polymorphic — the full statement of Exercise 8.27.

`hii_piD` combines (b)(3)(a)/(b)’s two halves into the full formula-(ii) iff for $\text{piD } d$; `isFinitaryProjection_piD` applies (b)(1)’s `isFinitaryProjection_piD_of_formula`; `isFinitaryProjection_piU` combines with (b)(0)’s `isFinitary_piU_of_isFinitary_piD` and (a)’s `isProjection_piU` — Exercise 8.27, in full, for *every* d (polymorphism not required).

Exercise 8.27(b)(5)

Exercise 8.27(b) is closed: `isFinitaryProjection_piU` holds for every d , not only polymorphic d .

4 Acknowledgments

- **Dana Scott** — *Lectures on a Mathematical Theory of Computation* (PRG-19) [Sco81], the monograph this development formalizes in full, including every exercise.

4.1 AI-assisted development

The human author retains sole responsibility for the mathematical content, the choice of formalization route, and every formal claim in this work. Following standard publisher practice (e.g., COPE guidance on authorship and AI tools [COPE24]), **no large language model is listed as a co-author** — authorship implies an accountability that automated systems cannot bear.

We gratefully acknowledge assistance from the following tools (auto-generated from `scripts/ai_model_cards.py` when building `arxiv.tex`):

- **Cursor [Cur26]** — agent-assisted editing in the Cursor IDE across the whole of this development: formalizing Scott’s 1980 neighborhood-system layer in Lean 4 / `mathlib`, `lake` build repair, transcription and upkeep of `sources/PRG19.md`, drafting and maintaining this narrative (`arxiv.md`), and tracking the per-exercise formalized inventory. Generated Lean was provisional until it compiled sorry-free under the pinned toolchain.
- **Cursor Composer [Cmp26]** — unattended multi-hour autorun sessions on Exercise 7.22 (regular-event/automata combinators over the neighborhood system `Ssys`): scaffolding, dependency-ordered module wiring, and routine proof obligations where the strategy was already fixed, under a checklist-driven playbook (`Exercise722-Composer-Playbook.md`) that escalated the hardest sub-goals to a higher-reasoning model rather than looping on Composer alone.
- **Anthropic Claude (Sonnet family) [Son26]** — the primary day-to-day formalization and proof-engineering model throughout this project, used in Cursor at varying reasoning tiers: inventory and narrative maintenance, module wiring, choice-discipline audits (`#print axioms`), and the majority of the Lean proof obligations across Lectures I–VIII,

including this file’s own housekeeping (Exercises 8.25-8.27, the `arxiv.md` “Lean Code” appendix repair, and this LaTeX/PDF pipeline).

- **Anthropic Claude Opus [Ant26]** — selective escalation for the heaviest proof obligations flagged in the Composer playbook as needing higher reasoning depth than routine autorun work (e.g. Exercise 7.22’s full language-equivalence decider). Every emitted proof term was checked by the Lean kernel regardless of which model produced it.

All definitions, constructivity audits, and final prose were reviewed by the human author, who takes full responsibility for them.

4.2 Artifact availability

The development [ER80] is at github.com/catskillsresearch/scott1980. The Palomar-preserved copy linked in the Appendix is <https://github.com/PalomarArchive/catskillsresearch-scott1980-01865c711747/tree/ef6c05671582b7883ecd40b7081b642ce5351963>. Rebuild the formalization and this document with:

```
lake exe cache get
lake build Scott1980
python3 scripts/generate_lecture_mermaid.py --write # sync chapter + per-lecture mermaid
          blocks in arxiv.md
bash scripts/build_arxiv_pdf.sh # narrative + Palomar-link appendix -> arxiv.pdf
bash scripts/build_arxiv_pdf.sh --pdf-only # PDF only when arxiv.tex already current
```

List of Figures

1	Chapter inclusion hierarchy (Lean imports).	4
2	Lean module dependencies for Lecture I: Domains by Neighbourhoods.	7
3	Lean module dependencies for Lecture II: Approximable Mappings.	13
4	Lean module dependencies for Lecture III: Domain Constructs.	18
5	Lean module dependencies for Lecture IV: Fixed Points and Recursion.	23
6	Lean module dependencies for Lecture V: Typed λ -Calculus.	27
7	Lean module dependencies for Lecture VI: Domain Equations.	31
8	Lean module dependencies for Lecture VII: Computability in Effectively Given Domains.	37
9	Lean module dependencies for Lecture VIII: Retracts of the Universal Domain. .	46

5 References

- [Sco69] D. S. Scott. *Lattice-theoretic models for the λ -calculus* (Unpublished manuscript). University of Oxford, 1969.
- [Sco72] D. S. Scott. Continuous lattices. In F. W. Lawvere (Ed.), *Toposes, Algebraic Geometry and Logic* (Lecture Notes in Mathematics, Vol. 274, pp. 97–136). Springer, Berlin, Heidelberg, 1972.
- [Sco81] D. S. Scott. *Lectures on a mathematical theory of computation* (Technical Monograph PRG-19). Oxford University Computing Laboratory, Programming Research Group, May 1981. Lectures delivered Michaelmas Term 1980.
- [Win93] G. Winskel. *The Formal Semantics of Programming Languages*. MIT Press, 1993.

- [ER80] Lars Warren Ericson. *scott1980* (this work). <https://github.com/catskillsresearch/scott1980>.
- [COPE24] Committee on Publication Ethics (COPE). *Authorship and AI tools: COPE position statement*. 2024. <https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools>
- [Cur26] Anysphere, Inc. *Cursor: AI-native code editor and agent environment*. <https://cursor.com> (accessed 2026).
- [Cmp26] Anysphere, Inc. *Composer*. Model announcement and documentation, <https://cursor.com/docs/models> (accessed 2026).
- [Son26] Anthropic. *Claude Sonnet*. System cards and model documentation, <https://www.anthropic.com/claude>; as integrated in Cursor, <https://cursor.com/docs/models> (accessed 2026).
- [Ant26] Anthropic. *Claude Opus*. System cards and model documentation, <https://www.anthropic.com/claude>; as integrated in Cursor, <https://cursor.com/docs/models> (accessed 2026).

A Lean sources in the Palomar archive

The complete Lean sources are not reproduced in this note. Each library file below is the copy preserved by Palomar at commit `ef6c05671582b7883ecd40b7081b642ce5351963`. The archive tree is <https://github.com/PalomarArchive/catskillsresearch-scott1980-01865c711747/tree/ef6c05671582b7883>. The working repository is <https://github.com/catskillsresearch/scott1980>.

A.1 `Scott1980.lean`

Root import module. It loads the `neighbourhood-system` library in `Scott1980.lean` import order so that `import Scott1980` typechecks the complete sorry-free development.

A.2 `Scott1980/Neighborhood/Basic.lean`

Lecture I core: `NeighborhoodSystem`, filters `Element`, principal filters, the approximation order, and the nested-or-disjoint constructor.

A.3 `Scott1980/Neighborhood/Example12.lean`

Example 1.2 of PRG-19, as formalized in this module.

A.4 `Scott1980/Neighborhood/Example13.lean`

Example 1.3 of PRG-19, as formalized in this module.

A.5 `Scott1980/Neighborhood/Example14.lean`

Example 1.4 of PRG-19, as formalized in this module.

A.6 `Scott1980/Neighborhood/Example15.lean`

Example 1.5 of PRG-19, as formalized in this module.

A.7 [Scott1980/Neighborhood/ExampleB.lean](#)

Support module `ExampleB` of the Scott 1981 formalization.

A.8 [Scott1980/Neighborhood/Theorem110.lean](#)

Theorem 1.10 of PRG-19, as formalized in this module.

A.9 [Scott1980/Neighborhood/Theorem111.lean](#)

Theorem 1.11 of PRG-19, as formalized in this module.

A.10 [Scott1980/Neighborhood/Exercise112.lean](#)

Exercise 1.12 of PRG-19, as formalized in this module.

A.11 [Scott1980/Neighborhood/Exercise113.lean](#)

Exercise 1.13 of PRG-19, as formalized in this module.

A.12 [Scott1980/Neighborhood/Exercise114.lean](#)

Exercise 1.14 of PRG-19, as formalized in this module.

A.13 [Scott1980/Neighborhood/Exercise115.lean](#)

Exercise 1.15 of PRG-19, as formalized in this module.

A.14 [Scott1980/Neighborhood/Exercise116.lean](#)

Exercise 1.16 of PRG-19, as formalized in this module.

A.15 [Scott1980/Neighborhood/Exercise117.lean](#)

Exercise 1.17 of PRG-19, as formalized in this module.

A.16 [Scott1980/Neighborhood/Exercise118.lean](#)

Exercise 1.18 of PRG-19, as formalized in this module.

A.17 [Scott1980/Neighborhood/Exercise119.lean](#)

Exercise 1.19 of PRG-19, as formalized in this module.

A.18 [Scott1980/Neighborhood/Exercise120.lean](#)

Exercise 1.20 of PRG-19, as formalized in this module.

A.19 [Scott1980/Neighborhood/Exercise121.lean](#)

Exercise 1.21 of PRG-19, as formalized in this module.

A.20 [Scott1980/Neighborhood/Exercise122.lean](#)

Exercise 1.22 of PRG-19, as formalized in this module.

A.21 [Scott1980/Neighborhood/Exercise123.lean](#)

Exercise 1.23 of PRG-19, as formalized in this module.

A.22 [Scott1980/Neighborhood/Exercise124.lean](#)

Exercise 1.24 of PRG-19, as formalized in this module.

A.23 [Scott1980/Neighborhood/Exercise125.lean](#)

Exercise 1.25 of PRG-19, as formalized in this module.

A.24 [Scott1980/Neighborhood/Exercise126.lean](#)

Exercise 1.26 of PRG-19, as formalized in this module.

A.25 [Scott1980/Neighborhood/Exercise127.lean](#)

Exercise 1.27 of PRG-19, as formalized in this module.

A.26 [Scott1980/Neighborhood/Approximable.lean](#)

Lecture II: approximable maps as relations, `toElementMap`, identity, composition, and maps induced by order isomorphisms.

A.27 [Scott1980/Neighborhood/ApproximableExercises.lean](#)

Lecture II exercises on approximable maps: monotonicity, directed suprema, and the `ofMono / ofContinuous` constructors.

A.28 [Scott1980/Neighborhood/Example23.lean](#)

Example 2.3 of PRG-19, as formalized in this module.

A.29 [Scott1980/Neighborhood/Example24.lean](#)

Example 2.4 of PRG-19, as formalized in this module.

A.30 [Scott1980/Neighborhood/Exercise213.lean](#)

Exercise 2.13 of PRG-19, as formalized in this module.

A.31 [Scott1980/Neighborhood/Exercise214.lean](#)

Exercise 2.14 of PRG-19, as formalized in this module.

A.32 [Scott1980/Neighborhood/Exercise215.lean](#)

Exercise 2.15 of PRG-19, as formalized in this module.

A.33 [Scott1980/Neighborhood/Exercise216.lean](#)

Exercise 2.16 of PRG-19, as formalized in this module.

A.34 [Scott1980/Neighborhood/Exercise218.lean](#)

Exercise 2.18 of PRG-19, as formalized in this module.

A.35 [Scott1980/Neighborhood/Exercise220.lean](#)

Exercise 2.20 of PRG-19, as formalized in this module.

A.36 [Scott1980/Neighborhood/Exercise221.lean](#)

Exercise 2.21 of PRG-19, as formalized in this module.

A.37 [Scott1980/Neighborhood/Exercise222.lean](#)

Exercise 2.22 of PRG-19, as formalized in this module.

A.38 [Scott1980/Neighborhood/Product.lean](#)

Lecture III product of neighbourhood systems, pairing, and the projections of Proposition 3.2.

A.39 [Scott1980/Neighborhood/FunctionSpace.lean](#)

Lecture III function space of approximable maps, `eval/curry`, and the cartesian-closed structure.

A.40 [Scott1980/Neighborhood/Exercise314.lean](#)

Exercise 3.14 of PRG-19, as formalized in this module.

A.41 [Scott1980/Neighborhood/Exercise315.lean](#)

Exercise 3.15 of PRG-19, as formalized in this module.

A.42 [Scott1980/Neighborhood/Exercise318.lean](#)

Exercise 3.18 of PRG-19, as formalized in this module.

A.43 [Scott1980/Neighborhood/Exercise319.lean](#)

Exercise 3.19 of PRG-19, as formalized in this module.

A.44 [Scott1980/Neighborhood/Exercise319Sum.lean](#)

Exercise 3.19Sum of PRG-19, as formalized in this module.

A.45 [Scott1980/Neighborhood/Exercise321.lean](#)

Exercise 3.21 of PRG-19, as formalized in this module.

A.46 [Scott1980/Neighborhood/Exercise322.lean](#)

Exercise 3.22 of PRG-19, as formalized in this module.

A.47 [Scott1980/Neighborhood/Exercise323.lean](#)

Exercise 3.23 of PRG-19, as formalized in this module.

A.48 [Scott1980/Neighborhood/Exercise324.lean](#)

Exercise 3.24 of PRG-19, as formalized in this module.

A.49 [Scott1980/Neighborhood/Exercise316.lean](#)

Exercise 3.16 of PRG-19, as formalized in this module.

A.50 [Scott1980/Neighborhood/Exercise317.lean](#)

Exercise 3.17 of PRG-19, as formalized in this module.

A.51 [Scott1980/Neighborhood/Exercise324Iter.lean](#)

Exercise 3.24Iter of PRG-19, as formalized in this module.

A.52 [Scott1980/Neighborhood/Exercise324Distrib.lean](#)

Exercise 3.24Distrib of PRG-19, as formalized in this module.

A.53 [Scott1980/Neighborhood/Exercise325.lean](#)

Exercise 3.25 of PRG-19, as formalized in this module.

A.54 [Scott1980/Neighborhood/Exercise327.lean](#)

Exercise 3.27 of PRG-19, as formalized in this module.

A.55 [Scott1980/Neighborhood/Exercise326.lean](#)

Exercise 3.26 of PRG-19, as formalized in this module.

A.56 [Scott1980/Neighborhood/Exercise326Sum.lean](#)

Exercise 3.26Sum of PRG-19, as formalized in this module.

A.57 [Scott1980/Neighborhood/Exercise328.lean](#)

Exercise 3.28 of PRG-19, as formalized in this module.

A.58 [Scott1980/Neighborhood/Theorem41.lean](#)

Theorem 4.1 of PRG-19, as formalized in this module.

A.59 [Scott1980/Neighborhood/Example43.lean](#)

Example 4.3 of PRG-19, as formalized in this module.

A.60 [Scott1980/Neighborhood/Example44.lean](#)

Example 4.4 of PRG-19, as formalized in this module.

A.61 [Scott1980/Neighborhood/Theorem46.lean](#)

Theorem 4.6 of PRG-19, as formalized in this module.

A.62 [Scott1980/Neighborhood/Exercise407.lean](#)

Exercise 4.07 of PRG-19, as formalized in this module.

A.63 [Scott1980/Neighborhood/Exercise408.lean](#)

Exercise 4.08 of PRG-19, as formalized in this module.

A.64 [Scott1980/Neighborhood/Exercise409.lean](#)

Exercise 4.09 of PRG-19, as formalized in this module.

A.65 [Scott1980/Neighborhood/Exercise410.lean](#)

Exercise 4.10 of PRG-19, as formalized in this module.

A.66 [Scott1980/Neighborhood/Exercise411.lean](#)

Exercise 4.11 of PRG-19, as formalized in this module.

A.67 [Scott1980/Neighborhood/Exercise412.lean](#)

Exercise 4.12 of PRG-19, as formalized in this module.

A.68 [Scott1980/Neighborhood/Exercise413.lean](#)

Exercise 4.13 of PRG-19, as formalized in this module.

A.69 [Scott1980/Neighborhood/Exercise414.lean](#)

Exercise 4.14 of PRG-19, as formalized in this module.

A.70 [Scott1980/Neighborhood/Exercise415.lean](#)

Exercise 4.15 of PRG-19, as formalized in this module.

A.71 [Scott1980/Neighborhood/Exercise416.lean](#)

Exercise 4.16 of PRG-19, as formalized in this module.

A.72 [Scott1980/Neighborhood/Exercise417.lean](#)

Exercise 4.17 of PRG-19, as formalized in this module.

A.73 [Scott1980/Neighborhood/Exercise418.lean](#)

Exercise 4.18 of PRG-19, as formalized in this module.

A.74 [Scott1980/Neighborhood/Exercise419.lean](#)

Exercise 4.19 of PRG-19, as formalized in this module.

A.75 [Scott1980/Neighborhood/Exercise420.lean](#)

Exercise 4.20 of PRG-19, as formalized in this module.

A.76 [Scott1980/Neighborhood/Exercise421.lean](#)

Exercise 4.21 of PRG-19, as formalized in this module.

A.77 [Scott1980/Neighborhood/Exercise422.lean](#)

Exercise 4.22 of PRG-19, as formalized in this module.

A.78 [Scott1980/Neighborhood/Exercise423.lean](#)

Exercise 4.23 of PRG-19, as formalized in this module.

A.79 [Scott1980/Neighborhood/Exercise424.lean](#)

Exercise 4.24 of PRG-19, as formalized in this module.

A.80 [Scott1980/Neighborhood/Exercise425.lean](#)

Exercise 4.25 of PRG-19, as formalized in this module.

A.81 [Scott1980/Neighborhood/Table55.lean](#)

Lecture V Table 5.5: combinators written in λ -notation.

A.82 [Scott1980/Neighborhood/Theorem51.lean](#)

Theorem 5.1 of PRG-19, as formalized in this module.

A.83 [Scott1980/Neighborhood/Theorem52.lean](#)

Theorem 5.2 of PRG-19, as formalized in this module.

A.84 [Scott1980/Neighborhood/Proposition53.lean](#)

Proposition 5.3 of PRG-19, as formalized in this module.

A.85 [Scott1980/Neighborhood/Proposition54.lean](#)

Proposition 5.4 of PRG-19, as formalized in this module.

A.86 [Scott1980/Neighborhood/Theorem56.lean](#)

Theorem 5.6 of PRG-19, as formalized in this module.

A.87 [Scott1980/Neighborhood/Theorem56Full.lean](#)

Theorem 5.6Full of PRG-19, as formalized in this module.

A.88 [Scott1980/Neighborhood/Exercise507.lean](#)

Exercise 5.07 of PRG-19, as formalized in this module.

A.89 [Scott1980/Neighborhood/Exercise508.lean](#)

Exercise 5.08 of PRG-19, as formalized in this module.

A.90 [Scott1980/Neighborhood/Exercise509.lean](#)

Exercise 5.09 of PRG-19, as formalized in this module.

A.91 [Scott1980/Neighborhood/Exercise510.lean](#)

Exercise 5.10 of PRG-19, as formalized in this module.

A.92 [Scott1980/Neighborhood/Exercise511.lean](#)

Exercise 5.11 of PRG-19, as formalized in this module.

A.93 [Scott1980/Neighborhood/Exercise512.lean](#)

Exercise 5.12 of PRG-19, as formalized in this module.

A.94 [Scott1980/Neighborhood/Exercise513.lean](#)

Exercise 5.13 of PRG-19, as formalized in this module.

A.95 [Scott1980/Neighborhood/Exercise514.lean](#)

Exercise 5.14 of PRG-19, as formalized in this module.

A.96 [Scott1980/Neighborhood/Exercise515.lean](#)

Exercise 5.15 of PRG-19, as formalized in this module.

A.97 [Scott1980/Neighborhood/Exercise516.lean](#)

Exercise 5.16 of PRG-19, as formalized in this module.

A.98 [Scott1980/Neighborhood/Exercise516ThueMorse.lean](#)

Exercise 5.16ThueMorse of PRG-19, as formalized in this module.

A.99 [Scott1980/Neighborhood/Exercise516Overlap.lean](#)

Exercise 5.16Overlap of PRG-19, as formalized in this module.

A.100 [Scott1980/Neighborhood/Example61.lean](#)

Example 6.1 of PRG-19, as formalized in this module.

A.101 [Scott1980/Neighborhood/Example62.lean](#)

Example 6.2 of PRG-19, as formalized in this module.

A.102 [Scott1980/Neighborhood/Example62C.lean](#)

Example 6.2C of PRG-19, as formalized in this module.

A.103 [Scott1980/Neighborhood/Example62A.lean](#)

Example 6.2A of PRG-19, as formalized in this module.

A.104 [Scott1980/Neighborhood/Example62Regular.lean](#)

Example 6.2Regular of PRG-19, as formalized in this module.

A.105 [Scott1980/Neighborhood/Definition63.lean](#)

Definition 6.3 of PRG-19, as formalized in this module.

A.106 [Scott1980/Neighborhood/Proposition66.lean](#)

Proposition 6.6 of PRG-19, as formalized in this module.

A.107 [Scott1980/Neighborhood/Proposition67.lean](#)

Proposition 6.7 of PRG-19, as formalized in this module.

A.108 [Scott1980/Neighborhood/Definition68.lean](#)

Definition 6.8 of PRG-19, as formalized in this module.

A.109 [Scott1980/Neighborhood/Theorem69.lean](#)

Theorem 6.9 of PRG-19, as formalized in this module.

A.110 [Scott1980/Neighborhood/Definition610.lean](#)

Definition 6.10 of PRG-19, as formalized in this module.

A.111 [Scott1980/Neighborhood/Proposition611.lean](#)

Proposition 6.11 of PRG-19, as formalized in this module.

A.112 [Scott1980/Neighborhood/Proposition612.lean](#)

Proposition 6.12 of PRG-19, as formalized in this module.

A.113 [Scott1980/Neighborhood/Definition613.lean](#)

Definition 6.13 of PRG-19, as formalized in this module.

A.114 [Scott1980/Neighborhood/Theorem614.lean](#)

Theorem 6.14 of PRG-19, as formalized in this module.

A.115 [Scott1980/Neighborhood/Lemma615.lean](#)

Lemma 6.15 of PRG-19, as formalized in this module.

A.116 [Scott1980/Neighborhood/Theorem616.lean](#)

Theorem 6.16 of PRG-19, as formalized in this module.

A.117 [Scott1980/Neighborhood/Exercise617.lean](#)

Exercise 6.17 of PRG-19, as formalized in this module.

A.118 [Scott1980/Neighborhood/Exercise617Gen.lean](#)

Exercise 6.17Gen of PRG-19, as formalized in this module.

A.119 [Scott1980/Neighborhood/Exercise618.lean](#)

Exercise 6.18 of PRG-19, as formalized in this module.

A.120 [Scott1980/Neighborhood/Exercise619.lean](#)

Exercise 6.19 of PRG-19, as formalized in this module.

A.121 [Scott1980/Neighborhood/Exercise619PartB.lean](#)

Exercise 6.19PartB of PRG-19, as formalized in this module.

A.122 [Scott1980/Neighborhood/Exercise621.lean](#)

Exercise 6.21 of PRG-19, as formalized in this module.

A.123 [Scott1980/Neighborhood/Exercise622.lean](#)

Exercise 6.22 of PRG-19, as formalized in this module.

A.124 [Scott1980/Neighborhood/Exercise623.lean](#)

Exercise 6.23 of PRG-19, as formalized in this module.

A.125 [Scott1980/Neighborhood/Exercise624.lean](#)

Exercise 6.24 of PRG-19, as formalized in this module.

A.126 [Scott1980/Neighborhood/Exercise625.lean](#)

Exercise 6.25 of PRG-19, as formalized in this module.

A.127 [Scott1980/Neighborhood/Exercise626.lean](#)

Exercise 6.26 of PRG-19, as formalized in this module.

A.128 [Scott1980/Neighborhood/Exercise627.lean](#)

Exercise 6.27 of PRG-19, as formalized in this module.

A.129 [Scott1980/Neighborhood/Exercise628.lean](#)

Exercise 6.28 of PRG-19, as formalized in this module.

A.130 [Scott1980/Neighborhood/Exercise629.lean](#)

Exercise 6.29 of PRG-19, as formalized in this module.

A.131 [Scott1980/Neighborhood/Recursive.lean](#)

Choice-free recursion theory used from Lecture VII: `Nat.Primrec` pairing, `RecDecidable`/`REPred`, and their closure lemmas.

A.132 [Scott1980/Neighborhood/Definition71.lean](#)

Definition 7.1 of PRG-19, as formalized in this module.

A.133 [Scott1980/Neighborhood/Definition72.lean](#)

Definition 7.2 of PRG-19, as formalized in this module.

A.134 [Scott1980/Neighborhood/Theorem74.lean](#)

Theorem 7.4 of PRG-19, as formalized in this module.

A.135 [Scott1980/Neighborhood/Theorem75.lean](#)

Theorem 7.5 of PRG-19, as formalized in this module.

A.136 [Scott1980/Neighborhood/Theorem76.lean](#)

Theorem 7.6 of PRG-19, as formalized in this module.

A.137 [Scott1980/Neighborhood/Proposition77.lean](#)

Proposition 7.7 of PRG-19, as formalized in this module.

A.138 [Scott1980/Neighborhood/Combinators77.lean](#)

Proposition 7.7 combinators: the computable maps used to close effectively given domains under the Lecture III constructs.

A.139 [Scott1980/Neighborhood/Example78.lean](#)

Example 7.8 of PRG-19, as formalized in this module.

A.140 [Scott1980/Neighborhood/Definition79.lean](#)

Definition 7.9 of PRG-19, as formalized in this module.

A.141 [Scott1980/Neighborhood/Proposition710.lean](#)

Proposition 7.10 of PRG-19, as formalized in this module.

A.142 [Scott1980/Neighborhood/Definition711.lean](#)

Definition 7.11 of PRG-19, as formalized in this module.

A.143 [Scott1980/Neighborhood/Proposition712.lean](#)

Proposition 7.12 of PRG-19, as formalized in this module.

A.144 [Scott1980/Neighborhood/Exercise713.lean](#)

Exercise 7.13 of PRG-19, as formalized in this module.

A.145 [Scott1980/Neighborhood/Exercise714.lean](#)

Exercise 7.14 of PRG-19, as formalized in this module.

A.146 [Scott1980/Neighborhood/Exercise715.lean](#)

Exercise 7.15 of PRG-19, as formalized in this module.

A.147 [Scott1980/Neighborhood/Exercise716.lean](#)

Exercise 7.16 of PRG-19, as formalized in this module.

A.148 [Scott1980/Neighborhood/Exercise717.lean](#)

Exercise 7.17 of PRG-19, as formalized in this module.

A.149 [Scott1980/Neighborhood/Exercise717Part2.lean](#)

Support module `Exercise717Part2` of the Scott 1981 formalization.

A.150 [Scott1980/Neighborhood/Exercise718.lean](#)

Exercise 7.18 of PRG-19, as formalized in this module.

A.151 [Scott1980/Neighborhood/Exercise719.lean](#)

Exercise 7.19 of PRG-19, as formalized in this module.

A.152 [Scott1980/Neighborhood/Exercise720.lean](#)

Exercise 7.20 of PRG-19, as formalized in this module.

A.153 [Scott1980/Neighborhood/Exercise721.lean](#)

Exercise 7.21 of PRG-19, as formalized in this module.

A.154 [Scott1980/Neighborhood/Exercise722.lean](#)

Exercise 7.22 algebraic core: `Ssys`, multiplication, and the embedding into streams.

A.155 [Scott1980/Neighborhood/Exercise722Regular.lean](#)

Exercise 7.22 regular events: `SExpr` syntax, denotation, and the regular-event characterization.

A.156 [Scott1980/Neighborhood/Exercise722DFA.lean](#)

Exercise 7.22 leaf DFAs and boolean combinations.

A.157 [Scott1980/Neighborhood/Exercise722Cat.lean](#)

Exercise 7.22 concatenation ε NFA.

A.158 [Scott1980/Neighborhood/Exercise722Decide.lean](#)

Exercise 7.22 emptiness and consistency deciders.

A.159 [Scott1980/Neighborhood/Exercise722Words.lean](#)

Exercise 7.22 bounded word enumeration.

A.160 [Scott1980/Neighborhood/Exercise722Equiv.lean](#)

Exercise 7.22 language-equivalence deciders (`subsetB / interEqB`).

A.161 [Scott1980/Neighborhood/Exercise722Presentation.lean](#)

Exercise 7.22 computable presentation of `Ssys`.

A.162 [Scott1980/Neighborhood/Exercise723.lean](#)

Exercise 7.23: computable operations on the powerset domain `PN`.

A.163 [Scott1980/Neighborhood/Exercise724.lean](#)

Exercise 7.24: the LUCID domain L , the isomorphism $\|L\| \simeq \backslash Gamma$, and computable combinators.

A.164 [Scott1980/Neighborhood/Definition81.lean](#)

Definition 8.1 of PRG-19, as formalized in this module.

A.165 [Scott1980/Neighborhood/Proposition82.lean](#)

Proposition 8.2 of PRG-19, as formalized in this module.

A.166 [Scott1980/Neighborhood/Definition83.lean](#)

Definition 8.3 of PRG-19, as formalized in this module.

A.167 [Scott1980/Neighborhood/Theorem85.lean](#)

Theorem 8.5 of PRG-19, as formalized in this module.

A.168 [Scott1980/Neighborhood/Theorem86.lean](#)

Theorem 8.6 of PRG-19, as formalized in this module.

A.169 [Scott1980/Neighborhood/Theorem86c.lean](#)

Theorem 8.6c of PRG-19, as formalized in this module.

A.170 [Scott1980/Neighborhood/Example84.lean](#)

Example 8.4 of PRG-19, as formalized in this module.

A.171 [Scott1980/Neighborhood/Example84b.lean](#)

Example 8.4b of PRG-19, as formalized in this module.

A.172 [Scott1980/Neighborhood/Definition87.lean](#)

Definition 8.7 of PRG-19, as formalized in this module.

A.173 [Scott1980/Neighborhood/RationalPrimrec.lean](#)

A choice-free Gödel numbering of $\mathbb{Q}$ with primitive-recursive comparison.

A.174 [Scott1980/Neighborhood/RecursiveCross.lean](#)

A choice-free primitive-recursive combination of two coded lists.

A.175 [Scott1980/Neighborhood/IntervalPrimrec.lean](#)

Theorem 8.8(b): primitive-recursive interval-code arithmetic for $\mathcal{U}$.

A.176 [Scott1980/Neighborhood/UComputablePresentation.lean](#)

Theorem 8.8(b): $\mathcal{U}$ is effectively given.

A.177 [Scott1980/Neighborhood/SplitU.lean](#)

Theorem 8.8(b): an explicit deterministic bisection of $\mathcal{U}$.

A.178 [Scott1980/Neighborhood/DAtomDecidable.lean](#)

Theorem 8.8(b): decidable emptiness of D -side atoms.

A.179 [Scott1980/Neighborhood/Theorem88.lean](#)

Theorem 8.8 of PRG-19, as formalized in this module.

A.180 [Scott1980/Neighborhood/Theorem88a.lean](#)

Theorem 8.8a of PRG-19, as formalized in this module.

A.181 [Scott1980/Neighborhood/Theorem88b.lean](#)

Theorem 8.8b of PRG-19, as formalized in this module.

A.182 [Scott1980/Neighborhood/Theorem88c.lean](#)

Theorem 8.8c of PRG-19, as formalized in this module.

A.183 [Scott1980/Neighborhood/Theorem88d.lean](#)

Theorem 8.8d of PRG-19, as formalized in this module.

A.184 [Scott1980/Neighborhood/Theorem88e.lean](#)

Theorem 8.8e of PRG-19, as formalized in this module.

A.185 [Scott1980/Neighborhood/Theorem88f.lean](#)

Theorem 8.8f of PRG-19, as formalized in this module.

A.186 [Scott1980/Neighborhood/Theorem88g.lean](#)

Theorem 8.8g of PRG-19, as formalized in this module.

A.187 [Scott1980/Neighborhood/Theorem88h.lean](#)

Theorem 8.8h of PRG-19, as formalized in this module.

A.188 [Scott1980/Neighborhood/Theorem88i.lean](#)

Theorem 8.8i of PRG-19, as formalized in this module.

A.189 [Scott1980/Neighborhood/Theorem88j.lean](#)

Theorem 8.8j of PRG-19, as formalized in this module.

A.190 [Scott1980/Neighborhood/Theorem88k.lean](#)

Theorem 8.8k of PRG-19, as formalized in this module.

A.191 [Scott1980/Neighborhood/Theorem88l.lean](#)

Theorem 8.8l of PRG-19, as formalized in this module.

A.192 [Scott1980/Neighborhood/Theorem88m.lean](#)

Theorem 8.8m of PRG-19, as formalized in this module.

A.193 [Scott1980/Neighborhood/Theorem88n.lean](#)

Theorem 8.8n of PRG-19, as formalized in this module.

A.194 [Scott1980/Neighborhood/Definition89.lean](#)

Definition 8.9 of PRG-19, as formalized in this module.

A.195 [Scott1980/Neighborhood/Proposition810.lean](#)

Proposition 8.10 of PRG-19, as formalized in this module.

A.196 [Scott1980/Neighborhood/Proposition810b.lean](#)

Proposition 8.10b of PRG-19, as formalized in this module.

A.197 [Scott1980/Neighborhood/Exercise811.lean](#)

Exercise 8.11 of PRG-19, as formalized in this module.

A.198 [Scott1980/Neighborhood/Exercise812.lean](#)

Exercise 8.12 of PRG-19, as formalized in this module.

A.199 [Scott1980/Neighborhood/LevelSetPrimrec.lean](#)

Exercise 8.12(b): a computable presentation of V .

A.200 [Scott1980/Neighborhood/MinLevel.lean](#)

Exercise 8.12: minimal-level canonicalization.

A.201 [Scott1980/Neighborhood/Exercise812c.lean](#)

Exercise 8.12c of PRG-19, as formalized in this module.

A.202 [Scott1980/Neighborhood/Exercise812cYseq.lean](#)

Exercise 8.12cYseq of PRG-19, as formalized in this module.

A.203 [Scott1980/Neighborhood/Exercise812d.lean](#)

Exercise 8.12d of PRG-19, as formalized in this module.

A.204 [Scott1980/Neighborhood/Exercise812e.lean](#)

Exercise 8.12e of PRG-19, as formalized in this module.

A.205 [Scott1980/Neighborhood/Exercise812eD.lean](#)

Exercise 8.12eD of PRG-19, as formalized in this module.

A.206 [Scott1980/Neighborhood/SplitV.lean](#)

Exercise 8.12: a computable canonical bisection of V .

A.207 [Scott1980/Neighborhood/VDiff.lean](#)

Exercise 8.12: a computable V -side set difference.

A.208 [Scott1980/Neighborhood/UBisection2.lean](#)

Exercise 8.12(f): a presentation-independent canonical bisection of $\mathcal{U}$.

A.209 [Scott1980/Neighborhood/Exercise812f.lean](#)

Exercise 8.12f of PRG-19, as formalized in this module.

A.210 [Scott1980/Neighborhood/Exercise812g1a.lean](#)

Support module Exercise812g1a of the Scott 1981 formalization.

A.211 [Scott1980/Neighborhood/Exercise812g1b.lean](#)

Support module Exercise812g1b of the Scott 1981 formalization.

A.212 [Scott1980/Neighborhood/Exercise812g2.lean](#)

Support module Exercise812g2 of the Scott 1981 formalization.

A.213 [Scott1980/Neighborhood/Exercise812g4.lean](#)

Support module Exercise812g4 of the Scott 1981 formalization.

A.214 [Scott1980/Neighborhood/Exercise813a.lean](#)

Exercise 8.13a of PRG-19, as formalized in this module.

A.215 [Scott1980/Neighborhood/Exercise813b.lean](#)

Exercise 8.13b of PRG-19, as formalized in this module.

A.216 [Scott1980/Neighborhood/Exercise813c.lean](#)

Exercise 8.13c of PRG-19, as formalized in this module.

A.217 [Scott1980/Neighborhood/Exercise814.lean](#)

Exercise 8.14 of PRG-19, as formalized in this module.

A.218 [Scott1980/Neighborhood/Exercise815.lean](#)

Exercise 8.15 of PRG-19, as formalized in this module.

A.219 [Scott1980/Neighborhood/Exercise816.lean](#)

Exercise 8.16 of PRG-19, as formalized in this module.

A.220 [Scott1980/Neighborhood/Exercise817.lean](#)

Exercise 8.17 of PRG-19, as formalized in this module.

A.221 [Scott1980/Neighborhood/Exercise818.lean](#)

Exercise 8.18 of PRG-19, as formalized in this module.

A.222 [Scott1980/Neighborhood/Exercise819.lean](#)

Exercise 8.19 of PRG-19, as formalized in this module.

A.223 [Scott1980/Neighborhood/Exercise820.lean](#)

Exercise 8.20 of PRG-19, as formalized in this module.

A.224 [Scott1980/Neighborhood/Exercise821.lean](#)

Exercise 8.21 of PRG-19, as formalized in this module.

A.225 [Scott1980/Neighborhood/Exercise822.lean](#)

Exercise 8.22 of PRG-19, as formalized in this module.

A.226 [Scott1980/Neighborhood/Exercise823.lean](#)

Exercise 8.23 of PRG-19, as formalized in this module.

A.227 [Scott1980/Neighborhood/Exercise824.lean](#)

Exercise 8.24 of PRG-19, as formalized in this module.

A.228 [Scott1980/Neighborhood/Exercise825Unit.lean](#)

Exercise 8.25Unit of PRG-19, as formalized in this module.

A.229 [Scott1980/Neighborhood/Exercise825Congr.lean](#)

Exercise 8.25Congr of PRG-19, as formalized in this module.

A.230 [Scott1980/Neighborhood/Exercise825Closing.lean](#)

Exercise 8.25Closing of PRG-19, as formalized in this module.

A.231 [Scott1980/Neighborhood/Exercise825Pow.lean](#)

Exercise 8.25Pow of PRG-19, as formalized in this module.

A.232 [Scott1980/Neighborhood/Exercise825Embed.lean](#)

Exercise 8.25Embed of PRG-19, as formalized in this module.

A.233 [Scott1980/Neighborhood/Exercise825FixedPoint.lean](#)

Exercise 8.25FixedPoint of PRG-19, as formalized in this module.

A.234 [Scott1980/Neighborhood/Exercise825.lean](#)

Exercise 8.25 of PRG-19, as formalized in this module.

A.235 [Scott1980/Neighborhood/Exercise826.lean](#)

Exercise 8.26 of PRG-19, as formalized in this module.

A.236 [Scott1980/Neighborhood/Exercise827.lean](#)

Exercise 8.27 of PRG-19, as formalized in this module.

A.237 [Scott1980/Neighborhood/PalomarExamples.lean](#)

Palomar corollaries `P4_embeds` and `Ssys_embeds`: Theorem 8.8 specialized to Example 1.5 and Exercise 7.22's system S .

A.238 [Challenge.lean](#)

Palomar statement of record. It locks the first sentence of Theorem 8.8 as `theorem_8_8` together with the new formal corollaries `P4_embeds` and `Ssys_embeds`, using only `Mathlib`.

A.239 [Solution.lean](#)

Palomar realization. It imports `Scott1980.Neighborhood.PalomarExamples` so the compared declarations are the sorry-free library proofs.